

UNIVERSITÀ DEGLI STUDI DI CAMERINO

SCHOOL OF ADVANCED STUDIES

DOCTOR OF PHILOSOPHY IN
COMPUTER SCIENCE & MATHEMATICS
XXXVI CYCLE



Static Analysis of WebAssembly Programs by Abstract Interpretation and Environment Specification

Advisor

Prof. Michele Loreti

PhD Candidate

Mattia Paccamiccio

Co-Advisor

Prof. Franco Raimondi

ACADEMIC YEAR 2025-2026

IMPRINT

*Static Analysis of WebAssembly Programs by Abstract Interpretation and
Environment Specification*

Copyright © 2026 by Mattia Paccamiccio.

All rights reserved. Printed in Italy.

Published by Università degli Studi di Camerino, Camerino, Italy.

Declaration

I herewith declare that I have produced this paper under the supervision of Prof.s Michele Loreti and Franco Raimondi at the University of Camerino, without the prohibited assistance of third parties and without making use of aids, other than those specified. Notions taken over directly or indirectly from other sources have been identified as such. This paper has not previously been presented in an identical or similar form to any other Italian or foreign examination board.

Mattia Paccamiccio
2026

* This dissertation is presented in partial fulfillment of the requirements for **Ph.D. degree** in the School of Advanced Studies of University of Camerino.

*If this took longer than expected,
so did the resistance.*

In this thesis, we aim to design methods for enabling sound static analysis of bytecode-level programs. The target of this work is primarily enabling sound static analysis for WebAssembly (Wasm), with a particular focus on call graph extraction. The call graph of a program plays a crucial role in inter-procedural verification, and greater precision directly improves analysis quality.

To this end, we formalize the standard semantics of a core subset of WebAssembly, the corresponding abstract semantics, and prove the abstract semantics sound w.r.t. the standard semantics. This forms the theoretical base of our analysis by Abstract Interpretation.

Furthermore, we address the challenges posed by the weak encapsulation methods of WebAssembly modules. This translates into unknown dependencies that may significantly impact the semantics of Wasm programs, and, for certain tasks, it is necessary to provide the analysis with this semantic knowledge. To that end, we design a specification language that aids the analyzer when external dependencies are concerned.

Building on top of the abstract semantics of WebAssembly and a specification engine for external dependencies, we validate our approach experimentally using our analyzer, dubbed *wassilly*, the first WebAssembly analyzer, to our knowledge, that enables sound analyses under unknown host dependencies. We observe that *wassilly* surpasses the state of the art in both soundness and precision, demonstrating improved results, thanks to its underlying methodology. We also analyze the limitations of our approach, in particular regarding the cost of manual specification for call graph analysis. This observation motivates strategies for mitigating this burden.

The novel strategy we devised is an analysis targeting JavaScript hosts, the most widely used host language for WebAssembly-based applications. We proceed by identifying WebAssembly-originated allocations and analyzing mutations of such allocations, detecting violations of pre-defined contracts of how hosts can interact with modules. We present an initial formalization of this approach, with intuitions on how such an analysis can reduce the need for manual specification.

Contents

LIST OF PUBLICATIONS	ix
LIST OF LISTINGS	xi
LIST OF FIGURES	xii
LIST OF TABLES	xiii
I BACKGROUND	1
1 INTRODUCTION	3
1.1 Why is WebAssembly Relevant?	3
1.2 The Trilemma of Program Analysis	3
1.3 What is a Call Graph and Why is it Important?	6
1.4 Challenges in Analyzing WebAssembly	8
1.5 Contribution and Outline	9
2 A DEEP DIVE INTO WEBASSEMBLY	11
2.1 What Does a WebAssembly Application Look Like?	12
2.2 Language Basics	13
2.3 Analysis of WebAssembly	17
2.4 Related Work	21
3 STATIC ANALYSIS BY ABSTRACT INTERPRETATION	23
3.1 A First Toy, SlobSinopie	23
3.2 Semantics of SlobSinopie	24
3.3 Introducing Abstractions	27
3.4 An Appetizer on Analyzing SlobSinopie Programs	47
3.5 Analysis of SlobSinopie Programs	48
II IN VITRO	53
4 IN VITRO ANALYSIS OF PWASM	55
4.1 Semantics of WebAssembly Program States	56
4.2 Semantic Domains	62
4.3 Declaration Semantics	63
4.4 Concrete Semantics for Standard WebAssembly	63
4.5 Abstract Semantics for Standard WebAssembly	66
5 PROOF OF SOUND SEMANTICS	75
5.1 Structure of the Soundness Proof	75
5.2 Bounded Abstract Semantics	76
5.3 Soundness of Abstract Bounded Semantics	79
5.4 Termination and Soundness of Fixpoint Semantics	84
6 FROM IN VITRO TO IN VIVO	89

6.1	Direct Impact	89
6.2	Methodologies of Analysis Compared	91
6.3	A First Proposal	92
III IN VIVO		95
7	WASSILLY	97
7.1	Pills on the Design of wassilly	97
7.2	The Specification DSL	102
7.3	Execution Model	106
7.4	Experiments and Considerations	107
7.5	Future Directions on Analysis of WebAssembly Modules . .	111
7.6	<i>isserious</i>	112
IV EPILOGUE		117
8	CONCLUSIONS	119
8.1	Preface	119
8.2	Reflection and Limitations	119
8.3	Summary of Contributions	120
8.4	Closing Thoughts	120
BIBLIOGRAPHY		121

List of Publications

- 2022, Paccamiccio and Mostarda, “**Reasoning About Inter-procedural Security Requirements in IoT Applications**” - Advanced Information Networking and Applications - Proceedings of the 36th International Conference on Advanced Information Networking and Applications (AINA-2022), Volume 3
- 2024, Paccamiccio et al., “**Building call graph of WebAssembly programs via abstract semantics**” - Proceedings of the 8th Italian Conference on Cyber Security (ITASEC 2024), Salerno, Italy, April 8-12, 2024
- 2026, Paccamiccio et al., “**wassilly: Spec-enhanced Call Graph Analysis for WebAssembly**” - Under Revision

List of Listings

2.1	Addition in WebAssembly.	15
2.2	WebAssembly direct function call.	15
2.3	Indirect call in WebAssembly.	15
2.4	Fibonacci's function in WebAssembly.	16
2.5	Block in WebAssembly.	17
2.6	Loop in WebAssembly.	17
2.7	Import in WebAssembly.	17
3.1	BNF grammar for SlobSinopie.	24
3.2	Fibonacci function in SlobSinopie.	24
3.3	Example of a potentially diverging program.	28
3.4	Example program that may yield an error.	48
3.5	Sketch of a big-step abstract interpreter.	49
3.1	Fixpoint algorithm iterating on the SCG.	50
6.1	Direct memory manipulation.	89
6.2	Direct table manipulation.	90
6.3	Direct global manipulation.	90
6.4	Behavior injection.	90
7.1	Symbolic-enhanced operands.	101
7.2	Pre-assignment substitution algorithm.	102
7.3	Syntax of the specification DSL.	103
7.4	Loose Fibonacci function.	104
7.5	Better Fibonacci function.	104
7.6	'Best' Fibonacci function.	104
7.7	Injection in WebAssembly.	105
7.8	Injection in <i>wassilly</i>	105
7.9	Host modifies reference table.	105
7.10	<code>preinv</code> in action.	105
7.11	Specifications of WASI functions.	106

List of Figures

1.1	The trilemma of program analysis.	4
2.1	Emscripten compiling C to Wasm.	12
2.2	Portability comparison: native vs WebAssembly.	12
2.3	Layout of a web application using Wasm.	13
3.1	Hasse diagram for the Interval abstract domain.	31
3.2	Hasse diagram for the abstract boolean domain.	33
3.3	Visualization of two disjoint integer intervals on a number line.	37
3.4	Trace of the analysis of a division by zero.	48
3.5	Trace of the abstract interpreter analyzing <code>fib</code>	51
4.1	Syntax of pWasm	55
7.1	Component architecture of <i>wassilly</i>	98

List of Tables

7.1	Benchmark programs under analysis.	109
7.2	Comparison of all tools for Wasm call graph construction. . .	110

Part I

BACKGROUND

1

Introduction

1.1 WHY IS WEBASSEMBLY RELEVANT?

WebAssembly [Haa+17] (Wasm) has emerged as a transformative technology. Aiming and succeeding in providing a portable and near-native performance execution environment that bridges the gap between native applications and web-based deployment. Designed as a low-level compilation target for languages like C, C++, and Rust, Wasm enables efficient execution across different platforms by leveraging an assembly-like binary format. Its adoption by all major browsers and compatibility with the LLVM toolchain have made it a ubiquitous runtime, akin to the JVM for Java. However, this widespread adoption also makes Wasm an attractive vector for malicious software [Mus+19]. Not only can Wasm offer a practically ubiquitous runtime system, but it also offers near-native performance and automatic client-side execution, making unsophisticated ‘cryptomining’ attacks particularly attractive [Mus+19; Tek+21]. More in general, the rapid proliferation of Wasm binaries has spurred significant interest in both the offensive and defensive sides of security, particularly in researching established and emerging security threats and how to defend against them [LKP20; Mus+19; Oz+25], developing analysis tools to ensure security and discover vulnerabilities [Bri+22; He+24]. Due to Wasm’s unique language and execution environment features, more ‘traditional’ frameworks often struggle to address these features with formal guarantees, leaving gaps in the analysis capabilities of existing frameworks.

1.2 THE TRILEMMA OF PROGRAM ANALYSIS

The field of program analysis and verification has to live with the fact that it is impossible to automatically demonstrate any non-trivial property of programs without resorting to approximation. As Henry Gordon Rice proved, “all non-trivial semantic properties of programs are undecidable” [Ric53]. Briefly put, extraction or proving of properties cannot be done by another program that will never fail to provide in finite time the correct result for any given program. This theorem generalized Alan Turing’s theorem on the undecidability of the halting problem [Tur36].

“Everyone, get in here!”
—Grim Patron, Hearthstone.

[Haa+17] Haas et al., “Bringing the web up to speed with WebAssembly”

[Mus+19] Musch et al., “New Kid on the Web: A Study on the Prevalence of WebAssembly in the Wild”

[Tek+21] Tekiner et al., “SoK: Cryptojacking Malware”

[LKP20] Lehmann et al., “Everything Old is New Again: Binary Security of WebAssembly”

[Oz+25] Oz et al., “Ransomware Over Modern Web Browsers: A Novel Strain and a New Defense Mechanism”

[Bri+22] Brito et al., “Wasmati: An efficient static vulnerability scanner for WebAssembly”

[He+24] He et al., “SeeWasm: An Efficient and Fully-Functional Symbolic Execution Engine for WebAssembly Binaries”

[Ric53] Rice, “Classes of Recursively Enumerable Sets and Their Decision Problems”

[Tur36] Turing, “On Computable Numbers, with an Application to the Entscheidungsproblem”

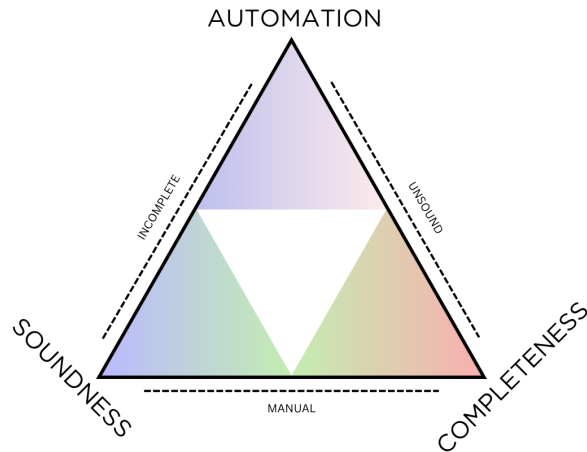


FIGURE 1.1: The trilemma of program analysis.

The ideal analyzer would always automatically provide correct analysis results in finite time. This analyzer existed; it would be:

- (i) **complete**, providing no false positives;
- (ii) **sound**, providing no false negatives;
- (iii) **automatic**, with no user interaction required;

Given that the problem is proven impossible to solve, researchers elaborated ways of circumventing this impossibility by giving up on the **guarantee** of at least one between completeness, soundness, and automatism. This is represented graphically in [Figure 1.1](#).

As can be expected of something born of compromise, each approach has its unique pros and cons, and there really is not a one-size-fits-all solution for all problems when it comes to program analysis and verification. While referring to [Figure 1.1](#), the “best” we can obtain are methodologies of analyzing programs that are: (i) sound and complete requiring manual intervention, (ii) sound, incomplete and automatic, or methodologies that are (iii) complete, unsound and automatic. “Best” is in quotes because there might be certain instances of analyzers that are capable of leveraging fully automated methodologies in which the manual aid of an expert is desirable.

Keep in mind that this formulation of the compromise completely ignores the aspect of the tractability of the problem. This relates more to automatic approaches, but simply put, to be closer to completeness in an automatic and sound approach is more computationally expensive than being farther from completeness. Conversely, this is true as well for soundness in automatic and complete approaches.

The various compromises that had to be taken by scientists gave birth to different methodologies grounded in formal methods, which will be briefly introduced in their most canonical forms because, as already stated, analysis methodologies can be hybridized for many reasons, some of which can be improving precision and/or performance.

GIVING UP COMPLETENESS Incomplete approaches encompass the most canonical forms of static analysis. Incomplete sound approaches allow to observe all possible program executions, terminating and non-

terminating, with spurious false positives. Instances of this type of analysis are the ones largely implemented by optimizing compilers, such as dead code analysis and constant propagation. This form of analysis is usually employed when it is desirable to have conservative analysis results (i.e. when performing dead code elimination, we want to be sure we are eliminating only dead code) and, possibly, fast analysis results. More sophisticated approaches allow for the demonstration of stronger properties, such as the absence of runtime errors in critical software. For instance, industrial-grade analyzers such as Astrée [Min13; Che20], Frama-C [BBY17; Büh17] have been successfully used to analyze and verify code running on critical infrastructures, such as software running on airplanes and nuclear power plants. Since analyzers require a high degree of precision, the language to be analyzed, C, is willingly restricted in its usage. For instance, it is forbidden to use jump instructions and dynamic memory allocation. In general, when verification of code is a necessity, it is desirable to have code that, in a sense, is analyzer-friendly. This is because precise, sound, and efficient analysis of sophisticated programming features still remains a challenge.

GIVING UP SOUNDNESS Unsound methodologies prioritize eliminating false positives at the expense of potentially missing true program defects (false negatives). These approaches are **complete**, any reported issue is guaranteed to be genuine, and **automatic**, but sacrifice soundness by under-approximating program behavior. Such analyses are often employed in scenarios where actionable results are critical, and manual inspection of false alarms is impractical. For example, bug-finding tools like bounded model checkers (e.g., CBMC [KT03]) or symbolic execution engines (e.g., KLEE [CDE08], angr [Sho+16], Manticore [Mos+19]) may be used to explore a subset of program paths within computational bounds, guaranteeing precision for detected issues but unable to prove universal correctness. Similarly, lightweight static analyzers targeting specific vulnerabilities (e.g., memory leaks) may use heuristic rules to prioritize high-confidence findings over exhaustive verification. The primary advantage of (lightweight) unsound approaches is in their practicality: they deliver precise results efficiently, making them suitable for integration into development workflows (e.g., CI/CD pipelines). However, their inability to guarantee full coverage limits their use in safety-critical domains. Verifiers based on symbolic execution (e.g., SeaHorn [GKN15]) or concolic testers (e.g., DART [GKS05]), studied in depth in [Bal+18], exemplify this trade-off, balancing automation and precision while accepting incompleteness, as such approaches suffer from the state explosion problem. Regardless of this fundamental flaw, researchers found applications to unsound techniques in symbolically directed fuzzing [BPR16] and machine learning-guided heuristics [LZK22], with the aim of maximizing error detection, addressing scalability by ‘localizing’ the area the analyzer has to reason about. In general, the other approach that unsound methodologies encourage is simplifying the problem by reducing a program to a finite state machine (or constraint the language such that it can be translated into an FSM), and verify it by model-checking or

[Min13] Miné, *Static analysis by abstract interpretation of concurrent programs. (Analyse statique par interprétation abstraite de programmes concurrents)*

[Che20] Chevalier, “Proving the security of software-intensive embedded systems by abstract interpretation. (Analyse de la sécurité de systèmes critiques embarqués à forte composante logicielle par interprétation abstraite)”

[BBY17] Blazy et al., “Structuring Abstract Interpreters Through State and Value Abstractions”

[Büh17] Bühler, “Structuring an Abstract Interpreter through Value and State Abstractions: EVA, an Evolved Value Analysis for Frama-C. (Structurer un interpréteur abstrait au moyen d’abstractions de valeurs et d’états :Eva, une analyse de valeur évoluée pour Frama-C)”

[KT03] Kroening and Tautschnig, “CBMC: C Bounded Model Checker”

[CDE08] Cadar et al., “KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs”

[Sho+16] Shoshitaishvili et al., “SoK: (State of) The Art of War: Offensive Techniques in Binary Analysis”

[Mos+19] Mossberg et al., “Manticore: A User-Friendly Symbolic Execution Framework for Binaries and Smart Contracts”

[GKN15] Gurfinkel et al., “SeaHorn: A Framework for Verifying C Programs”

[GKS05] Godefroid et al., “DART: Directed Automated Random Testing”

[Bal+18] Baldoni et al., “A Survey of Symbolic Execution Techniques”

[BPR16] Böhme et al., “Coverage-based Greybox Fuzzing as Markov Chain”

[LZK22] Liang et al., “Machine Learning-Guided Static Analysis for Bug Detection”

symbolic execution.

GIVING UP AUTOMATION This approach is usually undertaken in the field of deductive verification, and it potentially allows to observe all program executions without approximation. The tools produced with this set of methodologies require manual intervention and a lot of expertise on the user side. Approaches that are this complex are valid when trying to prove properties that would not be realistically provable with the aforementioned methodologies. An example of these properties would be semantic equivalence between implementation and specification. Examples of such an approach can be found in theorem-proving tools, such as Lean [Mou+15], Coq [GL02], Isabelle/HOL [Pau93], Agda [Nor08]. Other tools, such as Why3 [Bob+11], and Dafny [Lei10] require a program and a property to verify that the tool attempts to prove automatically using user-provided program invariants. Of course, the analyses they perform are dependent on the correctness of the specifications and invariants provided, requiring, in practice, significant effort and expertise.

Since the main goal of this thesis is to develop a type of analysis that is commonly performed by optimizing compilers, the most suited methodology is the one giving up completeness. This analysis is called graph analysis, and will be discussed in Section 1.3. To close this light introduction, we leave the reader with a pathological example of useless sound approximation, so that it can be understood what it means to have useful approximations as opposed to just caring about soundness.

[Mou+15] Moura et al., “The Lean Theorem Prover (System Description)”

[GL02] Grégoire and Leroy, “A compiled implementation of strong reduction”

[Pau93] Paulson, *Isabelle’s Object-Logics*

[Nor08] Norell, “Dependently Typed Programming in Agda”

[Bob+11] Bobot et al., “Why3: Shepherd Your Herd of Provers”

[Lei10] Leino, “Dafny: An Automatic Program Verifier for Functional Correctness”

Note that any **desirable** approximation is a **useful** approximation. Consider the following example of sound termination analysis:

```
type terminates = Yes | No | Maybe
program -> terminates
let sound_termination_analysis _ = Maybe
```

It is sound, as `Maybe` comprises both of `Yes` and `No`, and, in soundness, the analysis will always be right. It computes in $O(1)$! But it is as useless as it is fast, as no tangible information can be extracted by this kind of “analysis”.

1.3 WHAT IS A CALL GRAPH AND WHY IS IT IMPORTANT?

In the context of program analysis, a call graph is a prerequisite for verifying several inter-procedural properties of interest, such as information flow analysis, call-ordering requirements, safety, and liveness, and, as already mentioned is also employed by optimizing compilers. A call graph is a directed graph that encodes the calling relationships between functions or methods in a program. In such a graph, functions or methods are represented as vertices (i.e., f , g). An edge between two vertices ($f \rightarrow g$) represents that f calls g . Formally, a program’s call graph is a directed graph $G = (V, E)$ in which the set of nodes V is the set of

functions in the program, and edge $(f, g) \in E \subseteq V \times V$ if function f calls function g in the program.

The construction of a precise, non-approximate call graph is made more complex than a syntactical reachability analysis by features like dynamic dispatching. Dynamic dispatching is the mechanism by which function calls are resolved at runtime rather than compile time. For instance, Java employs dynamic dispatching to resolve calls to overridden methods based on the runtime type of the object. The runtime object's type might differ from the syntactical type of the object, as the syntactical type might pass through a superclass reference.

Stemming from information derivable only by the semantic properties of a program, call graph analysis is undecidable, and, as stated in [Section 1.2](#), its analysis has to be either approximate or partially manual. Given that partially manual analyses are more suited to demonstrating more sophisticated properties, we prefer to keep the analysis fully automated. Given this restriction, we must keep in mind that the soundness and completeness properties of the call graph obtained will be propagated to any analysis that uses the call graph. For this reason, as a rule of thumb, the closer a call graph is to the *ground truth*, the better it is. Excluding the target program, the precision and therefore computational cost of producing a call graph depends on the information the analyzer is aware of, commonly defined as *sensitivity*, which could include (or ignore) information about execution paths, context, and so on.

In order not to lose coverage of potential edges leading to undesirable execution paths, or altering the semantics of a program in the case of compilers, the conventional approach consists in soundly approximating all the possible call targets. This means that an edge $f \rightarrow g$, in a sound approximation, really means *f may call g*. Conversely, in a complete approximation, an edge $f \rightarrow g$ means *f will call g*.

As a note, the addition of spurious edges to the approximation might also not matter. Consider a hypothetical verification process that uses a sound call graph with some non-existing edges: if the analysis does not produce any violation, the false positives did not impact the results. If it does, a more precise call graph analysis can be used to rule out some false positives.

We now introduce some of the methods that can be used for call graph construction, both sound and unsound.

1.3.1 Run-time Call Graph Construction

Run-time call graph construction is, intuitively, the process by which a call graph is constructed at runtime. This is done by instrumenting the target program in order to discover edges of the call graph. This, of course, produces a complete call graph for a given program execution, but ignores all the other possible program executions, which can be infinite. Binary dynamic analysis frameworks and tools, such as Valgrind [\[NS07\]](#), explore this kind of approach.

This approach can be applied as well to analysis techniques that simulate execution, such as symbolic execution, already mentioned in

[NS07] Nethercote and Seward, “Valgrind: a framework for heavyweight dynamic binary instrumentation”

[Bal+18] Baldoni et al., “A Survey of Symbolic Execution Techniques”

[DGC95] Dean et al., “Optimization of Object-Oriented Programs Using Static Class Hierarchy Analysis”

[BS96] Bacon and Sweeney, “Fast Static Analysis of C++ Virtual Function Calls”

[TP00] Tip and Palsberg, “Scalable propagation-based call graph construction algorithms”

[AL05] Andersen and Lee, “Program Analysis and Specialization for the C Programming Language”

[Ste96] Steensgaard, “Points-to Analysis in Almost Linear Time”

[BS16] Balatsouras and Smaragdakis, “Structure-Sensitive Points-To Analysis for C and C++”

Section 1.2. Symbolic execution allows to reason about all possible inputs for a program. The explicit symbolic disjunctions employed to perform conditionals and loops can quickly lead into state explosion, making the methodology unsuited to medium and large programs, even though it has seen success when analyzing selective parts of programs, with both approaches covered and explored in [Bal+18].

1.3.2 Static Call Graph Construction

Given how relevant call graphs are to program verification, scientists devised a plethora of algorithms for static call graph construction. Class Hierarchy Analysis (CHA) [DGC95] is a widely adopted call graph construction algorithm, as it provides sound call graphs for a small computational cost. CHA is a type-based reference analysis that uses only the declared type information to compute a call graph. Intuitively, in the case of explicit calls, they are reported as such, and in the case of calls by super class references that cannot be reduced to a concrete type, approximate the call as multiple edges.

As one might guess, this algorithm is not very precise. Other solutions that build on top of CHA have been proposed, such as Rapid Type Analysis (RTA) [BS96], Field Type Analysis (FTA) [TP00]. Another possible approach in call graph construction involves producing and leveraging points-to information derived from pointer analysis with varying degrees of context-sensitivity [AL05; Ste96; BS16]. This is useful in order to exploit knowledge, even if approximate, about what a variable might hold: if it holds a value or if it is a pointer, what value it might be holding or what the pointer might reference during a program execution.

1.4 CHALLENGES IN ANALYZING WEBASSEMBLY

As will be explained more in depth in **Chapter 2**, WebAssembly’s architecture poses different challenges that make analyzing it quite different from higher-level languages. Whereas modern complex languages might employ complex data structures, dynamic typing, or other hard to analyze features, WebAssembly is statically typed and features numerical types and arrays of numerical types only. Another difference from other languages is its typical use case of acting as a library-like application without being necessarily bound to a specific client. Second, this library-like application is stateful and externally mutable, and without any indication or assumption on what kind of interactions the client can have with it, analysis results targeting realistic programs can turn out to be inconclusive, which, as previously stated, is undesirable. Lastly, the simple structure of the WebAssembly runtime allows for call graph construction via type inference, like the case of CHA, but this analysis can turn out to be both imprecise and unsound, as will be more in depth discussed later in the document. More precise approaches, such as value analysis, are typically not used for call graph extraction in other languages due to their computational cost.

1.5 CONTRIBUTION AND OUTLINE

REST OF PART I In the rest of **Part I**, we give a thorough explanation in **Chapter 2** on how WebAssembly works, why it matters, and what its typical use cases are. We also cover why it is challenging to analyze and the literature on the analysis of WebAssembly code. We also introduce static analysis by Abstract Interpretation in **Chapter 3**, by providing examples on how to analyze a small functional language.

PART II - IN VITRO **Part II** contains our first contributions (from [PRL24]), focusing on an *in vitro* approach to analyzing WebAssembly code, that is, ignoring the possible interactions host programs may have with modules. In **Chapter 4**, we define a core subset of WebAssembly of interest, and define its concrete and abstract state semantics in **Section 4.1**. **Section 4.2** concerns over the semantic domains for the semantics of WebAssembly declarations, and concrete and abstract semantics. Then, declaration semantics are presented in **Section 4.3**, and the *isolated* concrete and abstract semantics of WebAssembly are presented in **Section 4.4** and **Section 4.5**. **Chapter 5** is all about proving the soundness of the abstract semantics. First, we introduce the structure of our proof in **Section 5.1**. We then introduce a bounded version of the abstract semantics in **Section 5.2**, which we prove sound in **Section 5.3**. Then, we prove termination and soundness of our abstract semantics in **Section 5.4**. **Chapter 6** covers the notions that are necessary to understand how WebAssembly applications are typically run. **Section 6.1** contains explanations on how the WebAssembly encapsulation mechanism works, and shows how hosts can significantly impact the semantics of modules. Then, in **Section 6.2**, we discuss how the various methodologies of program and library analysis can be applied to the analysis of WebAssembly applications, each with its strengths and weaknesses, and lay out our first proposed analysis methodology in **Section 6.3**.

[PRL24] Paccamiccio et al., “Building call graph of WebAssembly programs via abstract semantics”

PART III - IN VIVO **Part III** covers the *in vivo* analysis of WebAssembly applications, that is, analyzing WebAssembly programs considering how hosts may impact their semantics. In **Chapter 7**, we introduce our prototype, *wassilly* [PRL26], the first, to our knowledge, analyzer for WebAssembly that allows for host behavior specification. We discuss its possibilities, design, and enhancements left outside the formalization in **Section 7.1**. We evaluate *wassilly* against the state of the art in **Section 7.4**, and discuss its limitations in **Section 7.3**. From its limitations, mainly in usability, and what we learned from the evaluation, in **Section 7.5** we discuss potential strategies to make *wassilly* more usable. In **Section 7.6** we discuss and formalize a novel analysis that synergizes with *wassilly*. This analysis targets JavaScript hosts, as it is the most widely used host language for WebAssembly. This analysis aims at detecting violations of mutability contracts of WebAssembly-originating allocation sites, thereby enabling verification of behavioral assumptions about how a certain host interacts with a WebAssembly module.

[PRL26] Paccamiccio et al., “wassilly: Spec-enhanced Call Graph Analysis for WebAssembly”

PART IV - EPILOGUE **Part IV** serves as a recap and conclusion of the work presented in this document, and presents general guidelines on how the analysis of WebAssembly modules can be improved.

2

A Deep Dive into WebAssembly

PREAMBLE

WebAssembly emerged with a bold promise: to bring near-native performance to the web while maintaining the security and portability that have long been hallmarks of web technologies. What began as a low-level, efficient compilation target for browsers has rapidly evolved into a platform that extends beyond the client-side, reaching into serverless, embedded, and edge-computing environments. At first glance, WebAssembly appears elegantly minimal: its stack-based virtual machine, compact binary format, and strict type discipline all contribute to a sense of simplicity. But as with many systems designed for performance and security, beneath this surface lies a rich and intricate design. Understanding WebAssembly fully requires navigating its workflow, from source-language compilation and module validation to runtime instantiation and execution. Along the way, one encounters a language shaped not only by the need for efficiency, but also by a deep concern for formal semantics and predictable behavior. This chapter is a guided tour through the landscape of WebAssembly. We begin with its workflow: how code in languages like C or Rust is compiled into WebAssembly modules, ready to run in a host environment. We then explore the execution model, its memory and stack structures, and its approach to control flow. To ground these concepts, we take a close look at the syntax of WebAssembly and discuss its intuitive semantics. But understanding how WebAssembly works is only part of the story. The second half of this chapter turns to the challenges of static analysis.

While WebAssembly was designed with verification and tooling in mind, analyzing Wasm programs statically is far from trivial. The challenges imposed by WebAssembly's structure compel us to rethink more traditional analysis techniques and adapt them to fit WebAssembly's unique characteristics. By the end of this chapter, the reader will have a solid foundation in both the operational structure and the semantic intuitions of WebAssembly. This foundation will support the more advanced discussions that follow, particularly those focused on formal analysis and tooling based on that formalism.

*“... or, as I like to put it: how does one
mitigate bad JavaScript implementations
from further contributing to global
warming?”*
—The author.

2.1 WHAT DOES A WEBASSEMBLY APPLICATION LOOK LIKE?

In a typical web browser, code is executed locally by a JavaScript engine. Unfortunately, JavaScript does not offer optimal performance for every task. Solving this performance issue is the original *mission* of WebAssembly [Haa+17]. Higher-level languages are compiled into WebAssembly typically by dedicated compiler toolchains, such as Emscripten [Zak11]. We show an example of this workflow in Figure 2.1.

[Haa+17] Haas et al., “Bringing the web up to speed with WebAssembly”
[Zak11] Zakai et al., *Emscripten - Complete compiler toolchain to WebAssembly*

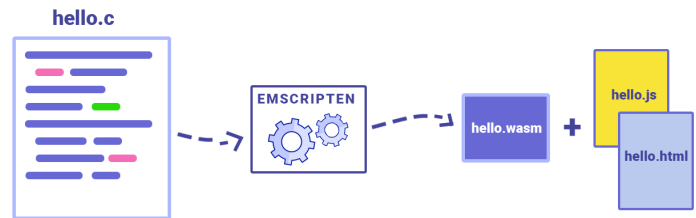


FIGURE 2.1: Emscripten toolchain compiling a C program into WebAssembly.

Compiling to WebAssembly allows, from **portable** high-level code, to obtain one **portable** binary, as opposed to different OS- and architecture-dependent binaries, sacrificing as little as 20% in performance, an overhead justified by the execution of WebAssembly code on top of a virtual machine as opposed to native OS-level execution.

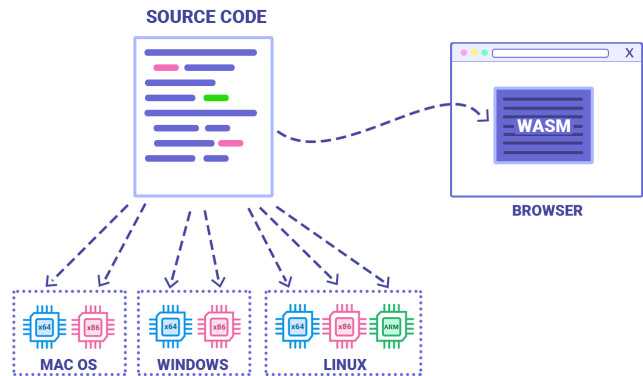


FIGURE 2.2: Portability comparison: native vs WebAssembly.

We display the potential of WebAssembly in terms of portability in Figure 2.2, where a single WebAssembly binary (and virtual machine) is capable of running on the most common operating systems and CPU architectures, whereas the native approach would require a total of seven binaries. This execution and embedding environment is still young and does not yet have a clear and defined API standard. As of 2025, two sets of APIs are most popular: the web API, which is dependent on the JavaScript engine integrated in a web browser; and the more general WASI (WebAssembly Application System Interface) API standard, focused on providing a standard set of APIs for WebAssembly to run on wider families of systems. As of 2025, the WebAssembly JavaScript API remains the dominant, widely deployed interface (via universal browser support),

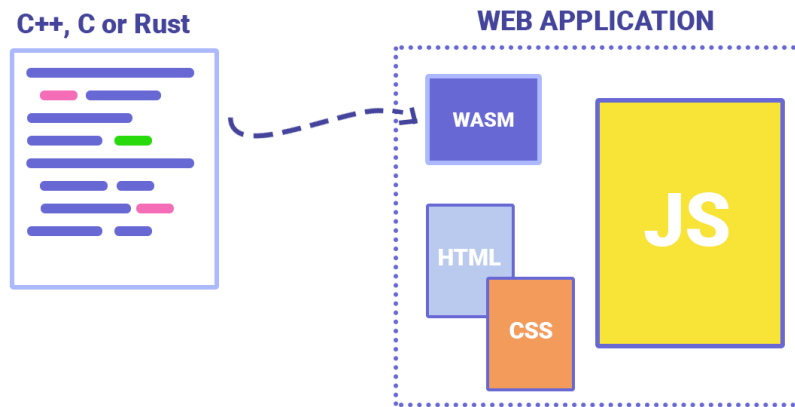


FIGURE 2.3: The layout of a web application that embeds WebAssembly.

while WASI is rapidly evolving and expanding on the server-side use, but has not yet reached comparable ubiquity

2.2 LANGUAGE BASICS

WebAssembly [Haa+17] is a typed assembly-like language. It uses a structured stack machine as an execution paradigm. It is a compilation target (processor architecture) for the LLVM [Lat02; LA04] family of programming languages. To date, the most common example of a WebAssembly runtime environment is a sandboxed environment for web browsers. It also features standalone runtime environments for resource-constrained devices [All19]. The overall paradigm Wasm adopts is very similar to the one that Java and the Java Virtual Machine adopt: source code is compiled into bytecode and then executed by a virtual machine.

The memory model used by WebAssembly is composed of a *store*, a *stack*, and an *instance* of the module. The *store* represents the global state that WebAssembly programs can manipulate. It consists of the runtime representation of all instances of functions, tables, memories, globals, element segments, and data segments allocated during the lifetime of the abstract machine. The module *instance* is a runtime representation of a module obtained by instantiating a module. It holds references to the objects managed by the *store*.

Besides the store, most instructions interact with an implicit operand *stack*. The operand stack contains three kinds of entries:

- (i) values: the operands of instructions;
- (ii) labels: active structured control instructions targetable by branch instructions;
- (iii) activations: call frames of active function calls.

Another valid stack representation sees the stack as an activation stack, with each activation having its own operand stack. Call graph analysis is the salient concern of this work, so an explanation of how WebAssembly manages control flow is due: function calls are performed by either the `call` operator or the `call_indirect` operator, with the latter used to perform function indirection and introducing possible imprecision when trying to resolve call targets statically, whereas intra-

[Lat02] Lattner, “LLVM: An Infrastructure for Multi-Stage Optimization”

[LA04] Lattner and Adve, “LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation”

[All19] Alliance, *WebAssembly Micro Runtime*

[Gro19] Group, *WebAssembly Control Instructions*

[FS20] Flores-Montoya and Schulte, “Data-log Disassembly”

[Arc+24] Arceri et al., “Towards a Sound Construction of EVM Bytecode Control-Flow Graphs”

procedural control flow is managed predictably with structured control instructions [Gro19]. The predictable management of intra-procedural control flow allows for the production of a correct control flow graph of a function by using trivial syntactical analyses, whereas, for other languages that allow for arbitrary jumps to happen, this represents a challenge, as shown in [FS20; Arc+24].

In WebAssembly’s naming convention, WebAssembly programs are called *modules*, and the client using them is called the *host*. WebAssembly modules are akin to stateful libraries; their role is to provide additional functionality to the host and have a mutable internal state that can be affected by both module and host.

2.2.1 Examples of WebAssembly

In this section we introduce the syntax of WebAssembly with an *appetizer* of simple programs to familiarize with the syntax, to then introduce the syntax and intuitive semantics of control instructions.

ONE PLUS ONE EQUALS TWO Listing 2.1 presents a basic example. The `(module [...])` instruction introduces the most minimal unit of a WebAssembly program: the module. In this example, we have one function, `$add`, that performs addition on the two parameters it accepts. They are first loaded into the operand stack by the `local.get` instructions, then consumed by `i32.add` that produces the result. Since the main block of the function has no more instructions to execute, and the stack is well-formed, meaning there are only *n* (in this case, one) well-typed values on the stack, the end of the function block performs an implicit return.

Note that the Wasm validator can check the stack shape syntactically and reject programs with erroneous stack shapes. This issue might occur in handwritten WebAssembly programs, as no properly working compiler should be able to produce an erroneously shaped stack.

Note that WebAssembly does not use names to address objects but uses indices, as shown in the `local.get` instructions. Names, such as `$add` are present for mere convenience and are not guaranteed to be preserved after compilation.

Lastly, `(export ename (func fid))` exports a function identified by `fid` as `ename` (e.g., `export name`), so that it can be invoked outside the module, similarly to public methods in Object-Oriented languages.

In short, the module in Listing 2.1 exposes an `add` function that both the host and the module can use.

CALLING The instructions `call` and `call_indirect` are responsible, respectively, for calling functions directly and via dereferencing. In Listing 2.2 we show an example of how direct function calls work. Intuitively, we export a `$main` function that calls `$add` transitively.

```

1 (module
2   (func $add (param i32 i32) (result i32)
3     local.get 0
4     local.get 1
5     i32.add)
6   (export "add" (func $add))
7 )

```

Listing 2.1: 32-bit integer addition between two parameters in WebAssembly.

```

1 (module
2   (func $add (param i32 i32) (result i32)
3     local.get 0
4     local.get 1
5     i32.add)
6   (func $main (param i32 i32) (result i32)
7     local.get 0
8     local.get 1
9     call $add)
10  (export "main" (func $main))
11 )

```

Listing 2.2: WebAssembly direct function call.

The role of `call_indirect` is to emulate function pointers and dynamic dispatching in general and, as expected, is slightly more complicated. Consider the example in Listing 2.3. It features three functions, with `$main` being exported. `$main` performs a `call_indirect` to a function of type $i32 \times i32 \rightarrow i32$. In this case, the reference is constant to index 0 (`i32.const 0`). But what does 0 reference? WebAssembly performs indirection using a reference table, defined by (`elem $table [...]`). The 0-th element of the `table` corresponds to the function with index 1, or `$sub`. In short, a WebAssembly reference table maps table indices to functions, and `call_indirect` uses table indices to retrieve functions.

```

1 (module
2   (func $add (param i32 i32) (result i32)
3     local.get 0
4     local.get 1
5     i32.add)
6   (func $sub (param i32 i32) (result i32)
7     local.get 0
8     local.get 1
9     i32.sub)
10  (func $main (param i32 i32 i32) (result i32)
11    local.get 0
12    local.get 1
13    i32.const 0
14    call_indirect (param i32 i32) (result i32))
15  (table $table 2 funcref)
16  (elem $table (i32.const 0) 1 0)
17  (export "main" (func $main))
18 )

```

Listing 2.3: Call via function indirection. Function names in table are hidden on purpose.

Given this small appetizer to familiarize with the basics of the syntax of WebAssembly, we now proceed to explain its local control-flow constructs.

```

1 (module
2 (func $fib (param i32 i32 i32) (result i32)
3   (if (result i32)
4     (i32.eqz (local.get 0))
5     (then (local.get 1))
6     (else (call $fib
7             (i32.sub (local.get 0) (i32.const 1))
8             (local.get 2)
9             (i32.add (local.get 1) (local.get 2)))
10    )
11  )
12 )
13 (func $main (param i32) (result i32)
14   (call $fib (local.get 0)
15             (i32.const 0) ;; seed value
16             (i32.const 1)) ;; seed value
17 (export "main" (func $main))
18 )

```

Listing 2.4: Fibonacci's function in WebAssembly.

N-THFIB() In Listing 2.4, we show a slightly more complex module that is responsible for calculating the *n*-th number of the well-known Fibonacci sequence. For the sake of readability, we make use of the S-expression syntax as opposed to the standard "stack machine" version. The module shown in Listing 2.4 exposes a `main` function that is responsible for calling `$fib` with a user-defined parameter and constant seed values of 0 and 1. The first local control instruction is now unveiled, as `$fib` uses the conditional `if (T) (E) then (C) else (C)`. In WebAssembly, control constructs need explicit type annotations if they are to return values at the end of control blocks, such as are `then` and `else`. The `(T)` in the conditional states exactly this: both `then` and `else` must return a *T*-typed vector of values. The boolean expression *E* is represented by `(i32.eqz (local.get 0))` (reads as *EqualsZero*) with `i32.eqz` returning 0 or 1 depending on the input being equal or non-equal to 0. This unveils another peculiarity that WebAssembly shares with other assembly-like languages and C: there is no explicit boolean value type, but the boolean evaluation instead consists of a function mapping 0 to *false* and all other values to *true*.

BLOCK, LOOP, BRANCHING WebAssembly features statements that create typed labels that can be branched out, in the case of `block` statements, or into, in the case of `loop` statements, by branching statements, with `br` being the unconditional branch instruction and `br_if` the conditional one. Of course, to perform a legal branch, the label must be "in scope" (e.g. it must not have ended yet): borrowing some canonical language knowledge from C, `br` and `br_if` behave like *break*-s, in case the targeted label is a `block` and *continue*-s, in case the targeted label is a `loop`. In Listings 2.5 and 2.6 we show examples for the aforementioned constructs.

Having sorted the basics in the syntax of WebAssembly, we now show some examples of importing external resources inside WebAssembly modules.

```

1 (module
2   (func (export "log_if_not_100") (param $num i32)
3     (block $my_block
4       ;; $num is equal to 100
5       local.get $num
6       i32.const 100
7       i32.eq
8       br_if $my_block
9       ;; not reachable when $num is 100
10      i32.const 42
11      return
12    )
13    local.get $num
14  )
15 )

```

Listing 2.5: Usage of the Block construct in WebAssembly.

```

1 (module
2   (func
3     (local $i i32)
4     (loop $my_loop
5       ;; add one to $i
6       local.get $i
7       i32.const 1
8       i32.add
9       local.set $i
10      ;; if $i is less than 10 branch to loop
11      local.get $i
12      i32.const 10
13      i32.lt_s
14      br_if $my_loop
15    )
16  )
17
18  (start 1) ;; run the first function automatically
19 )

```

Listing 2.6: Usage of the Loop construct in WebAssembly.

```

1 (module
2   (import "console" "log" (func $log (param i32)))
3 )

```

Listing 2.7: Import used in WebAssembly.

IMPORTS Intuitively, imports allow a WebAssembly module access to an external resource, whether it be a (global) variable, a reference table, or, most notably, a function. Listing 2.7 shows an example of importing the JavaScript `console.log` printing function. Imported functions can be functions embedded by compilers, such as the `malloc` and `free` functions of the C language.

2.3 ANALYSIS OF WEBASSEMBLY

Given the nature of WebAssembly of “aiding” other programs in a library-like fashion, when aiming to analyze WebAssembly code, one should first answer this question of **what** kind of WebAssembly application are we analyzing. The cases to consider are (assuming only a single WebAssembly module):

- (i) a stand-alone module without external dependencies;
- (ii) a module that provides specific functionalities that is to be used in possibly many settings as a library (i.e. a module providing cryptographic primitives);
- (iii) a module providing additional functionalities tied to a specific host application.

Depending on the answer to this question, different sets of analysis principles may or may not be applicable. For instance, point (i) could be considered a “textbook” form of program analysis. Point (ii) could be analyzed with techniques and methodologies that stem from principles of library analysis, and, finally, when analyzing (iii), it could be desirable to do so by exploiting techniques that work well in multilanguage analyses.

In this section, we gently and informally introduce the reader to the necessities an analyzer may have in the cases described above, the consequences of not meeting these necessities in realistic scenarios, and describe methodologies and frameworks to address them.

2.3.1 Closed Module Analysis

The analysis of a closed module closely relates to the analysis of bullet point (i) described in [Section 2.3](#), with a minor caveat: external dependencies are accepted, but do not impact the state of the module under analysis. This kind of analysis is simple but has little applicability to realistic scenarios: even stand-alone compiler-generated WebAssembly code still uses the WASI (WebAssembly System Interface) API [[Gro20](#)] as a de facto standard. In practicality, the resulting WebAssembly binary will import functions that **will** perform side-effects on the module state, in particular in its memory section. Ignoring these side effects will produce unsound analyses. With this in mind, as of today, sound analysis of WebAssembly is not realistic under the assumption of a closed module, unless the target programs are handwritten textbook examples, i.e., examples from [Section 2.2](#). The closed module example, though, is nonetheless helpful in laying the foundation and building up to more sophisticated analyses of open modules. In fact, the analysis of an open module is the analysis of a closed module with knowledge or assumptions about the side effects of external dependencies.

2.3.2 Inconclusive or Unsound?

With the elements discussed so far, we can begin to make preliminary considerations for the analysis of WebAssembly modules. First and foremost, closed modules are good toy examples, but, in the context of developing sound analyses, they lack applicability in realistic scenarios. This weakness of the closed-module paradigm is emphasized by the host’s ability to significantly modify the module state by impacting shared reference tables, memories, and variables, leaving us with three main options:

- (i) assume no knowledge about the host;
- (ii) analyze the module under a set of assumptions on host behavior that, if held, make the analysis result sound;

[Gro20] Group, *WebAssembly System Interface - WASI*

- (iii) provide information about host behavior that the analyzer can leverage.

All of these approaches have their pros and cons, and we will discuss them to better understand the implications of taking each path.

APPROACH (i) While enticing at first glance because it is sound and quite simple, it actually hides a significant drawback. By assuming no knowledge about the host environment, the analyzer must conservatively model that the host can perform any interaction with the shared resources at any time. Concretely: exported linear memory and reference tables must be treated as entirely mutable by the host at both instantiation time and at any time a host function is invoked. The consequence is problematic: any value stored to a shared resource becomes $\top$ (representing “any possible value”). Subsequently, any value loaded from such a resource is also $\top$. This $\top$ propagates through subsequent computations, rapidly infecting derived values. If a substantial portion of our knowledge is effectively destroyed by being overly conservative, the result is a sound but overwhelmingly imprecise analysis, often collapsing all interesting properties to “unknown”. Pursuing soundness and automation at all costs thus leads to analyses that are, in practice, useless. To the best of our knowledge, no WebAssembly analyzer released to date uses this method.

APPROACH (ii) Approach (ii) relies on the notion of Most General Client (MGC) [Rin+07; AKS15], that we explain more in depth, together with Reasonably Most General Clients (RMGCs) [KM19], in Section 6.2.1. Briefly, a MGC is a default safe assumption about client behavior that allows the abstract client to use a library in any possible way. This approach is generally more applicable to the analysis of languages that have strong encapsulation mechanisms, a property that WebAssembly does not enjoy. RMGCs can be seen as a restricted version of MGCs. This approach is tunable in its assumptions, which can be made stricter or more relaxed. A comparison with approach (i) sees (ii) as more complex but more attractive: if unknown imports are involved, as is the case with most practical Wasm programs, the analysis can still produce useful results. If the RMGC assumption is respected the analysis is sound, if it is not, useful information can still be collected. This approach has been adopted in literature to analyze JavaScript libraries in [KM19] and subsequently ported to Wasm [Bra+23]. Given that the existing analyses target pure WebAssembly code, to the best of our knowledge, there is no literature on the verification of the assumptions made by RMGCs. More broadly, static analyzers for Wasm, such as *wassail* [SDR20; SDR21], and *sturdy* [Bra+23], aim for *soundness*: “*soundness* attempts to capture the balance, prevalent in practice, of over-approximated handling of most language features, yet deliberately under-approximated handling of a feature subset well recognized by experts. [...] A *soundy* analysis aims to be as sound as possible without excessively compromising precision and/or scalability.” [Liv+15].

[Rin+07] Rinetzky et al., “Modular Shape Analysis for Dynamically Encapsulated Programs”

[AKS15] Allen et al., “Combining type-analysis with points-to analysis for analyzing Java library source-code”

[KM19] Kristensen and Møller, “Reasonably-most-general clients for JavaScript library analysis”

[Bra+23] Brandl et al., “Modular Abstract Definitional Interpreters for WebAssembly”

[SDR20] Stiévenart and De Roover, “Compositional information flow analysis for webassembly programs”

[SDR21] Stiévenart and De Roover, “Was-sail: a webassembly static analysis library”

[Liv+15] Livshits et al., “In defense of soundness: a manifesto”

[OM20] Ouadjaout and Miné, “A Library Modeling Language for the Static Analysis of C Programs”

[Mon21] Monat, “Static type and value analysis by abstract interpretation of Python programs with native C libraries. (Analyse statique, de type et de valeur, par interprétation abstraite, de programmes Python utilisant des librairies C)”

[BBY17] Blazy et al., “Structuring Abstract Interpreters Through State and Value Abstractions”

[Büh17] Bühler, “Structuring an Abstract Interpreter through Value and State Abstractions:EVA, an Evolved Value Analysis for Frama-C. (Structurer un interpréteur abstrait au moyen d’abstractions de valeurs et d’états :Eva, une analyse de valeur évoluée pour Frama-C)”

[BMP24] Blanchard et al., “Formally Expressing What a Program Should Do: The ACSL Language”

[Min13] Miné, *Static analysis by abstract interpretation of concurrent programs. (Analyse statique par interprétation abstraite de programmes concurrents)*

[Che20] Chevalier, “Proving the security of software-intensive embedded systems by abstract interpretation. (Analyse de la sécurité de systèmes critiques embarqués à forte composante logicielle par interprétation abstraite)”

[Gro20] Group, *WebAssembly System Interface - WASI*

APPROACH (iii) It is the most complex of the three, as it requires non-trivial knowledge of the semantics of external dependencies, obtainable either by manual specification or additional analysis. This necessity, while dependent on a substantial additional effort on top of analyzing WebAssembly, allows retaining soundness, as long as the semantics provided are sound, with the guarantee to obtain results at least as precise as the ones obtained by approach (ii). This approach shares with approach (i) the fact that there is no assumed “default” knowledge of the host. It differs in that it aims to maintain both precision and soundness, but this may require significant effort if a manual specification is necessary. Well known abstract interpreters leveraging annotations are MOPSA [OM20; Mon21], Frama-C [BBY17; Büh17; BMP24], Astrée [Min13; Che20].

APPROACH (ii $\frac{1}{2}$) The issue with approach (ii) lies in the fact that WebAssembly analyzers can never produce sound results under the RMGC assumption. Not because mutations of tables or linear memories provide a performance or any advantage, but because WebAssembly requires these kinds of mutations to be impactful in the real world. In fact, the de facto WASI [Gro20] standard systematically leverages mutations to the linear memory to enable efficient host-module communication. We call this approach (ii $\frac{1}{2}$) because we can leverage the knowledge of the semantics of WASI syscalls and combine them with the assumptions made by RMGCs so as not to necessarily need an explicit specification for each imported dependency. Unfortunately, this approach is not directly applicable and requires an existing take on approach (iii), which is the principal focus of this work.

2.4 RELATED WORK

WEBASSEMBLY IN GENERAL Since its inception [Haa+17], WebAssembly captured the attention of researchers. An alternative formalization to the original semantics has been proposed in [Wat18]. Work is also being done to improve performance when handling complex data structures [SMR24] and support concurrency [WRPP19]. It is also being studied in non-web environments [SM21; WKD22], with efforts on the engineering side to make it practical [Mé+21; LSS24]. Being a very pervasive language, as it runs in every widespread browser, its security implications are of great interest and have been studied in [Joh+23] and [LKP20].

WEBASSEMBLY STATIC CALL GRAPH CONSTRUCTION Lehmann et al. review in [Leh+23] static call graph construction in WebAssembly. Among the works covered there are: *wassail* [SDR20; SDR21], *Twiggy* [Twi22], *MetaDCE* from the *Binaryen* [Bin23] suite, and *WAVM* [WAVnd]. All of these offer means for implicit or explicit call graph extraction, but none of these tools base their answers on the specific semantics of a target program. Specifically, *wassail* can construct call graphs statically, resolving indirect calls via type signatures. It also does information-flow analysis and program slicing. *Twiggy* is a Wasm code profiler, aimed at binary optimization and debloating. *MetaDCE* is the dead code elimination module from *Binaryen*, a compiler toolchain for WebAssembly. *WAVM* is a LLVM-based Wasm-to-native compiler. To do so, it leverages a *lifter* from Wasm to LLVM. [Leh+23] compares all of these on 24 benchmarks designed to uncover the limitations of each analyzer. Other works not covered in [Leh+23] offer analyses based on syntax only [Bre+23; Bri+22]. *sturdy* [Bra+23] is another option capable of performing call graph constructions via interval- or constant-based abstract interpretation of Wasm code. *sturdy* supports static dead code elimination as well.

[Haa+17] Haas et al., “Bringing the web up to speed with WebAssembly”

[Wat18] Watt, “Mechanising and verifying the WebAssembly specification”

[SMR24] Silva et al., “Efficient Data Exchange between WebAssembly Modules”

[WRPP19] Watt et al., “Weakening WebAssembly”

[SM21] Spies and Mock, “An Evaluation of WebAssembly in Non-Web Environments”

[WKD22] Wallentowitz et al., “Potential of WebAssembly for Embedded Systems”

[Mé+21] Ménétrey et al., “Twine: An Embedded Trusted Runtime for WebAssembly”

[LSS24] Lauwaerts et al., “WARDuino: An embedded WebAssembly virtual machine”

[Joh+23] Johnson et al., “WaVe: a verifiably secure WebAssembly sandboxing runtime”

[LKP20] Lehmann et al., “Everything Old is New Again: Binary Security of WebAssembly”

[Leh+23] Lehmann et al., “That’s a Tough Call: Studying the Challenges of Call Graph Construction for WebAssembly”

[SDR20] Stiévenart and De Roover, “Compositional information flow analysis for webassembly programs”

[SDR21] Stiévenart and De Roover, “Wassail: a webassembly static analysis library”

[Twi22] Twiggy, *A code size profiler for Wasm*

[Bin23] Binaryen, *Optimizer and compiler/toolchain library for WebAssembly*.

[WAVnd] WAVM, *WebAssembly virtual machine, designed for use in non-browser applications*.

[Bre+23] Breitfelder et al., “WasmA: A Static WebAssembly Analysis Framework for Everyone”

[Bri+22] Brito et al., “Wasmati: An efficient static vulnerability scanner for WebAssembly”

[Bra+23] Brandl et al., “Modular Abstract Definitional Interpreters for WebAssembly”

[He+24] He et al., “SeeWasm: An Efficient and Fully-Functional Symbolic Execution Engine for WebAssembly Binaries”

[Mar+22] Marques et al., “Concolic Execution for WebAssembly”

[Mos+19] Mossberg et al., “Manticore: A User-Friendly Symbolic Execution Framework for Binaries and Smart Contracts”

[He+23] He et al., “Eunomia: Enabling User-Specified Fine-Grained Search in Symbolically Executing WebAssembly Binaries”

[Bri+22] Brito et al., “Wasmati: An efficient static vulnerability scanner for WebAssembly”

[SDR20] Stiévenart and De Roover, “Compositional information flow analysis for webassembly programs”

[Bas+22] Bastys et al., “SecWasm: Information Flow Control for WebAssembly”

[KM19] Kristensen and Møller, “Reasonably-most-general clients for JavaScript library analysis”

[Bra+23] Brandl et al., “Modular Abstract Definitional Interpreters for WebAssembly”

[AKS15] Allen et al., “Combining type-analysis with points-to analysis for analyzing Java library source-code”

[Rin+07] Rinetzky et al., “Modular Shape Analysis for Dynamically Encapsulated Programs”

[OM20] Ouadjaout and Miné, “A Library Modeling Language for the Static Analysis of C Programs”

[BMP24] Blanchard et al., “Formally Expressing What a Program Should Do: The ACSL Language”

WEBASSEMBLY GENERAL ANALYSIS Many approaches in WebAssembly analysis employ symbolic/concolic execution, such as *SeeWasm* [He+24], *WASP* [Mar+22], *Manticore* [Mos+19], and *Eunomia* [He+23]. These tools are mostly geared at finding bugs and, given the computation-intensive nature of symbolic execution, are not particularly suited for call graph analysis. A notable limitation of *Manticore* is the lack of support for imported functions. Support for external dependencies on more recent tools is, in one way or another, a standard feature. *SeeWasm* supports external dependencies via WASI semantic stubs or behavioral contracts. *Eunomia* and *SeeWasm* are related: *Eunomia* specializes *SeeWasm*’s core engine for bug finding. *WASP*, being based on Wasm’s reference interpreter, inherits JavaScript host support from it. *Wasmati* [Bri+22] takes a different approach: it employs a graph-based representation of programs and performs checks on security properties on these. It requires a call graph to function and can generate one using type-based constraints. Static information flow analysis has also been of interest to researchers [SDR20; Bas+22], focusing mainly on preventing information leaks from secret inputs to observable outputs.

ANALYSIS OF PROGRAMS WITH UNSPECIFIED FUNCTIONS The analysis of **open** modules is strictly related to the analysis of libraries, and in general, close to the analysis of programs with unavailable code. The work presented in [KM19] proposes a *reasonable* most-general client approach that constrains what a client can do to JavaScript library instances, which, analogously to WebAssembly module instances, are stateful and mutable. This approach is also used in [Bra+23] to analyze WebAssembly modules. Most-general clients have been proposed [AKS15; Rin+07], but require strict encapsulation mechanisms (which WebAssembly lacks) to perform precise analyses reliably. In [OM20], the authors propose a modeling language geared to C libraries, simplifying the work that the static analyzer must perform. To the same end, the Frama-C static analyzer uses ACSL [BMP24].

3

Static Analysis by Abstract Interpretation

PREAMBLE

One of the broad goals of static analyses is to prove program properties. Consider the example of proving that a program does not reach erroneous states. This is done by extracting, via analysis, a set of reachable program states and verifying if all of them conform to a certain specification (i.e. all computations are performed successfully). In this chapter in particular, we want to guide the reader in an as simple as possible tutorial-like fashion in formalizing and implementing a static analyzer that proves if a given program can ever fail. Such tool will be based on the theory of Abstract Interpretation, a most general theoretical framework introduced in [CC77], that allows to express any sound static analysis. We also introduce much of the notation that will be used in the rest of this document.

A MOST GENERAL(LY HARD) FRAMEWORK Abstract Interpretation, as a general framework, can be quite challenging to grasp, and the goal of the following chapters is to explain static analysis using Abstract Interpretation as intuitively as possible while maintaining a formal approach. More formal and general introductions can be found in Miné [Min17], Rival and Yi [RY20], and Cousot [Cou21]. Additional resources may be found in the slides¹ of the many presentations given by Patrick Cousot on the subject. Besides the general theoretical background of Abstract Interpretation and order theory, we try to cover (most of) the formalization steps and intuitive design pills of simple static analyzers for `SlobSinopie`, a toy example for a functional language. The full implementation is freely available².

3.1 A FIRST TOY, `SLOB SINOPIE`

This section covers the analysis of `SlobSinopie` programs.

`SlobSinopie` is a toy ML-like functional language, designed to facilitate the understanding of underlying concepts of Abstract Interpretation without the issue of formalizing features typical of more expressive languages, such as mutability, loops, jumps, and similar complex constructs. It is statically typed, with `int` being the only type available. It features

“The crystal ball says $\top$, muttered the wizard. Little did he know, $\top$ was all the ball ever said — and it was never wrong.”
—Et al.

[CC77] Cousot and Cousot, “Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints”

[Min17] Miné, “Tutorial on Static Inference of Numeric Invariants by Abstract Interpretation”

[RY20] Rival and Yi, *Introduction to Static Analysis - An Abstract Interpretation Perspective*

[Cou21] Cousot, *Principles of Abstract Interpretation*

¹<https://pcousot.github.io/talks.html>

²<https://github.com/ringzer0x00/slobsinopie>

functions, so call graph analysis is applicable, and operations that can fail at runtime, as can be the case with the infamous *division by zero* runtime error. The `SlobSinopie` language is described in [Listing 3.1](#). As a functional language, `SlobSinopie` programs are a single expression composed of sub-expressions. Expressions can be constants, variables, function declarations, applications, conditional expressions, binary operations, and comparisons between two expressions. The clause of the conditional statement can only be a binary comparison between two expressions. `SlobSinopie` does not allow for higher-order functions.

$$\begin{array}{ll}
 x & ::= \text{<IDENTIFIER>} \\
 \text{cmp} & ::= \leq \\
 & \quad | = \\
 & \quad | > \\
 \text{cond} & ::= e_1 \text{ cmp } e_2 \\
 \text{bop} & ::= + \\
 & \quad | - \\
 & \quad | * \\
 & \quad | / \\
 e & ::= (e) \\
 & \quad | z \in \mathbb{Z} \\
 & \quad | e_1 \text{ bop } e_2 \\
 & \quad | \text{if cond then } e_2 \text{ else } e_3 \\
 & \quad | \text{let } x_1 \ x^* = e_1 \text{ in } e_2 \\
 & \quad | x \ e^*
 \end{array}$$
Listing 3.1: BNF grammar for `SlobSinopie`.

We show in [Listing 3.2](#) an example of `SlobSinopie` implementation for a program that computes the fifth element of the Fibonacci sequence.

```

1 let fib x =
2   if x = 0 then 0
3   else if x = 1 then 1
4   else (fib (x - 1)) + (fib x - 2) in
5 fib 5

```

Listing 3.2: Example of a program implementing and calling the Fibonacci function written in `SlobSinopie`.

3.2 SEMANTICS OF SLOBSINOPIE

We define the semantics of `SlobSinopie`. Semantics are a formal mathematical definition of the behaviors associated with each syntactical construct, whether it be an expression or a statement. The semantics of imperative languages describe program behavior over input and output program states, whereas in functional languages, semantics describe program behavior over input environments and output values. We represent an environment with σ , consisting of a mapping from variable to value.

Definition 1. A variable environment σ having type $\mathcal{M}$ is defined as a mapping from variable to value. Formally, $\mathcal{M} : \mathcal{V} \rightarrow \mathbb{Z}$.

Given how the language allows for function declarations, a mapping from function identifier to function body and parameters is available in a separate environment ϕ .

Remark 1. This approach of separating function and variable environments is not conventional, as in functional languages even constant declarations are still considered functions (they are, in fact, constant functions). This separation is done exclusively to simplify the design and implementation of the abstract and concrete interpreters.

Definition 2. A function environment ϕ of type $\mathcal{F}$ is defined as a mapping from function identifier f of type $\mathcal{ID}$ to a tuple containing the corresponding expression, of type $\mathcal{P}$ and the set of parameter variables $\mathbb{V}$ that will be used to build the environment inside the function scope. This is formally denoted by the following definition: $\mathcal{F} : \mathcal{ID} \rightarrow \mathcal{P} \times \mathbb{V}$.

Functional languages reason about functions and values. As such, our programs will evaluate to either: (i) one value of type $\mathbb{Z}$, (ii) a catastrophic error of type $\mathcal{E}$ or will not terminate, and be represented as $\perp$. Based on this and the previously defined elements, we define the semantic domain for our evaluation function $\mathcal{E}$.

Definition 3. The evaluation function $\mathcal{E}[\![\cdot]\!]$ evaluates the input tuple of type $\langle \mathcal{P} \times \mathcal{M} \times \mathcal{F} \rangle$ to a output or non-output of type $\mathbb{Z} \cup \mathcal{E} \cup \{\perp\}$.

An additional evaluation function is adopted to perform boolean comparisons. We refer to this function as $\mathcal{C}[\![\cdot]\!]$.

Definition 4. The boolean evaluation function $\mathcal{C}[\![\cdot]\!]$ evaluates the input tuple of type $\langle \mathcal{P} \times \mathcal{M} \times \mathcal{F} \rangle$ to a output or non-output of type $\mathcal{B} \cup \mathcal{E} \cup \{\perp\}$.

The semantics that follow are written in operational big-step style. Once a semantic notation is introduced a brief description that explains it will follow.

3.2.1 Semantics of Simple Expressions

We define as *simple* those expressions that are not comprised of sub-expressions. In our language, constant and variable expressions belong to this category.

$$\text{const} \frac{\llbracket z \rrbracket \sigma \phi = v}{\langle z, \sigma, \phi \rangle \Longrightarrow v} \quad (3.1)$$

Equation (3.1) is the function that evaluates constant numerical expressions.

$$\text{lookup} \frac{\sigma(x) = v}{\langle x, \sigma, \phi \rangle \Longrightarrow v} \quad (3.2)$$

Equation (3.2) evaluates variable expressions. It performs a lookup on the environment σ , retrieving and returning the value associated with the identifier.

3.2.2 Semantics of Operations Between Expressions

$$\text{eq} \frac{\llbracket e1 \rrbracket \sigma\phi = v1 \quad \llbracket e2 \rrbracket \sigma\phi = v2 \quad v1 \equiv v2}{\langle e1 = e2, \sigma, \phi \rangle \Longrightarrow \text{true}} \quad (3.3)$$

$$\text{eq} \frac{\llbracket e1 \rrbracket \sigma\phi = v1 \quad \llbracket e2 \rrbracket \sigma\phi = v2 \quad v1 \not\equiv v2}{\langle e1 = e2, \sigma, \phi \rangle \Longrightarrow \text{false}} \quad (3.4)$$

Equations (3.3) and (3.4) describes how equality comparisons between two expressions are performed. After evaluating them to values, the values are checked for equality and `true` or `false` are returned accordingly.

$$\text{leq} \frac{\llbracket e1 \rrbracket \sigma\phi = v1 \quad \llbracket e2 \rrbracket \sigma\phi = v2 \quad v1 \leq v2}{\langle e1 \leq e2, \sigma, \phi \rangle \Longrightarrow \text{true}} \quad (3.5)$$

$$\text{leq} \frac{\llbracket e1 \rrbracket \sigma\phi = v1 \quad \llbracket e2 \rrbracket \sigma\phi = v2 \quad v1 \not\leq v2}{\langle e1 \leq e2, \sigma, \phi \rangle \Longrightarrow \text{false}} \quad (3.6)$$

Equations (3.5) and (3.6) describes how the less-or-equal comparisons between two expressions are performed. Similarly to the equality comparison, after evaluating them to values, the values are compared and `true` or `false` are returned accordingly.

$$\text{gt} \frac{\llbracket e1 \rrbracket \sigma\phi = v1 \quad \llbracket e2 \rrbracket \sigma\phi = v2 \quad v1 > v2}{\langle e1 > e2, \sigma, \phi \rangle \Longrightarrow \text{true}} \quad (3.7)$$

$$\text{gt} \frac{\llbracket e1 \rrbracket \sigma\phi = v1 \quad \llbracket e2 \rrbracket \sigma\phi = v2 \quad v1 \not> v2}{\langle e1 > e2, \sigma, \phi \rangle \Longrightarrow \text{false}} \quad (3.8)$$

Equations (3.7) and (3.8) describes how equality comparisons between two expressions are performed. After evaluating them to values, the values are checked for equality and `true` or `false` are returned accordingly.

$$\text{add} \frac{\llbracket e1 \rrbracket \sigma\phi = v1 \quad \llbracket e2 \rrbracket \sigma\phi = v2 \quad v1 + v2 = v3}{\langle e1 + e2, \sigma, \phi \rangle \Longrightarrow v3} \quad (3.9)$$

$$\text{sub} \frac{\llbracket e1 \rrbracket \sigma\phi = v1 \quad \llbracket e2 \rrbracket \sigma\phi = v2 \quad v1 - v2 = v3}{\langle e1 - e2, \sigma, \phi \rangle \Longrightarrow v3} \quad (3.10)$$

$$\text{mul} \frac{\llbracket e1 \rrbracket \sigma\phi = v1 \quad \llbracket e2 \rrbracket \sigma\phi = v2 \quad v1 * v2 = v3}{\langle e1 * e2, \sigma, \phi \rangle \Longrightarrow v3} \quad (3.11)$$

Equations (3.9) to (3.11) describe how the binary operations of addition, subtraction and multiplication between expressions are handled. In these operations the expressions of the operands are first reduced to values and then the operator is applied to the reductions. Since these operations do not contemplate failure, they are handled in a single case.

$$\text{div} \frac{\llbracket e1 \rrbracket \sigma\phi = v1 \quad \llbracket e2 \rrbracket \sigma\phi = v2 \quad v1/v2 = v3}{\langle e1 / e2, \sigma, \phi \rangle \Longrightarrow v3} \quad (3.12)$$

$$\text{div} \frac{\llbracket e2 \rrbracket \sigma\phi = v2 \quad v2 \equiv 0}{\langle e1 / e2, \sigma, \phi \rangle \Longrightarrow \text{DIVBYZERO}} \quad (3.13)$$

Equations (3.12) and (3.13) cover how to evaluate the binary operation of division. The two cases are to reason about invalid executions, such as Equation (3.13), that yields a division by zero error.

3.2.3 Semantics of Complex Expressions

Function declaration, function application, and conditionals are what we refer to as *complex* expressions.

$$\text{let-func} \frac{\phi[f \leftarrow p, e1] = \phi' \quad \llbracket e2 \rrbracket \sigma \phi' = v2}{\langle \text{let } fp = e1 \text{ in } e2, \sigma, \phi \rangle \Longrightarrow v2} \quad (3.14)$$

$$\text{let-const} \frac{p \equiv \text{nil} \quad \llbracket e1 \rrbracket \sigma \phi = v1 \quad \sigma[f \leftarrow v1] = \sigma' \quad \llbracket e2 \rrbracket \sigma' \phi = v2}{\langle \text{let } fp = e1 \text{ in } e2, \sigma, \phi \rangle \Longrightarrow v2} \quad (3.15)$$

The semantics in [Equations \(3.14\) and \(3.15\)](#) is the semantics of let-style expressions, consisting of the evaluation of a declaration followed by another expression. A declaration is defined by its identifier x and a list of parameters p . The two semantics distinguish constant declarations from function declarations, which are managed respectively by adding the variable-value binding to the environment and by adding the function body and parameter list to the function store. Expression e_2 is then evaluated under the new environment or function store. In typical functional languages, declarations are managed differently, in a single environment, but, for the sake of simplicity, we decided to use this “easier to digest” approach.

$$\text{application} \frac{\begin{array}{l} \phi(f) = n, e \quad \lambda x. \llbracket x \rrbracket \sigma \phi = r \\ \{n_i \leftarrow r(p_i) \mid n_i \in n, p_i \in p\} = \sigma' \\ \sigma \oplus \sigma' = \sigma'' \quad \llbracket e \rrbracket \sigma'' \phi = v \end{array}}{\langle fp, \sigma, \phi \rangle \Longrightarrow v} \quad (3.16)$$

[Equation \(3.16\)](#) describes the semantics of function application. The function is looked up in ϕ , obtaining parameters names n and the function body e . The parameters expressions p are then reduced to values and then bound to their names in the environment σ . The auxiliary operator $\oplus$ adds them to the old environment, producing a new environment σ' .

$$\text{if-true} \frac{\llbracket c \rrbracket \sigma \phi = \text{true} \quad \llbracket e1 \rrbracket \sigma \phi = v}{\langle \text{if } c \text{ then } e1 \text{ else } e2, \sigma, \phi \rangle \Longrightarrow v} \quad (3.17)$$

$$\text{if-false} \frac{\llbracket c \rrbracket \sigma \phi = \text{false} \quad \llbracket e2 \rrbracket \sigma \phi = v}{\langle \text{if } c \text{ then } e1 \text{ else } e2, \sigma, \phi \rangle \Longrightarrow v} \quad (3.18)$$

[Equations \(3.17\) and \(3.18\)](#) is the canonical if-then-else. A comparison expression is evaluated, then, according to the result, one between e_t or e_f is evaluated.

3.3 INTRODUCING ABSTRACTIONS

Having introduced the concrete semantics of our toy language, it is now time to start reasoning about how these semantics can be transformed into analysis-computable ones, e.g., a static analyzer. A typical goal of static analyzers is to prove program properties, and one way to visualize this is to think of program properties as sets of inputs, outputs, or program configurations that satisfy a given property.

Consider the example program in [Listing 3.3](#): we are interested in proving a property that, for instance, might be termination, given

```

1 let move x by times =
2   if times = 0 then x
3   else move (x + by) by (times - 1)
4 move x by times

```

Listing 3.3: Example of a potentially diverging program.

that the example program is clearly diverging for negative inputs on parameter `times`. In these examples we use $\mathbb{Z}_+$ as a shorthand for $\{z \in \mathbb{Z} \mid z \geq 0\}$. We can express the set of terminating input environments with the following set Σ_t :

$$\Sigma_t = \{\sigma \mid \sigma(x), \sigma(by) \in \mathbb{Z} \wedge \sigma(times) \in \mathbb{Z}_+\} \quad (3.19)$$

We might also express input environments causing diverging executions as $\Sigma_\perp$:

$$\{\sigma \mid \sigma(x), \sigma(by) \in \mathbb{Z} \wedge \sigma(times) \in \mathbb{Z} \setminus \mathbb{Z}_+\} \quad (3.20)$$

And we can also express all possible input environments as $\Sigma_\top$:

$$\Sigma_\top = \{\sigma \mid \sigma(x), \sigma(by), \sigma(times) \in \mathbb{Z}\} \quad (3.21)$$

Note that $\Sigma_\top$ is equal to the set denoted by $\Sigma_t \cup \Sigma_\perp$. Intuitively, the sets described, endowed with the set inclusion $\sqsubseteq$ relation (reads as ‘is smaller than’), form a partially ordered set, or *poset*. This structure is widely used to reason about program properties, and is foundational when reasoning about abstractions in general. As per our examples, we can state: (i) $\Sigma_t \sqsubseteq \Sigma_\top$, (ii) $\Sigma_\perp \sqsubseteq \Sigma_\top$.

Definition 5. A partially ordered set (poset) is an ordered pair $P = (X, \sqsubseteq)$ consisting of a set X and a partial order $\sqsubseteq$ on X . In case of no ambiguity, X itself can be called a poset. Partial order relations, in general, for all $a, b, c \in X$:

- reflexive: $a \sqsubseteq a$;
- antisymmetric: $a \sqsubseteq b \wedge b \sqsubseteq a \implies a = b$;
- transitive: $a \sqsubseteq b \wedge b \sqsubseteq c \implies a \sqsubseteq c$.

3.3.1 Abstract Environment

Rice’s theorem states that no non-trivial semantic property is decidable for all programs in a sufficiently expressive language. Termination is such a property, and our concrete semantics is sufficiently expressive (e.g., Turing-complete); therefore, termination is undecidable in the general case. The concrete semantics yields an undecidable property checking problem. To obtain a terminating and sound static analysis, we must reason about multiple potential program executions simultaneously. We achieve this by “collapsing” multiple concrete objects into an abstract representative. Considering the previously defined environment sets, we define a poset $P(\mathcal{V} \rightarrow \mathbb{Z}, \sqsubseteq_\sigma)$ of environments. From this definition of environment, we can obtain a poset $\mathcal{V} \rightarrow P(\mathbb{Z}, \sqsubseteq_\sigma^\#)$ that, for each variable in the environment, keeps knowledge of a set of potential integer values. This definition is an *abstraction* of the environment. As a convention

Example 1. Given a set of environments Σ :

$$\Sigma = \{(x \rightarrow 4, y \rightarrow 2); (x \rightarrow 42, y \rightarrow 24)\}$$

We can obtain $\alpha(\Sigma)$ such as:

$$\alpha(\Sigma) = \{x \rightarrow (4, 42), y \rightarrow (2, 24)\}$$

By concretizing $\alpha(\Sigma)$ with $\gamma(\alpha(\Sigma))$ we obtain:

$$\begin{aligned} \gamma(\alpha(\Sigma)) = & \{(x \rightarrow 4, y \rightarrow 2); (x \rightarrow 4, y \rightarrow 24); \\ & (x \rightarrow 42, y \rightarrow 2); (x \rightarrow 42, y \rightarrow 24)\} \end{aligned}$$

Notice that Σ is strictly included in $\gamma(\alpha(\Sigma))$. Sound γ and α functions should always follow this property.

throughout this work, when dealing with posets of abstract objects, we will denote them with the $\#$ symbol. As it is computable, the function store ϕ does not need abstractions. The function that maps a concrete entity to an abstract entity is an abstraction function α . In the case of environments, we have:

$$\alpha_\sigma \begin{cases} P(\mathcal{V} \rightarrow \mathbb{Z}, \sqsubseteq) & \rightarrow \mathcal{V} \rightarrow P(\mathbb{Z}, \sqsubseteq) \\ \Sigma & \rightarrow \lambda v. \{\sigma(v) \mid \sigma \in \Sigma\} \end{cases}$$

It is possible to also define the dual of the abstraction function, called *concretization*, γ .

$$\gamma_\sigma \begin{cases} \mathcal{V} \rightarrow P(\mathbb{Z}, \sqsubseteq) & \rightarrow P(\mathcal{V} \rightarrow \mathbb{Z}, \sqsubseteq) \\ f & \rightarrow \{\sigma \mid \forall v \in \mathcal{V}, \sigma(v) \in f(v)\} \end{cases}$$

Example 1 provides an example of how γ and α functions behave, and how the use of abstraction α leads to the introduction of environments not originally in Σ when concretizing the abstracted set. In case we were interested in proving Σ as the only possible set of valid environments, the abstraction function used would produce false positives. Adopting more precise abstractions can mitigate this issue, at the expense of the tractability of the problem.

The property of α and γ of preserving order relations can be generalized as a Galois connection, which is how abstract interpretation was originally formulated in [CC77].

Definition 6 (Galois Connection). A *Galois connection* between two complete lattices $(C, \sqsubseteq)$ (concrete domain) and $(A, \sqsubseteq)$ (abstract domain) consists of two monotonic functions:

$$\begin{aligned} \alpha : C &\rightarrow A \quad (\text{abstraction function}) \\ \gamma : A &\rightarrow C \quad (\text{concretization function}) \end{aligned}$$

satisfying for all $c \in C$ and $a \in A$:

$$\alpha(c) \sqsubseteq a \iff c \subseteq \gamma(a)$$

[CC77] Cousot and Cousot, “Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints”

A GC is denoted by: $(C, \sqsubseteq) \xleftrightarrow[\alpha]{\gamma} (A, \sqsubseteq)$.

Theorem 1 (Soundness Preservation). *Given a Galois connection (α, γ) and a concrete transfer function $f : C \rightarrow C$, if an abstract transfer function $f^\sharp : A \rightarrow A$ satisfies:*

$$\forall c \in C : \alpha(f(c)) \sqsubseteq f^\sharp(\alpha(c))$$

then for any concrete computation, the abstract computation soundly approximates all concrete behaviors.

Proof. Let $f : C \rightarrow C$ be the big-step concrete semantics, and $f^\sharp : A \rightarrow A$ be its abstract counterpart. Assume $\alpha(f(c)) \sqsubseteq f^\sharp(\alpha(c))$ holds for all $c \in C$ (i.e., local soundness). Then, for any concrete input $c \in C$, if $c' = f(c)$, then:

$$\alpha(c') = \alpha(f(c)) \sqsubseteq f^\sharp(\alpha(c))$$

thus, the abstraction of the concrete result is over-approximated by the abstract evaluation. $\square$

Property 1 (Key Characteristics). For any Galois connection (α, γ) :

- α and γ are monotonic.
- $\alpha \circ \gamma$ is extensive: $a \sqsubseteq \alpha(\gamma(a))$.
- $\gamma \circ \alpha$ is reductive: $\gamma(\alpha(c)) \supseteq c$.
- The composition $\gamma \circ \alpha : C \rightarrow C$ is a *closure operator* on C .
- A GC makes it is always possible to mechanically derive the *best abstraction* $f^\sharp$ of a function f by $f^\sharp : A \rightarrow A = \alpha \circ f \circ \gamma$.

We show an example of GC between the poset of integers $P(\mathbb{Z}, \sqsubseteq)$ and the interval poset $(\mathcal{I}, \sqsubseteq)$, and an example of application in [Example 2](#).

Example 2 (Interval Analysis). Let $C = (P(\mathbb{Z}), \sqsubseteq)$ and $A = (\mathcal{I}, \sqsubseteq)$ where $[a, b] \sqsubseteq [c, d] \iff c \leq a \wedge b \leq d$. Define:

$$\begin{aligned} \alpha(S) &= [\min S, \max S] \\ \gamma([a, b]) &= \{x \in \mathbb{Z} \mid a \leq x \leq b\} \\ &\text{where } S \text{ is a set of integers.} \end{aligned}$$

For the concrete operation $f(S) = \{x + 1 \mid x \in S\}$, we propose a sound abstract counterpart:

$$f^\sharp([a, b]) = [a + 1, b + 1]$$

recalling the best abstraction of a function f : $f^\sharp$ satisfies the best abstraction property: demonstrating exact abstraction.

Property 2 (Best Abstraction). Galois connections can also be exploited to derive the best abstraction for operators: Given a Galois connection between $(C, \sqsubseteq)$ and $(A, \sqsubseteq)$ and a function $f \in C \rightarrow C$, the best abstraction of f is $\alpha \circ f \circ \gamma$.

3.3.2 Intervals As an Abstraction of Values

Our abstraction for environments represents directly sets of integers, and as such is still incomputable. An abstraction that solves this issue is to represent sets of integers as intervals. Same as environments, intervals are definable as posets. We define the (already introduced) interval abstract domain as a poset $(\mathcal{I}, \subseteq^\#)$ and provide an extensive definition of its properties in [Definition 7](#).

Definition 7 (Interval Domain). We define an interval as a tuple of values belonging to integers, with the additional values representing negative and positive infinities. Furthermore, to represent non-defined computations, a $\perp_{\mathcal{I}}$ is added as a special interval, representing the *infimum*, the smallest possible value. We also leverage infinities so that we can have a *supremum*, the order-wise biggest possible value.

$$\mathcal{I} = \{[l, u] \mid l \in \mathbb{Z} \cup \{-\infty\}, u \in \mathbb{Z} \cup \{+\infty\}\} \cup \{\perp_{\mathcal{I}}\}$$

$$[l, u] \begin{cases} [l, u] \iff l \leq u \\ \perp_{\mathcal{I}} \end{cases}$$

$$\mathcal{I}_1 \subseteq^\# \mathcal{I}_2 \begin{cases} \mathcal{I}_1 = \perp_{\mathcal{I}} \implies \text{true} \\ \mathcal{I}_2.l \leq \mathcal{I}_1.l \wedge \mathcal{I}_1.u \leq \mathcal{I}_2.u \implies \text{true} \\ \text{false} \end{cases}$$

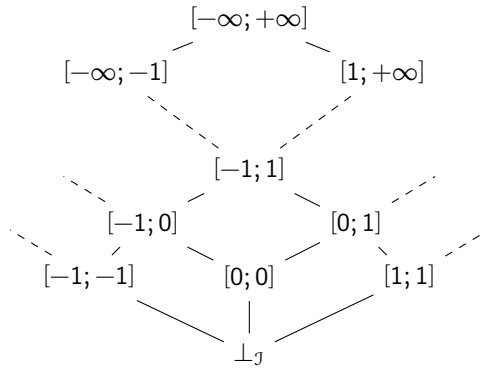


FIGURE 3.1: Hasse diagram for the Interval domain. Continuous lines connect from the bottom to the top elements having a $\subseteq^\#$ relationship.

In [Figure 3.1](#) we show what ‘bigger’ and ‘smaller’ mean in the order sense: reading from the bottom to top of the diagram, continuous lines represent the holding of the partial order relationship $\subseteq^\#$.

To close the definition of our interval domain, we define the GC between $\mathbb{Z}$ and $\mathcal{I}$.

Definition 8 (Galois Connection of $\mathbb{Z}$ and $\mathcal{I}$).

$$\alpha_{\mathcal{I}} \begin{cases} \mathcal{P}(\mathbb{Z}) \rightarrow & \mathcal{I} \\ \emptyset \rightarrow & \perp_{\mathcal{I}} \\ S \rightarrow & [\min S, \max S] \end{cases}$$

$$\gamma_J \begin{cases} J \rightarrow & \mathcal{P}(\mathbb{Z}) \\ \perp_J \rightarrow & \emptyset \\ [l, u] \rightarrow & \{z \in \mathbb{Z} \mid l \leq z \leq u\} \end{cases}$$

We now define our environments in terms of intervals.

Definition 9 (Abstract Environments). By point-wise lifting of $(J, \sqsubseteq_i)$ we can obtain an abstract environment:

$$(\mathcal{V} \dot{\rightarrow} J, \dot{\sqsubseteq}_i)$$

that is an abstraction w.r.t. all the previous definition of environments.

Definition 10 (GC of Environments and Abstract Environments). Galois connections allow for composition. As such, we define our relation from concrete environments to abstract environments by composition of GCs. Given the GCs:

$$(P(\mathbb{V} \rightarrow \mathbb{Z}), \subseteq) \xleftrightarrow[\alpha]{\gamma} (\mathbb{V} \rightarrow P(\mathbb{Z}), \subseteq)$$

and

$$(\mathbb{V} \rightarrow P(\mathbb{Z}), \subseteq) \xleftrightarrow[\alpha_z]{\gamma_z} (\mathcal{V} \dot{\rightarrow} J, \dot{\sqsubseteq}_i)$$

by composition we define:

$$(P(\mathbb{V} \rightarrow \mathbb{Z}), \subseteq) \xleftrightarrow[\alpha_i \circ \alpha]{\gamma \circ \gamma_i} (\mathcal{V} \dot{\rightarrow} J, \dot{\sqsubseteq}_i)$$

that is the GC between concrete and abstract environments.

3.3.3 Abstract Booleans

Our language uses Boolean values to represent the results of comparison operations. The interval abstraction for values introduced above might lead to unsound comparisons (i.e., the result of a comparison can be both True and False), as will be shown in [Example 3](#) if boolean operations are left as they are defined in the concrete, and we ought to address this issue. We address the issue by abstracting the evaluation from Booleans to evaluations to sets of Booleans. We define our abstraction of Booleans as a poset $(\mathbb{B}^\#, \subseteq_{\mathbb{B}^\#}^\#)$:

Definition 11 (Abstract Booleans).

$$\begin{aligned} \top_{\mathbb{B}^\#} &= \{\text{True}, \text{False}\} \\ \perp_{\mathbb{B}^\#} &= \emptyset \\ \mathbb{B}^\# &= \{\text{True}\} \cup \{\text{False}\} \cup \perp_{\mathbb{B}^\#} \cup \top_{\mathbb{B}^\#} \\ \mathbb{B}_1^\# \subseteq_{\mathbb{B}^\#}^\# \mathbb{B}_2^\# &\begin{cases} \mathbb{B}_1^\# = \perp_{\mathbb{B}^\#} & \implies \text{true} \\ \mathbb{B}_2^\# = \top_{\mathbb{B}^\#} & \implies \text{true} \\ \mathbb{B}_1^\# = \mathbb{B}_2^\# & \implies \text{true} \\ & \text{false} \end{cases} \end{aligned}$$

The set $\{\text{True}, \text{False}\}$, $\top_{\mathbb{B}^\#}$ as a shorthand, represents the result of boolean computations that can be either True or False. Boolean computations can never be undefined, but $\perp_{\mathbb{B}^\#}$ is still added so that we can exploit the properties of complete lattices.

Example 3. Given $a = [1, 5]$, $b = [0, 6]$, $a \leq b$, what is the result of the comparison? For example, we can state that $a \leq b$ holds true if every element in the concretization of a is smaller or equal to any element in the concretization of b . In logic notation:

$$a \leq b \iff \forall (l \in \gamma_I(a), r \in \gamma_I(b)), l \leq r \quad (3.22)$$

Conversely, we can apply the same principle for the negated comparison. In this example, neither is applicable: picking the value 1 from a , and value 0 from b , $1 \leq 0$ is false. This happens because we are evaluating a set of potential comparisons; hence, we must cover the case of uncertain results. This abstraction of Booleans allows us to express that the inequality of this example can evaluate to either True or False, whilst still allowing us to express comparisons like in Equation (3.22).

The representation as Hasse diagram for the abstract Boolean domain is shown in Figure 3.2.

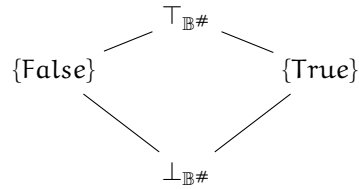


FIGURE 3.2: Hasse diagram for the abstract boolean domain.

To close the definition of our abstract Booleans, we define their abstraction and concretization functions in terms of a trivial Galois connection.

Definition 12.

$$\alpha_{B\#} \left\{ \begin{array}{ll} B \rightarrow & B\# \\ \emptyset \rightarrow & \perp_{B\#} \\ \{b \in B\} \rightarrow & \{b\} \\ \{\text{True}, \text{False}\} \rightarrow & T_{B\#} \end{array} \right.$$

$$\gamma_{B\#} \left\{ \begin{array}{ll} B\# \rightarrow & B \\ \perp_{B\#} \rightarrow & \emptyset \\ \{b \in B\} \rightarrow & \{b\} \\ T_{B\#} \rightarrow & \{\text{True}, \text{False}\} \end{array} \right.$$

3.3.4 Interval-based Abstract Operators

ARITHMETIC Performing arithmetic on intervals requires more nuance compared to performing the same operations on integer values. Fortunately, we can exploit ‘shortcuts’ provided by the property of Galois connection introduced in Property 2:

Definition 13. Let $[l_1, u_1]$ and $[l_2, u_2]$ be two non-empty intervals and $f_{\mathbb{Z}}$ a function $(\mathbb{Z} \times \mathbb{Z}) \rightarrow \mathbb{Z}$. We compute the best abstraction for the operation between the posets of type $\mathbb{Z}$ with:

$$\begin{aligned} & \alpha(f_{\mathbb{Z}}(\gamma([l_1, u_1]), \gamma([l_2, u_2]))) \\ &= \alpha(\mathbb{S}) \\ &= [\min(\mathbb{S}), \max(\mathbb{S})] \end{aligned}$$

where $\mathbb{S} = \{f(x, y) \mid l_1 \leq x \leq u_1, l_2 \leq y \leq u_2\}$.

In this low-level interval arithmetic approach, operations concerning $\perp_{\mathcal{J}}$ always return $\perp_{\mathcal{J}}$. While these computations are undefined, this behavior and similar ones, such as division by zero, will be caught by the interpreter.

Using [Definition 13](#), we define the operators needed for the arithmetic in `SlobSinopie`: division $/_{\mathcal{J}}$, multiplication $\times_{\mathcal{J}}$, addition $+_{\mathcal{J}}$, and subtraction $-_{\mathcal{J}}$.

COMPARISON

Definition 14. Let $[l_1, u_1]$ and $[l_2, u_2]$ be two non-empty intervals, and $f_{\mathbb{B}}$ a function $(\mathbb{Z} \times \mathbb{Z}) \rightarrow \mathbb{B}$. We compute the best abstraction for the operation between the posets of type $\mathbb{Z}$ with:

$$\begin{aligned} & \alpha(f_{\mathbb{B}}(\gamma([l_1, u_1]), \gamma([l_2, u_2]))) \\ &= \alpha(\mathbb{B}) \\ &= \sqcup_{\mathbb{B}^{\#}}(\mathbb{B}) \end{aligned}$$

where $\mathbb{B} = \{f_{\mathbb{B}}(x, y) \mid l_1 \leq x \leq u_1, l_2 \leq y \leq u_2\}$.

Operations concerning $\perp_{\mathcal{J}}$ are $\perp_{\mathbb{B}^{\#}}$.

Using [Definition 14](#), we define the operators needed for the comparisons operations defined in `SlobSinopie`: smaller, $<_{\mathcal{J}}$, equals $=_{\mathcal{J}}$, smaller or equal $\leq_{\mathcal{J}}$.

RECAP To produce a computable abstraction of our target language, we have so far introduced means of abstracting numerical values and their relative arithmetic, abstraction for Booleans, and abstraction of program environments. What we will introduce next are: **(i)** the fixpoint operator `fix`, necessary for computing (potentially) recursive function definitions and loops alike, **(ii)** the filter operator, used to narrow the state space in conditional statements, obtaining more precise analyses with little cost, and **(iii)** our abstract computable semantics.

3.3.5 Abstract Semantics of *SlobSinopie*

It is now time to introduce the abstract semantics for our language. As an abstract construct, this type of semantics allows us to reason about multiple possible evaluations. We can leverage the already introduced abstraction for environment and values to define abstractions of the structures we introduced previously.

Definition 15. An abstract variable environment $\widehat{\sigma}$ of type $\mathcal{M}^\#$ is defined as a mapping from variables to values abstracted as intervals. Formally, $\mathcal{M}^\# : \mathcal{V} \rightarrow \mathcal{I}$.

We abstract potential errors as sets of type $\mathcal{E}^\#$ and define the semantic domain for our evaluation function $\mathcal{E}^\#$.

Definition 16. The evaluation function $\mathcal{E}^\#[\![\cdot]\!]$ evaluates the input tuple of type $\langle \mathcal{P} \times \mathcal{M}^\# \times \mathcal{F} \rangle$ to an abstract output of type $\langle \mathcal{I} \times \mathcal{E}^\# \rangle$. With respect to [Definition 3](#), the domain of intervals already contains the $\perp$ value (indicating non-termination), so it is left out in this definition. $\mathcal{E}^\#[\![\cdot]\!]$ corresponds to the transition relation $\xRightarrow{\#}$.

We also define a new semantic domain for the boolean evaluation function $\mathcal{C}^\#[\![\cdot]\!]$.

Definition 17. The boolean evaluation function $\mathcal{C}^\#[\![\cdot]\!]$ evaluates the input tuple of type $\langle \mathcal{P} \times \mathcal{M}^\# \times \mathcal{F} \rangle$ to an abstract output or non-output of type $\mathbb{B}^\# \cup \mathcal{E}^\#$.

3.3.6 Abstract Semantics of Simple Expressions

$$\text{const} \frac{\llbracket z \rrbracket \widehat{\sigma} \phi = \widehat{v}}{\langle z, \widehat{\sigma}, \phi \rangle \xRightarrow{\#} \widehat{v}, \emptyset} \quad (3.23)$$

[Equation \(3.23\)](#) is the function that evaluates constant numerical expressions. This function can never generate an error, so we return the empty set together with the interval abstraction of the singleton containing the constant value z .

$$\text{lookup} \frac{\widehat{\sigma}(x) = \widehat{v}}{\langle x, \widehat{\sigma}, \phi \rangle \xRightarrow{\#} \widehat{v}, \emptyset} \quad (3.24)$$

[Equation \(3.24\)](#) evaluates variable expressions. It performs a lookup on the variable environment $\widehat{\sigma}$, retrieving and returning the value associated with the identifier. Again, this function can never fail, and the empty set is returned as the right hand side part of the tuple together with the value corresponding to variable x .

3.3.7 Abstract Semantics of Operations Between Expressions

$$\text{eq-true} \frac{\begin{array}{l} \llbracket e1 \rrbracket \widehat{\sigma} \phi = \widehat{v1}, \widehat{f1} \quad \llbracket e2 \rrbracket \widehat{\sigma} \phi = \widehat{v2}, \widehat{f2} \\ \forall v1 \in \gamma_i(v1), v2 \in \gamma_i(v2) : v1 \equiv v2 \end{array}}{\langle e1 = e2, \widehat{\sigma}, \phi \rangle \xRightarrow{\#} \text{True}, \widehat{f1} \cup \widehat{f2}} \quad (3.25)$$

$$\text{eq-false} \frac{\begin{array}{l} \llbracket e1 \rrbracket \widehat{\sigma} \phi = \widehat{v1}, \widehat{f1} \quad \llbracket e2 \rrbracket \widehat{\sigma} \phi = \widehat{v2}, \widehat{f2} \\ \forall v1 \in \gamma_i(v1), v2 \in \gamma_i(v2) : v1 \not\equiv v2 \end{array}}{\langle e1 = e2, \widehat{\sigma}, \phi \rangle \xRightarrow{\#} \text{False}, \widehat{f1} \cup \widehat{f2}} \quad (3.26)$$

$$\text{eq-uncertain} \frac{\llbracket e1 \rrbracket \widehat{\sigma} \phi = \widehat{v1}, \widehat{f1} \quad \llbracket e2 \rrbracket \widehat{\sigma} \phi = \widehat{v2}, \widehat{f2}}{\langle e1 = e2, \widehat{\sigma}, \phi \rangle \xRightarrow{\#} \top_{\mathbb{B}^\#}, \widehat{f1} \cup \widehat{f2}} \quad (3.27)$$

[Equations \(3.25\) to \(3.27\)](#) describes how to perform abstract equality comparisons between two expressions. These abstract semantics were

mechanically derived using the best abstraction property of GCs. After evaluating the two expressions to abstract values and capturing the potential failures in $\widehat{f1}$ and $\widehat{f2}$, all the concretized abstract values are compared for equality, returning *True* or *False* accordingly. These semantics, as described in their naive instance, are correct, but we can optimize for performance. For example, it is obvious in Equation (3.25) that an abstract equality comparison in a non-relational domain, such as the domain interval, can never be *True* unless the concretized intervals can concretize into two equal singletons (i.e., $[1; 1]$ and $[1; 1]$). Whereas, in Equation (3.26), we can assert by a trivial intuition that, if the intersection $\gamma_i(v1) \cap \gamma_i(v2)$ is empty, the values can never be equal. Formally:

$$\gamma_i(v1) \cap \gamma_i(v2), c1, c2 \begin{cases} \emptyset, _, _ & \longrightarrow \text{False} \\ \gamma_i(v1), 1, 1 & \longrightarrow \text{True} \\ & \longrightarrow \top_{\mathbb{B}\#} \end{cases}$$

where $c1$ and $c2$ are the cardinalities of $\gamma_i(v1)$ and $\gamma_i(v2)$ respectively. With this insight, we rewrite the optimized versions of the mechanically derived Equations (3.25) and (3.26), with Equation (3.27) remaining the same.

$$\text{eq-true} \frac{\begin{array}{c} \llbracket e1 \rrbracket \widehat{\sigma} \phi = \widehat{v1}, \widehat{f1} \quad \llbracket e2 \rrbracket \widehat{\sigma} \phi = \widehat{v2}, \widehat{f2} \\ \text{card}(\gamma_i(v1)) = 1 \quad \text{card}(\gamma_i(v2)) = 1 \quad \gamma_i(v1) \equiv \gamma_i(v2) \end{array}}{\langle e1 = e2, \widehat{\sigma}, \phi \rangle \xRightarrow{\#} \text{True}, \widehat{f1} \cup \widehat{f2}} \quad (3.28)$$

$$\text{eq-false} \frac{\begin{array}{c} \llbracket e1 \rrbracket \widehat{\sigma} \phi = \widehat{v1}, \widehat{f1} \quad \llbracket e2 \rrbracket \widehat{\sigma} \phi = \widehat{v2}, \widehat{f2} \\ \gamma_i(v1) \cap \gamma_i(v2) = \emptyset \end{array}}{\langle e1 = e2, \widehat{\sigma}, \phi \rangle \xRightarrow{\#} \text{False}, \widehat{f1} \cup \widehat{f2}} \quad (3.29)$$

We prove the two approaches are equivalent and, therefore, the correctness of the shortcut in Section 3.3.7.

Proof. Let $\widehat{a}, \widehat{b} \in \mathbb{I}$, and let the concrete equality be:

$$==: \mathcal{P}(\mathbb{Z}) \times \mathcal{P}(\mathbb{Z}) \rightarrow \{\text{true}, \text{false}\}$$

defined for singleton elements as:

$$x == y = \begin{cases} x = y \longrightarrow \text{true} \\ x \neq y \longrightarrow \text{false} \end{cases}$$

The best correct abstraction of equality, by the best abstraction property of Galois connection, is:

$$a ==^{\#} b = \alpha(\{x == y \mid x \in \gamma(a), y \in \gamma(b)\})$$

- If $\gamma(a) = \gamma(b) = \{c\}$, then the set of comparisons is $\{c == c\} = \{\text{true}\}$, hence $a ==^{\#} b \xRightarrow{\#} \text{true}$
- If $\gamma(a) \cap \gamma(b) = \emptyset$, then for all $x \in \gamma(a), y \in \gamma(b), x \neq y$, hence $x == y = \text{false}$, and so the result is $\{\text{false}\}$, and $a ==^{\#} b \xRightarrow{\#} \text{false}$

- Otherwise, there exist $x \in \gamma(a), y \in \gamma(b)$ such that $x = y$, and others such that $x \neq y$, so the result set is $\{\text{true}, \text{false}\}$, and thus $a ==^\# b \xRightarrow{\#} \top_{\mathbb{B}^\#}$

This matches exactly the proposed shortcut definition. Therefore, the shortcut is equivalent to the best correct abstraction of concrete equality as defined by the best abstraction property of Galois Connections. $\square$

$$\text{leq-true} \frac{\begin{array}{l} \llbracket e1 \rrbracket \hat{\sigma} \phi = \hat{v}1, \hat{f}1 \quad \llbracket e2 \rrbracket \hat{\sigma} \phi = \hat{v}2, \hat{f}2 \\ \forall v1 \in \gamma_i(v1), v2 \in \gamma_i(v2) : v1 \leq v2 \end{array}}{\langle e1 \leq e2, \hat{\sigma}, \phi \rangle \xRightarrow{\#} \text{True}, \hat{f}1 \cup \hat{f}2} \quad (3.30)$$

$$\text{leq-false} \frac{\begin{array}{l} \llbracket e1 \rrbracket \hat{\sigma} \phi = \hat{v}1, \hat{f}1 \quad \llbracket e2 \rrbracket \hat{\sigma} \phi = \hat{v}2, \hat{f}2 \\ \forall v1 \in \gamma_i(v1), v2 \in \gamma_i(v2) : v1 > v2 \end{array}}{\langle e1 \leq e2, \hat{\sigma}, \phi \rangle \xRightarrow{\#} \text{False}, \hat{f}1 \cup \hat{f}2} \quad (3.31)$$

$$\text{leq-false} \frac{\begin{array}{l} \llbracket e1 \rrbracket \hat{\sigma} \phi = \hat{v}1, \hat{f}1 \quad \llbracket e2 \rrbracket \hat{\sigma} \phi = \hat{v}2, \hat{f}2 \end{array}}{\langle e1 \leq e2, \hat{\sigma}, \phi \rangle \xRightarrow{\#} \top_{\mathbb{B}^\#}, \hat{f}1 \cup \hat{f}2} \quad (3.32)$$

Equations (3.30) to (3.32) describe how the less-or-equal comparisons between two expressions are performed. Similarly to the equality comparison, after evaluating them to values, the values are compared and True, False, or $\top_{\mathbb{B}^\#}$ are returned accordingly. Similarly to the equality semantics, this comparison also enjoys a shortcut solution. For instance,

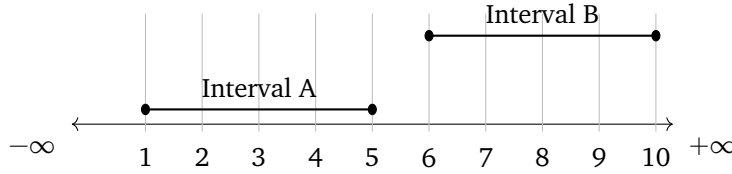


FIGURE 3.3: Visualization of two disjoint integer intervals on a number line.

consider the interval visualization in Figure 3.3. Interval A and B are disjoint, and, intuitively, B is always greater than A. We can express this in terms of maximum and minimums of A and B:

$$A \leq B \begin{cases} A.\min \leq B.\min \wedge A.\max \leq B.\max & \rightarrow \text{True} \\ A.\min > B.\min \wedge A.\max > B.\max & \rightarrow \text{False} \\ & \rightarrow \top_{\mathbb{B}^\#} \end{cases}$$

conversely, for the ‘greater than’ comparison:

$$A > B \begin{cases} A.\min > B.\min \wedge A.\max > B.\max & \rightarrow \text{True} \\ A.\min \leq B.\min \wedge A.\max \leq B.\max & \rightarrow \text{False} \\ & \rightarrow \top_{\mathbb{B}^\#} \end{cases}$$

In light of this, we rewrite the optimized semantics of the less or equal comparison, such that enumeration of the concretization of abstract values is avoided, and write the greater than comparison in an already optimized form.

$$\text{leq-true} \frac{\llbracket e1 \rrbracket \hat{\sigma} \phi = \hat{v1}, \hat{f1} \quad \llbracket e2 \rrbracket \hat{\sigma} \phi = \hat{v2}, \hat{f2} \quad v1.min \leq v2.min \wedge v1.max \leq v2.max}{\langle e1 \leq e2, \hat{\sigma}, \phi \rangle \xRightarrow{\#} \text{True}, \hat{f1} \cup \hat{f2}} \quad (3.33)$$

$$\text{leq-false} \frac{\llbracket e1 \rrbracket \hat{\sigma} \phi = \hat{v1}, \hat{f1} \quad \llbracket e2 \rrbracket \hat{\sigma} \phi = \hat{v2}, \hat{f2} \quad v1.min > v2.min \wedge v1.max > v2.max}{\langle e1 \leq e2, \hat{\sigma}, \phi \rangle \xRightarrow{\#} \text{False}, \hat{f1} \cup \hat{f2}} \quad (3.34)$$

$$\text{gt-false} \frac{\llbracket e1 \rrbracket \hat{\sigma} \phi = \hat{v1}, \hat{f1} \quad \llbracket e2 \rrbracket \hat{\sigma} \phi = \hat{v2}, \hat{f2} \quad v1.min > v2.min \wedge v1.max > v2.max}{\langle e1 > e2, \hat{\sigma}, \phi \rangle \xRightarrow{\#} \text{True}, \hat{f1} \cup \hat{f2}} \quad (3.35)$$

$$\text{gt-true} \frac{\llbracket e1 \rrbracket \hat{\sigma} \phi = \hat{v1}, \hat{f1} \quad \llbracket e2 \rrbracket \hat{\sigma} \phi = \hat{v2}, \hat{f2} \quad v1.min \leq v2.min \wedge v1.max \leq v2.max}{\langle e1 > e2, \hat{\sigma}, \phi \rangle \xRightarrow{\#} \text{False}, \hat{f1} \cup \hat{f2}} \quad (3.36)$$

$$\text{gt-false} \frac{\llbracket e1 \rrbracket \hat{\sigma} \phi = \hat{v1}, \hat{f1} \quad \llbracket e2 \rrbracket \hat{\sigma} \phi = \hat{v2}, \hat{f2}}{\langle e1 > e2, \hat{\sigma}, \phi \rangle \xRightarrow{\#} \top_{\mathbb{B}^{\#}}, \hat{f1} \cup \hat{f2}} \quad (3.37)$$

$$\text{add} \frac{\llbracket e1 \rrbracket \hat{\sigma} \phi = \hat{v1}, \hat{f1} \quad \llbracket e2 \rrbracket \hat{\sigma} \phi = \hat{v2}, \hat{f2} \quad \hat{v1} +^{\#} \hat{v2} = \hat{v3}}{\langle e1 + e2, \hat{\sigma}, \phi \rangle \xRightarrow{\#} \hat{v3}, \hat{f1} \cup \hat{f2}} \quad (3.38)$$

$$\text{sub} \frac{\llbracket e1 \rrbracket \hat{\sigma} \phi = \hat{v1}, \hat{f1} \quad \llbracket e2 \rrbracket \hat{\sigma} \phi = \hat{v2}, \hat{f2} \quad \hat{v1} -^{\#} \hat{v2} = \hat{v3}}{\langle e1 - e2, \hat{\sigma}, \phi \rangle \xRightarrow{\#} \hat{v3}, \hat{f1} \cup \hat{f2}} \quad (3.39)$$

$$\text{mul} \frac{\llbracket e1 \rrbracket \hat{\sigma} \phi = \hat{v1}, \hat{f1} \quad \llbracket e2 \rrbracket \hat{\sigma} \phi = \hat{v2}, \hat{f2} \quad \hat{v1} *^{\#} \hat{v2} = \hat{v3}}{\langle e1 * e2, \hat{\sigma}, \phi \rangle \xRightarrow{\#} \hat{v3}, \hat{f1} \cup \hat{f2}} \quad (3.40)$$

Equations (3.38) to (3.40) describe how the abstract operations of addition, subtraction and multiplication between expressions are handled. Same as their concrete counterparts, in these operations the expressions of the operands are first reduced to abstract values and then the abstract operators ($+^{\#}$, $-^{\#}$, $*^{\#}$) are applied to the reductions. Again, since these operations do not contemplate failure, they are handled in a single case.

$$\text{div} \frac{\llbracket e1 \rrbracket \hat{\sigma} \phi = \hat{v1}, \hat{f1} \quad \llbracket e2 \rrbracket \hat{\sigma} \phi = \hat{v2}, \hat{f2} \quad \gamma_i(\hat{v2}) \equiv \{0\}}{\langle e1 / e2, \hat{\sigma}, \phi \rangle \xRightarrow{\#} \perp_j, \hat{f1} \cup \hat{f2} \cup \{\text{DIVBYZERO}\}} \quad (3.41)$$

$$\text{div} \frac{\llbracket e1 \rrbracket \hat{\sigma} \phi = \hat{v1}, \hat{f1} \quad \llbracket e2 \rrbracket \hat{\sigma} \phi = \hat{v2}, \hat{f2} \quad \hat{v1} /^{\#} \hat{v2} = \hat{v3} \quad 0 \in \gamma_i(\hat{v2})}{\langle e1 / e2, \hat{\sigma}, \phi \rangle \xRightarrow{\#} \hat{v3}, \hat{f1} \cup \hat{f2} \cup \{\text{DIVBYZERO}\}} \quad (3.42)$$

$$\text{div} \frac{\llbracket e1 \rrbracket \hat{\sigma} \phi = \hat{v1}, \hat{f1} \quad \llbracket e2 \rrbracket \hat{\sigma} \phi = \hat{v2}, \hat{f2} \quad \hat{v1} /^{\#} \hat{v2} = \hat{v3} \quad 0 \notin \gamma_i(\hat{v2})}{\langle e1 / e2, \hat{\sigma}, \phi \rangle \xRightarrow{\#} \hat{v3}, \hat{f1} \cup \hat{f2}} \quad (3.43)$$

Equations (3.41) to (3.43) cover the cases of abstract division. Equation (3.41) covers the ‘certain’ division by zero and propagates other possible errors from the abstract evaluations of $e1$ and $e2$. Equation (3.42) covers the case of the possible, *maybe*, division by zero, and Equation (3.43) covers the cases of division where having 0 as a divisor is not possible.

3.3.8 Abstract Semantics of Complex Expressions

$$\text{let-func} \frac{\phi[f \leftarrow p, e1] = \phi' \quad \llbracket e2 \rrbracket \hat{\sigma} \phi' = \hat{v}, \hat{f}}{\langle \text{let } fp = e1 \text{ in } e2, \hat{\sigma}, \phi \rangle \xRightarrow{\#} \hat{v}2, \hat{f}} \quad (3.44)$$

$$\text{let-const} \frac{p \equiv \text{nil} \quad \llbracket e1 \rrbracket \hat{\sigma} \phi = \hat{v}1, \hat{f}1 \quad \hat{\sigma}[f \leftarrow \hat{v}1] = \hat{\sigma}' \quad \llbracket e2 \rrbracket \hat{\sigma}' \phi = \hat{v}2, \hat{f}2}{\langle \text{let } fp = e1 \text{ in } e2, \sigma, \phi \rangle \xRightarrow{\#} \hat{v}2, \hat{f}1 \cup \hat{f}2} \quad (3.45)$$

The semantics in [Equations \(3.44\) and \(3.45\)](#) are the abstract counterpart of our let-style expressions. They are almost identical to their concrete counterpart, and, therefore, need no further explanations.

$$\text{application} \frac{\begin{array}{l} \phi(f) = n, e \quad \lambda x. \llbracket x \rrbracket \hat{\sigma} \phi = r^\sharp \\ \{n_i \leftarrow r^\sharp(p_i) \mid n_i \in n, p_i \in p\} = \hat{\sigma}', \hat{f}1 \\ \hat{\sigma} \oplus \hat{\sigma}' = \hat{\sigma}'' \quad \llbracket e \rrbracket \hat{\sigma}'' \phi = \hat{v}, \hat{f}2 \end{array}}{\langle fp, \sigma, \phi \rangle \xRightarrow{\#} \hat{v}, \hat{f}1 \cup \hat{f}2} \quad (3.46)$$

[Equation \(3.46\)](#) describes the abstract semantics of function application. It is very close to its concrete counterpart, except for the part related to the propagation of possible errors.

$$\text{if-true} \frac{\llbracket c \rrbracket \hat{\sigma} \phi = \text{True}, \hat{f}1 \quad \llbracket e1 \rrbracket \hat{\sigma} \phi = \hat{v}, \hat{f}2}{\langle \text{if } c \text{ then } e1 \text{ else } e2, \hat{\sigma}, \phi \rangle \xRightarrow{\#} \hat{v}, \hat{f}1 \cup \hat{f}2} \quad (3.47)$$

$$\text{if-false} \frac{\llbracket c \rrbracket \hat{\sigma} \phi = \text{False}, \hat{f}1 \quad \llbracket e2 \rrbracket \hat{\sigma} \phi = \hat{v}, \hat{f}2}{\langle \text{if } c \text{ then } e1 \text{ else } e2, \hat{\sigma}, \phi \rangle \xRightarrow{\#} \hat{v}, \hat{f}1 \cup \hat{f}2} \quad (3.48)$$

$$\text{if-both} \frac{\begin{array}{l} \llbracket c \rrbracket \hat{\sigma} \phi = \top_{\mathbb{B}^\sharp}, \hat{f}1 \\ \text{filter}(c, \hat{\sigma}) = \hat{\sigma}_t \quad \text{filter}(\neg c, \hat{\sigma}) = \hat{\sigma}_f \\ \llbracket e1 \rrbracket \hat{\sigma}_t \phi = \hat{v}1, \hat{f}2 \quad \llbracket e2 \rrbracket \hat{\sigma}_f \phi = \hat{v}2, \hat{f}3 \end{array}}{\langle \text{if } c \text{ then } e1 \text{ else } e2, \sigma, \phi \rangle \xRightarrow{\#} \hat{v}1 \sqcup_i \hat{v}2, \hat{f}1 \cup \hat{f}2 \cup \hat{f}3} \quad (3.49)$$

[Equations \(3.47\) and \(3.48\)](#) cover the if-then-else. Specifically, they cover the cases in which the condition of the conditional statement can evaluate only to True and False, respectively. Besides the propagation of possible failures, it does not differ much from its concrete counterpart. Instead, the uncertain version in [Equation \(3.49\)](#) applies an assumption to the abstract environment via the filter auxiliary function (we will introduce filter in the following paragraph). In one case, it applies the condition that evaluates as True (i.e., $\text{filter}(c, \hat{\sigma})$), producing all possible environments in which the evaluation of the then branch happens. Conversely, we produce another abstract environment by applying filter to the negated condition. The results and errors from the evaluations of the branches are then joined by the interval join operator $\sqcup_i$, joining errors as well. The join operator is one of the core elements that allow analyses based on abstract interpretation to scale, as it allows for the ‘collapse’ of the results of flow joins into a single flow, such as the results of if-then-else computations.

Definition 18 (Interval Join Operator). Let $\mathcal{J}$ be the domain of intervals with bottom. We define the *join operator* $\sqcup_{\mathcal{J}} : \mathcal{J} \times \mathcal{J} \rightarrow \mathcal{J}$ as follows:

$$x \sqcup_{\mathcal{J}} y = \begin{cases} y & \text{if } x = \perp \\ x & \text{if } y = \perp \\ [\min(a_1, a_2), \max(b_1, b_2)] & \text{if } x = [a_1, b_1], y = [a_2, b_2] \end{cases}$$

That is:

- The join of any interval with $\perp_{\mathcal{J}}$ (empty interval) yields the interval itself.
- The join of two defined intervals is the smallest interval containing both.

This operator is the least upper bound of the interval lattice. As such, it is:

- **Commutative:** $x \sqcup_{\mathcal{J}} y = y \sqcup_{\mathcal{J}} x$
- **Associative:** $(x \sqcup_{\mathcal{J}} y) \sqcup_{\mathcal{J}} z = x \sqcup_{\mathcal{J}} (y \sqcup_{\mathcal{J}} z)$
- **Idempotent:** $x \sqcup_{\mathcal{J}} x = x$
- **Monotonic:** $x \sqsubseteq_{\mathcal{J}} x \sqcup_{\mathcal{J}} y$ and $y \sqsubseteq_{\mathcal{J}} x \sqcup_{\mathcal{J}} y$

3.3.9 *Filter(c, σ)*

The `filter()` auxiliary function is, intuitively, a function that, given a condition c , (i.e., $x > 5$) and an environment σ , produces a σ' where the condition is always either True or $\top_{\mathbb{B}}$, in cases where there is no certainty (i.e., comparison between two variables). This function aims to improve the precision of the analysis potentially significantly, adding little computational cost. Consider the following environment, condition, and negation of condition:

$$\begin{aligned} \sigma &= [\quad x \rightarrow [0; 10], \\ &\quad y \rightarrow [1; 15] \quad] \\ c &= x > 5 \quad \neg c = x \leq 5 \end{aligned}$$

At the intuitive level, what `filter()` would do by applying c and $\neg c$ is producing the following environments:

$$\begin{aligned} \text{filter}(c, \sigma) &= \sigma_t = [\quad x \rightarrow [6; 10], \\ &\quad y \rightarrow [1; 15] \quad] \\ \text{filter}(\neg c, \sigma) &= \sigma_f = [\quad x \rightarrow [0; 5], \\ &\quad y \rightarrow [1; 15] \quad] \end{aligned}$$

How it manages to do so is by performing the *meet* of two environments: one is ‘implicit’ (the condition), and one is explicit. The *meet* $\sqcap$ (intersection in sets) is the dual operator of the *join* operator, and it computes the greatest lower bound.

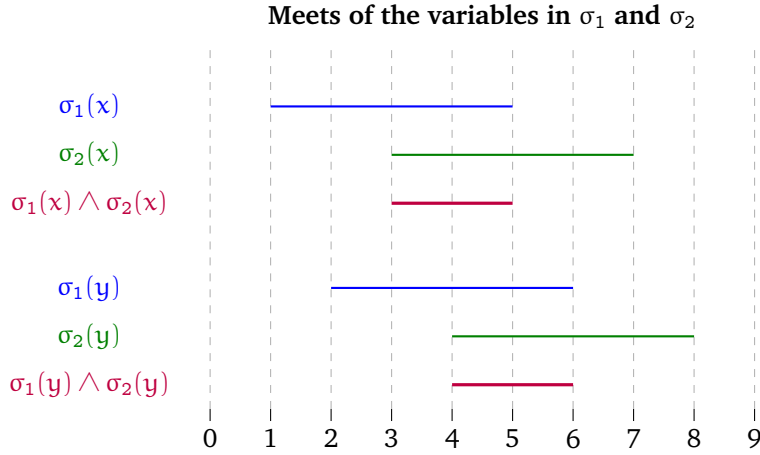
Definition 19 (Meet of Intervals). Let two intervals be defined as:

$$I_1 = [a_1, b_1], \quad I_2 = [a_2, b_2]$$

Then their **meet** $\sqcap_i$ is defined as:

$$I_1 \sqcap_i I_2 = \begin{cases} [\max(a_1, a_2), \min(b_1, b_2)] & \text{if } \max(a_1, a_2) \leq \min(b_1, b_2) \\ \perp_{\mathcal{I}} & \text{otherwise} \end{cases}$$

$$\begin{aligned} \sigma_1 &= \{x \mapsto [1, 5], \quad y \mapsto [2, 6]\} \\ \sigma_2 &= \{x \mapsto [3, 7], \quad y \mapsto [4, 8]\} \\ \sigma_1 \wedge \sigma_2 &= \{x \mapsto [3, 5], \quad y \mapsto [4, 6]\} \end{aligned}$$



The meet, as we did with the join, can be point-wise lifted to $\sqcap_i$ so that it operates on environments. It is now a matter of finding a way to transform a condition into an environment. We preface this explanation by stating that two environments are ‘compatible’ (i.e., can be *met*, *joined*, or *widened*) with each other as long as they track knowledge of the same set of variables. With this clarified, let us consider an environment σ with just x as variable:

$$\sigma = [x \rightarrow [0; +\infty]]$$

An alternative representation, more suited for this explanation, is an environment with variable x satisfying $x \geq 0$. In this representation, a condition c can be treated as a constraint on the abstract environment σ' having the same variables as σ , but with no prior knowledge besides the one provided by c .

Example 4 (Conditions as Environments). Given the following environment, condition, and negated condition:

$$\begin{aligned} \sigma &= [\quad x \rightarrow [0; 10], \\ &\quad y \rightarrow [4; 20], \\ &\quad z \rightarrow [6; 66] \quad] \\ c &= x > 5 \quad \neg c = x \leq 5 \end{aligned}$$

We define the environment described by c as the environment with no bounds on y, z and x with an inferior bound only:

$$\begin{aligned} \sigma &= [\quad x \rightarrow [6; +\infty], \\ &\quad y \rightarrow \top_{\mathcal{I}}, \\ &\quad z \rightarrow \top_{\mathcal{I}} \quad] \end{aligned}$$

and, conversely, the environment described by $\neg c$ as:

$$\begin{aligned}\sigma = [& \ x \rightarrow [-\infty; 5], \\ & \ y \rightarrow \top_J, \\ & \ z \rightarrow \top_J \]\end{aligned}$$

Following the sketch in [Example 4](#), it should now be clear how filter is just the point-wise lifted meet operator $\hat{\sqcap}_i$ on environments applied to an abstract environment $\hat{\sigma}$ and an implicit abstract environment with its bounds described by a condition c . We now address, before formalizing filter, how filter should behave when variable-to-variable conditions are concerned (i.e., $x > y$).

Example 5 (Variable-to-Variable filtering). Consider the environment σ , condition c and its negation $\neg c$:

$$\begin{aligned}\sigma = [& \ x \rightarrow [0; 10], \\ & \ y \rightarrow [4; 20], \\ & \ z \rightarrow [6; 66] \] \\ c = x > y \quad \neg c = x \leq y\end{aligned}$$

The desired behavior is to obtain a σ' such that the evaluation of $x >^\# y$ is either True or $\top_{\mathbb{B}^\#}$. Conversely, we want a σ'' such that $x \leq^\# y$ is either True or $\top_{\mathbb{B}^\#}$. The variables that this operation impacts are obviously x and y .

Let our goal G be $\{\text{True}, \top_{\mathbb{B}^\#}\}$, and our starting variable x , and its comparison target y and an abstract comparison function f with semantics defined in [Equations \(3.35\) to \(3.37\)](#) (semantics of greater than). We obtain a σ' such that the result of the comparison is in G by the following means:

$$\sigma' = \sigma[x \rightarrow \alpha_i(\{v \iff f(\alpha_i(v), y) \in G \mid v \in \gamma_i(x)\})]$$

that in this example produces:

$$\begin{aligned}\sigma' = [& \ \mathbf{x} \rightarrow [5; 10], \\ & \ y \rightarrow [4; 20], \\ & \ z \rightarrow [6; 66] \]\end{aligned}$$

But this yet only impacts variable x of the abstract environment. We perform so the same operations but considering this time the newly produced σ' and the *inverted* comparison $y <^\# x$.

$$\sigma'' = \sigma'[y \rightarrow \alpha_i(\{v \iff f(\alpha_i(v), x) \in G \mid v \in \gamma_i(y)\})]$$

that produces:

$$\begin{aligned}\sigma' = [& \ \mathbf{x} \rightarrow [5; 10], \\ & \ \mathbf{y} \rightarrow [4; 9], \\ & \ z \rightarrow [6; 66] \]\end{aligned}$$

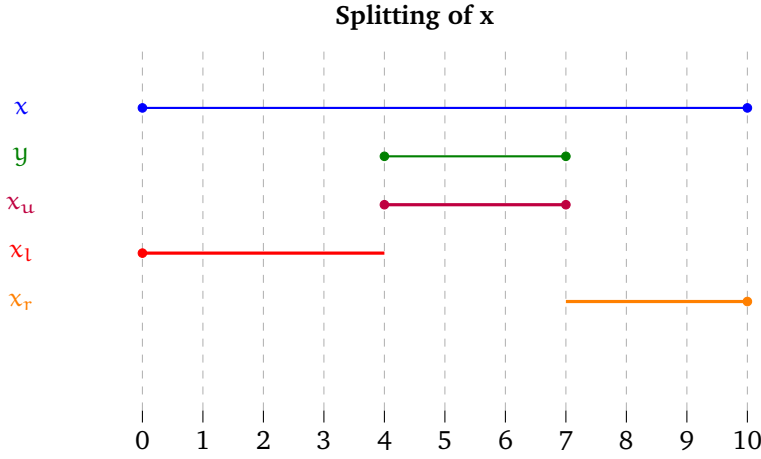
In this specific example though, applying these steps to $\neg c$ does not help in reducing the size of the abstract environment.

Now that we have provided the reader with the intuition of how `filter()` works, we introduce a more general theory, first by example, and then we generalize it.

Example 6. Let the environment σ , and condition c be:

$$\begin{aligned}\sigma &= [\quad x \rightarrow [0; 10], \\ &\quad y \rightarrow [4; 7] \quad] \\ c &= x > y\end{aligned}$$

and evaluation goal G be $\{\text{True}, \top_{\mathbb{B}^{\#}}\}$. Our evaluation result is in G when x is certain to be greater than y and when it is uncertain. We obtain the ‘uncertain’ sub-interval of x by $x_u = x \sqcap_i y$, we obtain the remaining left side of x by $x_l = [x.\text{inf}; \text{pred}(x_u.\text{inf})]$ and, conversely, the right side $x_r = [\text{succ}(x_l.\text{sup}); x.\text{sup}]$. Visualized (the dots indicate that the value is included in the interval):



From this view, it is clear that x_l and x_r tell us the values for which x is sure to be greater or lower than y , that x_u tells us the uncertainty region, and corresponds as well to the potential equality region.

Definition 20 (Theory of `filter()`). Let σ be an abstract environment with variables x and y . Let c be a condition. The *primitive* that allows for `filter()` to happen is the **complement** of two intervals, defined as:

$$\text{comp } x \ y = \begin{aligned} &\alpha_i\{l < r \mid l \in \gamma_i(x), r \in \gamma_i(y)\}, \\ &\alpha_i\{l > r \mid l \in \gamma_i(x), r \in \gamma_i(y)\} \end{aligned} \quad (3.50)$$

Intuitively, the left complement of two intervals represents the relation $x < y$, conversely the right complement represents $x > y$, and the **meet** of two intervals represents the relation $x = y$. In order to represent the $\leq$ or $\geq$ relations one must consider the **meet** of the two underlying intervals and either the left or right complement.

3.3.10 Addressing Non-termination

One limitation of the abstract semantics that we just laid out is that termination is not guaranteed. While this is normal for expressive programming languages (i.e., some programs need to run forever, for instance),

this is not desirable for semantics oriented to static analysis, as we are interested in having analysis results in **finite** time. So far, we have never employed in our semantics the tool and the operators that make this possible: a fixpoint algorithm paired with the right *widening* operator(s). Informally, widening operators are operators performing *extrapolations*. We start our journey in understanding how to enforce convergence in the analysis by first explaining the most basic widening operators needed. Then, in [Section 3.3.11](#) we explain how fixpoint algorithms operate.

Definition 21 (Interval Widening Operator). Let $\mathcal{J}$ be the abstract domain of intervals with bottom, as defined previously. The *widening operator* $\nabla : \mathcal{J} \times \mathcal{J} \rightarrow \mathcal{J}$ is used to ensure termination of fix point computations in the presence of potentially infinite ascending chains (e.g., due to loops or recursion).

Given two intervals $x = [a_1, b_1]$ and $y = [a_2, b_2]$, their widening is defined as:

$$x \nabla y = \begin{cases} \perp & \text{if } x = \perp \\ x & \text{if } y = \perp \\ [a', b'] & \text{otherwise} \end{cases}$$

where:

$$a' = \begin{cases} a_1 & \text{if } a_2 \geq a_1 \\ -\infty & \text{otherwise} \end{cases} \quad \text{and} \quad b' = \begin{cases} b_1 & \text{if } b_2 \leq b_1 \\ \infty & \text{otherwise} \end{cases}$$

This operator is designed to detect and generalize growth on both upper and lower bound:

- If the lower bound decreases (i.e., $a_2 < a_1$), it is widened to $-\infty$.
- If the upper bound increases (i.e., $b_2 > b_1$), it is widened to $+\infty$.
- Otherwise, the original bound is preserved.

Intuitively, the widening operator ∇ is an over-approximation of the union of x and y . Widening is typically used during abstract interpretation to accelerate convergence when computing least fixed points, especially in loops where interval bounds might not stabilize by just applying an upper bound operator. It is also a mandatory operator to guarantee convergence of analysis when working with lattices of infinite height, such as the interval lattice.

Definition 22 (Environment Widening Operator). Let ∇ be the widening operator for intervals. The *widening operator* for environments can be defined as $\dot{\nabla}$ by point-wise lifting of the $\mathcal{J}$ poset to $\mathbb{V} \rightarrow \mathcal{J}$, obtaining $\dot{\nabla} : \mathbb{V} \rightarrow \mathcal{J} \times \mathbb{V} \rightarrow \mathcal{J} \rightarrow \mathbb{V} \rightarrow \mathcal{J}$. We use this operator to ensure that the recurrent calls to the abstract interpreter (i.e., tuples of expressions and pre-conditions) to be evaluated are always growing, so that convergence of analysis can be enforced.

3.3.11 Fixpoint Algorithms for Big-step Interpreters

Big-step abstract interpreters elegantly mirror big-step semantics, but naively written ones will diverge on potentially diverging programs, unlike their small-step counterparts, which operate over finite transition

systems and have built-in loop detection capabilities (i.e., small-step interpreters usually work on explicit control-flow graphs as opposed to proceeding by induction on the syntax). We must therefore adopt more sophisticated algorithms for fixed-point iteration that ensure termination by construction when we are dealing with big-step abstract interpretation. The foundational work regarding fix-point algorithms for big-step abstract interpreters is described in [Sch95]. More modern approaches to computing fix points have been studied more recently, where big-step interpreters have been more of interest ([KEH23; Bod+19; Dar+17; WCR19; KE19]). We describe how fix-point algorithms for big-step interpreters work by *borrowing* some examples from [KEH23], which extends the foundational work in [Sch95], and combine it with chaotic iteration [Bou93]. These algorithms need to satisfy the following three properties to ensure termination (convergence) of the analysis:

- (i) ability to detect recurrent calls;
- (ii) ability to enforce recurrence in all potentially infinite call chains;
- (iii) ability to be able to converge the result iteration process in a finite number of steps.

Before diving into the nuances of how this family of fixed-point algorithms works, we first define what a *call* is in this context. A *call* refers to a call to the interpreter, and it is an evaluation of a given program expression under a given precondition. In the case of *SlobSinopie*, a *call* is an evaluation of an expression e under a variable environment σ and function store ϕ .

3.3.12 Three Termination Conditions

Consider the function `fact`, the factorial function. The actions we should

```

1 let fact n =
2   if n = 0 then 1
3   else if n = 1 then 1
4   else n * fact (n - 1)

```

take when analyzing such a function, one that features recursion, are fundamentally the following three:

1. **Cutting off recurrent calls.** Whenever the interpreter encounters a call whose syntactic position and abstract environment have already appeared on the current derivation branch, evaluation must stop expanding that branch and treat the result of the call as a (yet-to-be-determined) *unknown*, a $\perp$ value. Otherwise, even simple programs such as the interval analysis of `fact` would diverge.
2. **Forcing recurrence in every infinite chain.** Merely cutting off exact repeats is not enough. The abstract arguments in the environment or state can grow strictly (e.g. $[-\infty, -2] \sqsubset [-\infty, -1] \sqsubset \dots$ in the negative factorial example), preventing any exact repetition. The algorithm must therefore *widen* arguments so that every potentially infinite call chain eventually hits a recurrent node. This

[Sch95] Schmidt, “Natural-Semantics-Based Abstract Interpretation (Preliminary Version)”

[KEH23] Keidel et al., “Combinator-Based Fixpoint Algorithms for Big-Step Abstract Interpreters”

[Bod+19] Bodin et al., “Skeletal semantics and their interpretations”

[Dar+17] Darais et al., “Abstracting definitional interpreters (functional pearl)”

[WCR19] Wei et al., “Staged abstract interpreters: fast and modular whole-program analysis via meta-programming”

[KE19] Keidel and Erdweg, “Sound and reusable components for abstract interpretation”

[Bou93] Bourdoncle, “Efficient chaotic iteration strategies with widenings”

argument widening is based on the ∇ operator. What it does, intuitively, is leverage information about the *biggest* environment under which an expression has been evaluated and, if such an expression is evaluated again, widen the known one accordingly. If the invocation's environment is *smaller* than the known one, the known environment is returned as-is so that smaller environments are not used for evaluation more than once.

3. **Bounding per-node iteration.** After identifying recurrent nodes, their associated abstract equations are solved by iteration. To avoid non-termination in infinite ascending chains (e.g. $[1, 1] \sqsubset [1, 2] \sqsubset \dots$), each equation must be iterated only a finite number of times, typically enforced by a *result widening* (i.e., an imperative language will implement a state-based widening, a functional language will widen a value-based one). This *result widening* corresponds, in our example language, to the previously defined ∇ operator.

The widening of arguments and result **must** over-approximate the join operator and accelerate convergence (they eventually stabilize each ascending chain).

3.3.13 Global Termination Guarantee

Theorem 2 (Termination of Big-Step Fixpoint Algorithms). *Let every semantic rule have finite branching degree. This is standard for conventional big-step evaluation rules of programming languages. If a fix-point algorithm satisfies Conditions 3.3.12 1–3, then its evaluation tree is finite; hence the algorithm terminates.*

Proof sketch. C1 and C2 jointly ensure that every branch of the derivation tree is finite: either it ends in a base value, or it is cut at a recurrent node introduced by argument widening. C3 bounds the number of iterations performed at each node. With finite branching, König's lemma yields a finite tree, so the algorithm terminates. A full proof is given by [KEH23]. $\square$

[KEH23] Keidel et al., “Combinator-Based Fixpoint Algorithms for Big-Step Abstract Interpreters”

3.3.14 Running Examples

Consider again the function `fact`.

```

1 let fact n =
2   if n = 0 then 1
3   else if n = 1 then 1
4   else n * fact (n - 1)
```

We are interested in performing the interval (value) analysis of `fact([0; +∞])`, which is the evaluation of the body of `fact` under environment $\sigma = [n \mapsto [0; +\infty]]$. The first recurrent call detected is

$$\langle [n \mapsto [0, \infty]], \text{fact}(n) \rangle \quad \text{and} \quad \langle [n \mapsto [0, \infty]], \text{fact}(n) \rangle$$

and the algorithm cuts the self-loop (satisfying C1). If the argument interval were to shrink strictly, the stack-widening operator ∇_{Stack} lifts

it back to the earlier interval, ensuring recurrence (C2). The resulting equation $X = [1, 1] \sqcup (X + [1, 1])$ (with the first $[1, 1]$ coming from the interval join of the results of `if-then` and `else-if` branches) is then iterated with value widening ∇ so that X stabilizes after finitely many steps (C3).

Definition 23 (Argument widening). Let $\rho : \mathbb{R}$ be a *stack* function mapping expressions $e \in E$ and environments $\sigma \in \Sigma$ under which they are evaluated. We define the lookups on such function as:

$$\rho(e) = \begin{cases} \sigma & \text{if } \{e \leftarrow \sigma\} \in \rho \\ \perp & \text{otherwise} \end{cases}$$

Each time a (potentially diverging) expression is evaluated, the environment corresponding to $\rho(e)$ is widened with σ , producing a new environment $\sigma' = \rho(e) \dot{\nabla} \sigma$ and a new *widened* stack, $\rho' = \rho\{e \leftarrow \sigma'\}$. This operator is defined as ∇_{Stack} , formally:

$$\begin{aligned} \nabla_{\text{Stack}} : \mathbb{R} \times (E \times \Sigma) &\rightarrow \mathbb{R} \times (E \times \Sigma) \\ \rho \nabla_{\text{Stack}} (e, \sigma) &\begin{cases} \rho, (e, \sigma) & \text{if } \rho(e) \neq \perp \wedge \sigma \sqsubseteq \rho(e) \\ \rho\{e \leftarrow \sigma'\}, (e, \sigma') & \text{if } \rho(e) \neq \perp \wedge \sigma \not\sqsubseteq \rho(e) \\ \rho\{e \leftarrow \sigma\}, (e, \sigma) & \text{otherwise} \end{cases} \\ &\text{where } \sigma' = \rho(e) \dot{\nabla} \sigma \end{aligned}$$

3.3.15 Summary

Section 3.3.11 distilled the essence of Section 2.1 of [KEH23]: termination for big-step abstract interpretation hinges on (i) identifying recurring control points, (ii) guaranteeing their existence via widening, and (iii) limiting fixpoint iterations for any given control point. The next section builds on these foundations to present a practical terminating analysis that realizes the three conditions by iterating over strongly-connected sub-graphs of the dynamically discovered graph of calls to the interpreter.

3.4 AN APPETIZER ON ANALYZING SLOB SINOPIE PROGRAMS

With the abstract semantics from **Section 3.3.5** and the *recipe* to build fix-point algorithms from **Section 3.3.11**, we can now make an abstract interpreter capable of working with loops and recursion. As can be deduced from the abstract semantics, we can be interested in proving what range of values `SlobSinopie` programs return. We may also be interested in proving that a given program is free from errors. This category, in our toy example, is represented by the division by zero error.

As an example, consider the program in **Listing 3.4**. Let's pretend for a moment that we allow an `input()` construct in our syntax. Intuitively, the program accepts an input, passed to the function f as the *divisor* (`dvr`) parameter. By doing so, we can emulate non-determinism; this, of course, allows for arbitrary numbers to act as a divisor, including the

```

1 let f dvd dvr =
2   dvd / dvr in
3 f 10 input ()

```

Listing 3.4: Example program that may yield an error.

‘culprit’ 0. In Figure 3.4, we follow a simple intuition on how a naive (non-terminating) abstract interpreter may analyze this toy example, which we call P . The formal parameter of the declaration is called p as a shorthand, same as the expression, called $e1$, whereas $f\ 10\ input\ ()$ is called $e2$.

```

1  $eval^e \langle \rho_1, \sigma_1, (P) \rangle$ 
2    $eval^d \langle \rho_1, \sigma_1, (let\ f\ p = e1) \rangle$ 
3      $\Downarrow^d \langle \rho_2, \sigma_1 \rangle$  where  $\rho_2 = \rho_1[f \leftarrow p, e1]$ 
4    $eval^e \langle \rho_2, \sigma_1, (e2) \rangle$ 
5      $eval^e \langle \rho_2, \sigma_2, (e1) \rangle$  where  $\sigma_2 = \{dvr \mapsto [-\infty; +\infty]; dvd \mapsto [10; 10]\}$ 
6        $\Downarrow^e \langle \top, \{DIVBYZERO\} \rangle$ 
7        $\Downarrow^e \langle \top, \{DIVBYZERO\} \rangle$ 
8      $\Downarrow^e \langle \top, \{DIVBYZERO\} \rangle$ 

```

FIGURE 3.4: Trace of the abstract interpreter analyzing the program in Listing 3.4.

In Figure 3.4, we call $eval^e$ the calls to the expression interpreter and $eval^d$ the calls to the interpreter of declarations, each with its own $\Downarrow$ that represents the termination of a given call to the interpreter. The evaluation that generates the error is clearly $eval^e \langle \rho_2, \sigma_2, (e1) \rangle$, corresponding to $[10; 10]/[-\infty; +\infty]$. We propagate both the result $\top$ and the error to the outer evaluations: the first for the call corresponding to expression $e2$ and then to the evaluation of the program P . This brief and intuitive example, together with the semantics described earlier in Section 3.3.5, allows one to grasp the essence of this kind of analysis, which can be easily augmented by lifting the error type to a tuple containing the error itself and the expression it was generated by. By this intuition, we can correctly infer that the analysis of an error-free program will have a value on the left-hand side of the result tuple and an empty set on the right-hand side. For this reason, from now on, we will refer to such tuples as if they were values.

3.5 ANALYSIS OF SLOB SINOPIE PROGRAMS

To perform the analysis of `SlobSinopie` programs, we first need an interpreter. We have already defined the semantics of our language, so those should act as a reference for how the interpreter operates. In Listing 3.5 we sketch our abstract interpreter in pseudo-OCaml, whereas in Listing 3.1 we provide a sketch of how the function `fix`, corresponding to the fixpoint algorithm described previously, works. This algorithm is a slightly revised version³ of the monolithic fixpoint algorithm presented in [KEH23]. Regarding Listing 3.1, we preserve the original notation in pseudo-Haskell as opposed to writing in pseudo-OCaml. As can be observed from the code examples, the abstract interpreter `eval` mutu-

³Special thanks to Dr. Sven Keidel for the fruitful conversation on the topic and in helping better address the issue.

[KEH23] Keidel et al., “Combinator-Based Fixpoint Algorithms for Big-Step Abstract Interpreters”

```

1 let binop env l r f funcs stack cache =
2   let vall, c', scgl = fix (env,l) funcs stack cache in
3   let valr, c'', scgr = fix (env,r) funcs stack c' in
4   let vals = f vall valr env in
5   (valr, c'', Set.union scgl scgr)
6
7 let app env f p funcs stack cache =
8   let pars, fb = Map.lookup funcs f in
9   (*compute fixpoints for all parameters p,
10    then bind to pars and make env'*)
11   let env' = ... in
12   fix (env',fb) funcs stack cache
13
14 let decl d env funcs = ...
15
16 let rec eval (env,expr as call) funcs stack cache =
17   match expr with
18   | Decl (e1,e2) ->
19     let env', funcs' = decl e1 env funcs in
20     eval (env', e2) funcs' stack cache
21   | Add (l,r) -> binop env l r Maths.add funcs stack cache
22   | Div (l,r) -> binop env l r Maths.div funcs stack cache
23   | App (f,p) -> app env f p funcs stack cache
24   | ...

```

Listing 3.5: Sketch of a big-step abstract interpreter.

ally recursively calls `fix`. In this implementation, `eval` is required to reason only about the semantics and is relieved of any reasoning about detecting, cutting, and iterating on recursive calls to the interpreter. Such functionalities are embedded in `fix` (detection, cutting of recursion) and `iterate` (iteration). `fix` does also discriminate between surely non-recursive expressions (i.e. addition `Add`) and potentially recursive ones (i.e., function application `App`).

Now that we have introduced these two fundamental components, we can finally show how an abstract interpreter would handle the analysis of more *interesting* programs, possibly with interesting inputs as well.

In [Figure 3.5](#) we show the analysis of a call to `fib` under abstract environment $\{x \mapsto [1; \infty]\}$, in short `fib [1;  $\infty$ ]`. In order to show how the algorithm operates, there are arrows in the trace, with $\Downarrow$ representing a return, $\Join$ an iteration, and $\Downarrow$ a call. For example, $\Downarrow \langle c, \rho, \kappa \rangle$ represents a call to the interpreter for call `c`, with stack ρ and cache κ . $\Downarrow \langle v, \kappa, \theta \rangle$ represents a return from a call to the interpreter with value v , cache κ , and SCG θ . At times, along the trace, the return value v is shown in terms of expressions that evaluate into intervals. In the beginning, the algorithm explores the recursive calls of the `fib` function until it hits the recurrent calls (lines 2, 4, and 5 in [Figure 3.5](#)). In these cases, the algorithm returns $\perp$ to avoid non-termination and adds the call to the SCG (line 13 in [Listing 3.1](#)). Later, the algorithm returns to the call of `fib[0,  $\infty$ ]` (line 6 in [Figure 3.5](#)) and iterates because the call is in SCG set θ_2 and the result interval has grown from $\perp$ to $[0,1]$ (lines 22 and 28 in [Listing 3.1](#)). The following iteration propagates the new analysis result `fib[0,  $\infty$ ]  $\mapsto [0, 1]$` throughout the inner SCG. After returning to call `fib[0,  $\infty$ ]` (line 10 in [Figure 3.5](#)), the result did not grow, and the algorithm returns to the surrounding call `fib[1,  $\infty$ ]`, removing call `fib[0,  $\infty$ ]` from the SCG set (line 30 in [Listing 3.1](#)). Since the result for `fib[1,  $\infty$ ]` has grown from

Listing 3.1: Big-step fixpoint algorithm iterating on the innermost SCG

```

1 Stack = Map Expr → Env
2 Cache = Map Call → (Stable, Val)
3 Funcs = Map String → (String List, Expr)
4 SCG = Set Call
5 Call = (Env, Expr)
6 Stable = Stable | Unstable
7
8 fix :: Call → Funcs → Stack → Cache → (Val, Cache, SCG)
9 fix call funcs stack cache =
10   | not (loopable call) = eval funcs call stack cache
11   | callw ∈ cache && stab == Stable = (valC, cache, ∅)
12   | callw ∈ cache && stab == Unstable = (valC, cache, {callw})
13   | callw ∉ cache && callw ∈ stackw && sm = (⊥, cache, {callw})
14   | otherwise = iterate funcs callw stackw cache
15   where (stab, valC) = cache(call)
16         (stackw, callw) = stack ∇Stack call
17         sm = stack == stackw
18
19 iterate :: Call → Funcs → Stack → Cache → (Val, Cache, SCG)
20 iterate call funcs stack cache1 =
21   let (valn, cache2, scg) = eval call funcs stack cache1
22   in if call ∈ scg
23     then let val0 = if call ∈ cache2 then cache2(call) else ⊥
24           valw = val0 ∇Val valn
25           stable = if valw ⊆ val0 && size scg == 1
26                   then Stable else Unstable
27           cache3 = cache2[call → (stable, valw)]
28           in if val0 ⊆ valw
29             then iterate call funcs stack cache3
30             else (valw, cache3, scg \ {call})
31     else (valn, cache2, scg)
32
33 ∇Stack stack (env1, expr)
34   | expr ∈ dom stack ∧ env1 ⊆ env2 = (stack, (env2, expr))
35   | expr ∈ dom stack ∧ env1 ⊈ env2 = (stack[expr → envw], (envw, expr))
36   | otherwise = (stack[expr → env1], (env1, expr))
37   where env2 = stack(expr)
38         envw = env2 ∇Env env1
39
40 ∇Env :: Env → Env → Env
41 ∇Val :: Val → Val → Val

```

$\perp$ to $[1, 1]$, the algorithm iterates again and propagates the new analysis result $\text{fib}[1, \infty] \mapsto [1, 1]$ throughout the outer SCG. After returning to $\text{fib}[0, \infty] \mapsto [0, 1]$ (line 17 in Figure 3.5), the result has grown from $[0, 1]$ to $[0, 2]$ and hence the algorithm widens the result to $[0, \infty]$ (line 24 in Listing 3.1). Since the result is greater after widening, the algorithm iterates the call $\text{fib}[0, \infty]$ again until it does not grow anymore. Finally, after one more iteration of call $\text{fib}[1, \infty]$, the result $[1, \infty]$ does not grow anymore (line 25 in Figure 3.5) and the algorithm sets the cache entry to stable (line 25 in Listing 3.1). This algorithm we just explained iterates on the strongly-connected subgraphs of the graph-shaped trace of the calls to the abstract interpreter and satisfies the termination conditions introduced in Section 3.3.12, and was introduced in [KEH23].

[KEH23] Keidel et al., “Combinator-Based Fixpoint Algorithms for Big-Step Abstract Interpreters”

[CC92] Cousot and Cousot, “Comparing the Galois connection and widening/narrowing approaches to abstract interpretation”

[Tar55] Tarski, “A lattice-theoretical fixpoint theorem and its applications.”

Soundness proof of Monolithic Fixpoint Algorithms. [CC92; Tar55]

Let $\text{eval}^\sharp : D^\sharp \rightarrow D^\sharp$ be an abstract interpreter and $\text{eval} : D \rightarrow D$ the standard semantics of the concrete interpreter over the complete lattices $(D^\sharp, \sqsubseteq, \sqcup)$ and $(D, \sqsubseteq, \sqcup)$. Let $\gamma : D^\sharp \rightarrow D$ be a monotone concretization function such that:

$$\forall x \in D. \hat{x} \in D^\sharp. x \sqsubseteq \gamma(\hat{x}) \implies \text{eval}(x) \sqsubseteq \gamma(\text{eval}^\sharp(\hat{x}))$$

A fixpoint algorithm for the abstract interpreter is sound if it yields a post-fixpoint of $\text{eval}^\sharp$, i.e. an element $\hat{p} \in D^\sharp$ with $\text{eval}^\sharp(\hat{p}) \sqsubseteq \hat{p}$.

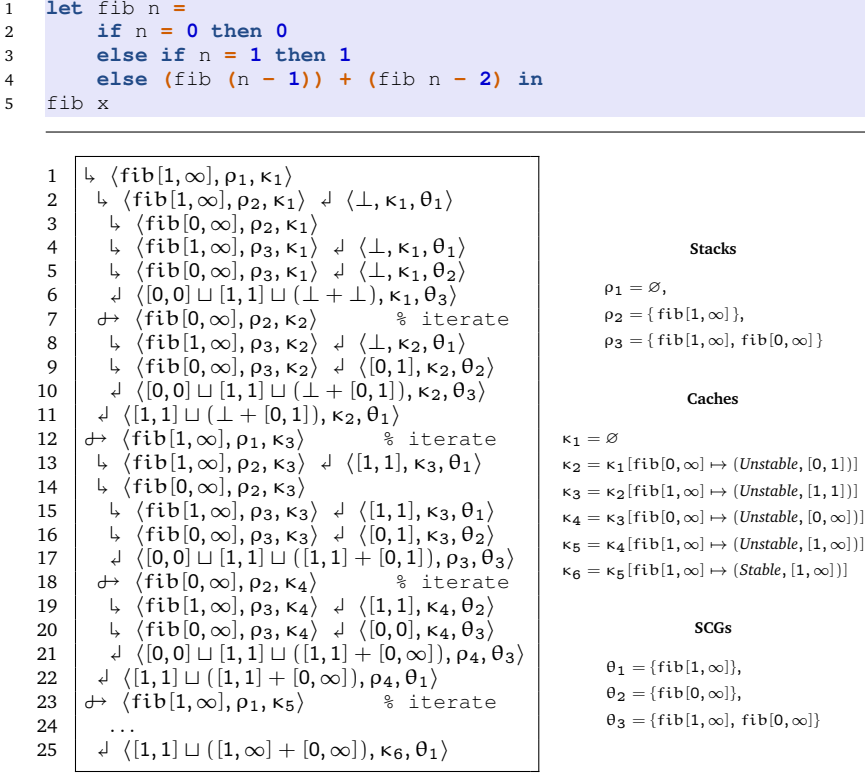


FIGURE 3.5: The trace of the abstract interpreter analyzing `fib` with starting abstract environment $x = [1; \infty]$.

$$\begin{aligned}
& \text{eval}^\#(\hat{p}) \sqsubseteq \hat{p} && (\hat{p} \text{ is a post-fixpoint of } \text{eval}^\#) \\
\Rightarrow & \gamma(\text{eval}^\#(\hat{p})) \sqsubseteq \gamma(\hat{p}) && (\gamma \text{ is monotone}) \\
\Rightarrow & \text{eval}(\gamma(\text{eval}^\#(\hat{p}))) \sqsubseteq \gamma(\text{eval}) && (\text{eval}^\# \text{ is sound w.r.t. } \text{eval}) \\
\Rightarrow & \text{lfp}(\text{eval}(p)) \sqsubseteq \gamma(\text{eval}^\#(\hat{p})) && (\text{Tarski's fixpoint theorem.})
\end{aligned}$$

□

3.5.1 Tracing/Reachable Call Graph Analysis

Given the sketches presented in the previous section, it is quite intuitive how to perform call graph analysis in whole-program analyses. By enriching the inputs and outputs of the abstract interpreter `eval`, and relative components (e.g., `fix`, `iterate`) of a graph accumulator endowed with a join for graphs (e.g., set union applied to the sets of vertices and edges), it is possible to return the call graph for reachable functions, representing all possible calling relationships in a program.

Part II

IN VITRO

4

In Vitro Analysis of pWasm

This chapter focuses on providing the tools to perform static analysis of closed WebAssembly modules [PRL24; PRL26]. We start by defining the syntax of interest, dubbed pWasm(pico-WebAssembly), shown in Figure 4.1.

<i>module</i>	$::=$	(module <i>type</i> [*] <i>global</i> [*] <i>import</i> [*] <i>func</i> [*] <i>table</i> ? <i>elem</i> ? <i>start</i> ? <i>mem</i>)
<i>type</i>	$::=$	(type (func <i>ft</i>))
<i>global</i>	$::=$	(global (mut <i>t</i>) (t.const <i>n</i>))
<i>import</i>	$::=$	(import <i>env</i> <i>id</i> <i>importable</i>)
<i>importable</i>	$::=$	(func <i>ft</i>) (global <i>t</i>) (table <i>n</i> funcref)
<i>bt, ft</i>	$::=$	<i>t</i> [*] → <i>t</i> [*]
<i>t</i>	$::=$	i32
<i>func</i>	$::=$	(func (type <i>ft</i>) <i>instr</i> [*])
<i>table</i>	$::=$	(table <i>n</i> [*] funcref)
<i>start</i>	$::=$	(start <i>n</i>)
<i>elem</i>	$::=$	(elem <i>n</i> (data) <i>n</i> [*])
<i>mem</i>	$::=$	(memory 1)
<i>instr</i>	$::=$	<i>control</i> <i>data</i>
<i>control</i>	$::=$	block <i>bt instr</i> [*] end loop <i>bt instr</i> [*] end call <i>n</i> call_indirect <i>ft</i> br_if <i>l</i> br <i>l</i> return
<i>data</i>	$::=$	drop t.const <i>n</i> t.binop t.unop local.get <i>n</i> local.set <i>n</i> global.get <i>n</i> global.set <i>n</i> t.load t.store nop
<i>n, l, tidx</i>	$::=$	NUM_LIT
<i>env, id</i>	$::=$	STRING_LIT

FIGURE 4.1: Syntax of pWasm

*“If you know neither the enemy nor yourself,
you will succumb in every battle.”*
—Sun Tzu, The Art of War.

[PRL24] Paccamiccio et al., “Building call graph of WebAssembly programs via abstract semantics”

[PRL26] Paccamiccio et al., “wassily: Spec-enhanced Call Graph Analysis for WebAssembly”

One notable inclusion is the (import ...) statement, which we

will ignore for now. At this time, we will cover analyzing pure WebAssembly code, as the title suggests, *in vitro*.

We first need a viable definition of program “data” states. The following sections will cover the concern of providing a valid formalization for WebAssembly program states. The original formulation, consisting of stack, store, and instance, is modified with an equivalent, more “mathematically flavored”, and, by exploiting the type system of WebAssembly, the definitions are simplified. First, we introduce the concrete components of the state, then abstract them, with each concrete-abstract couple enjoying its Galois Connections (GC). After defining states and the definitions they depend on, we lay out the semantics for a pure WebAssembly program analyzer.

Convention 1. As a convention, the abstract form of type A will be denoted as $A^\#$, and the abstract counterpart of function f will be denoted as $\hat{f}$.

4.1 SEMANTICS OF WEBASSEMBLY PROGRAM STATES

As mentioned in [Chapter 2](#), a *canonical* program state in WebAssembly consists of a triple composed of stack, store, and instance. To simplify the structure of a running WebAssembly module, we ignore instance and immutable parts of the store (i.e., functions and data sections) and work directly on the store when the semantics involve mutable objects. We also reformulate the stack definition in an equivalent but simpler form. This reformulation is one of the possible equivalent reformulations of the original WebAssembly specification.

We describe the stack, as in the sequence of activation frames, in terms of (function-local) value stacks, local variables, and jump maps, and define the store in terms of globally available mutable data.

Convention 2. We will refer to an entity x of type X as $x \in X$, and indices are all represented by natural numbers $\mathbb{N}$. We will use $\mathbb{Z}$ (mathematical integers) to represent integer values to lighten the notation.

We first define values, denoted by $\mathbb{V}$, as typed numerical values that are allowed in our language. Given that the only allowed type is `i32` (32-bit integer), we can state that:

$$\mathbb{V} : \mathbb{Z}_{32}$$

where $\mathbb{Z}_{32}$ represents 32-bit integers.

An operand stack $\sigma \in \mathbb{S}$ is a sequence of values:

$$\mathbb{S} : (\mathbb{V} \in \mathbb{V})^*$$

Then we define locals, globals, and linear memory functions, denoted as mapping from indices (natural numbers) to values.

$$\mathbb{L}, \mathbb{G}, \mathbb{LM} : \mathbb{N} \rightarrow \mathbb{V}$$

We remark that declarations of global variables and local variables in a function are always statically known.

Then we define function reference tables as a mapping from indices to indices representing the functions in the module.

$$\mathbb{RT} : \mathbb{N} \rightarrow \mathbb{N}$$

Control-flow labels are, in the canonical WebAssembly specification, a type of operand that is pushed into or popped from the stack whenever a control label is entered or exited, and are used to maintain knowledge of where to jump to when a branch instruction targets a label. They can be either `loop` or `block` labels. In our reformulation, to address the issue, we use a mapping of the form:

$$\mathbb{J} : \mathbb{N} \rightarrow \mathbb{C}$$

where $\mathbb{C}$ denotes the *instr* construct (see Figure 4.1). We call this construct a ‘jump map’.

Definition 24. Data state.

We define a ‘data’ state $\mu \in \mathbb{ST}$ as:

$$\mathbb{ST} : \mathbb{S} \times \mathbb{L} \times \mathbb{G} \times \mathbb{LM} \times \mathbb{RT}$$

and endow it with the self-explaining accessors $\mu.\text{stack}$, $\mu.\text{locs}$, $\mu.\text{globs}$, $\mu.\text{linmem}$, $\mu.\text{reftab}$.

Furthermore, we define an answer $a \in \mathbb{A}$ as:

$$\mathbb{A} : \mathbb{ST} \cup \{\perp\}$$

that is the state-wise result of an evaluation for any given statement. We remark that a $\perp$ answer is used to represent both non-termination and evaluations that yield errors.

4.1.1 Abstract State Semantics

Starting from the definitions in Section 4.1, we now define their abstract counterparts, endowed with the necessary Galois connections (GC) and join and widening operators.

4.1.2 Semantics of Abstract Values

As already stated in Section 4.1, in our formalization, operands are values $\mathbb{V}$ only. We represent values in the abstract (abstract values) in the interval domain, endowed with well-known join $\sqcup_i$ and widening ∇_i operators [CC76].

Definition 25. GC of values and abstract values

Given the poset of values $(P(\mathbb{V}), \subseteq)$ and of abstract values $(\mathbb{V}^\#, \sqsubseteq_i)$, we define concretization γ_v and abstraction α_v functions such that they form a Galois connection $(P(\mathbb{V}), \subseteq) \xleftrightarrow[\alpha_v]{\gamma_v} (\mathbb{V}^\#, \sqsubseteq_i)$ defined as:

$$\alpha_v : \begin{cases} P(\mathbb{V}) & \longrightarrow \mathbb{V}^\# \\ \emptyset & \longmapsto \perp_v \\ S & \longmapsto [\min S, \max S] \end{cases} \quad \gamma_v : \begin{cases} \mathbb{V}^\# & \longrightarrow P(\mathbb{V}) \\ \perp_v & \longmapsto \emptyset \\ [l, u] & \longmapsto \{v \in \mathbb{V} \mid l \leq v \leq u\} \end{cases} \quad (4.1)$$

where $\subseteq$ and $\sqsubseteq_i$ denote the set and interval inclusion operators respectively.

[CC76] Cousot and Cousot, “Static determination of dynamic properties of programs”

4.1.3 General Abstractions

[Mon21] Monat, “Static type and value analysis by abstract interpretation of Python programs with native C libraries. (Analyse statique, de type et de valeur, par interprétation abstraite, de programmes Python utilisant des bibliothèques C)”

We preface this section by stating that its main goal is to provide some templates for GCs that will be re-used in the definitions coming in the following sections. Some formalization steps are inspired by the work presented in [Mon21]. The GCs defined in this section concern with maps of indices to values, but this is not exclusive to values: as long as a proper GC is defined, this definition can be extended to references as well.

We define the poset of natural-to-value (N-T-V) maps.

Definition 26. N-T-V poset

Given a function $M_0 : \mathbb{N} \rightarrow \mathbb{V}$, we define its poset:

$$(\mathcal{P}(\mathbb{N} \rightarrow \mathbb{V}), \subseteq)$$

where $\mathbb{N}$ denotes the set of naturals, and $\subseteq$ is the set inclusion. As a final remark, we state that this definition allows tracking sets of maps.

The goal is to define abstractions that lead to maps $\mathbb{N} \rightarrow \mathcal{P}(\mathbb{V}^\#)$, starting from Definition 26. To that end, we first define maps of natural-to-value poset (N-T-P(V)).

Definition 27. N-T-P(V) poset

Given a function $M_1 : \mathbb{N} \rightarrow \mathcal{P}(\mathbb{V})$, we define its poset:

$$(\mathbb{N} \rightarrow \mathcal{P}(\mathbb{V}), \subseteq)$$

This is still a poset thanks to the pointwise poset lifting property. We close the definition by clarifying that this structure tracks, for each *key*, a set of potential values, representing an abstraction of the N-T-V poset.

We now define the abstraction function α from $\mathcal{P}(\mathbb{N} \rightarrow \mathbb{V}) \rightarrow \mathbb{N} \rightarrow \mathcal{P}(\mathbb{V})$, and its converse function γ :

Definition 28. GC between N-T-V and N-T-P(V)

$$\alpha : \begin{cases} \mathcal{P}(\mathbb{N} \rightarrow \mathbb{V}) \longrightarrow \mathbb{N} \rightarrow \mathcal{P}(\mathbb{V}) \\ M \longmapsto \lambda n. \{m(n) \mid m \in M\} \end{cases} \quad (4.2)$$

$$\gamma : \begin{cases} \mathbb{N} \rightarrow \mathcal{P}(\mathbb{V}) \longrightarrow \mathcal{P}(\mathbb{N} \rightarrow \mathbb{V}) \\ f \longmapsto \{m \mid \forall n \in \mathbb{N}, m(n) \in f(n)\} \end{cases} \quad (4.3)$$

α and γ form a GC:

$$(\mathcal{P}(\mathbb{N} \rightarrow \mathbb{V}), \subseteq) \xrightleftharpoons[\alpha]{\gamma} (\mathbb{N} \rightarrow \mathcal{P}(\mathbb{V}), \subseteq) \quad (4.4)$$

By coalescent point-wise lifting of the abstract value $\mathbb{V}^\#$ poset, we define the poset for a natural-to-abstract-value mapping $(\mathbb{N} \rightarrow \mathbb{V}^\#, \subseteq_i)$ (N-T-AV).

Definition 29. N-T-AV mapping

$$\mathbb{N} \rightarrow \mathbb{V}^\# = (\mathbb{N} \rightarrow (\mathbb{V}^\# \setminus \perp_v)) \cup \perp \quad (4.5)$$

$$f \dot{\sqsubseteq}_i g = \begin{cases} f = \perp \rightarrow \text{true} \\ f \neq \perp \wedge g \neq \perp \rightarrow \text{true} \iff \forall n \in \mathbb{N}, f(n) \sqsubseteq_i g(n) \\ _ \rightarrow \text{false} \end{cases} \quad (4.6)$$

As coalescent point-wise lifting can be applied to Galois connections as well, we apply it to the GC between values and abstract values so that we can define the GC between N-T-P(V) and N-T-AV.

We recall **Definition 25** (GC of values and abstract values):

$$(P(\mathbb{V}), \subseteq) \xleftrightarrow[\alpha_v]{\gamma_v} (\mathbb{V}^\#, \dot{\sqsubseteq}_i) \quad (4.7)$$

and lift **Equation (4.7)** with the set $\mathbb{N}$, obtaining the GC:

$$(\mathbb{N} \rightarrow P(\mathbb{V}), \subseteq) \xleftrightarrow[\alpha_v]{\gamma_v} (\mathbb{N} \dot{\rightarrow} \mathbb{V}^\#, \dot{\sqsubseteq}_i) \quad (4.8)$$

with $\subseteq$ still denoting the set inclusion, as the pointwise lifting does not change its semantics, and $\dot{\sqsubseteq}_i$ denoting a point-wise lifted partial order operator.

Upper bound $\dot{\sqcup}_i$ and widening $\dot{\nabla}_i$ operators are defined by pointwise lifting as well.

We close this section with the definition that we are going to use the most throughout the next section.

Definition 30. GC between N-T-V sets of maps and N-T-AV maps

Galois connections allow for composition. As such, we define our relation from concrete N-T-V maps to abstract map N-T-AV by composition. Given the GCs:

$$\begin{aligned} (P(\mathbb{N} \rightarrow \mathbb{V}), \subseteq) &\xleftrightarrow[\alpha]{\gamma} (\mathbb{N} \rightarrow P(\mathbb{V}), \subseteq) \\ \text{and} \\ (\mathbb{N} \rightarrow P(\mathbb{V}), \subseteq) &\xleftrightarrow[\alpha_v]{\gamma_v} (\mathbb{N} \dot{\rightarrow} \mathbb{V}^\#, \dot{\sqsubseteq}_i) \end{aligned}$$

by composition we define:

$$(P(\mathbb{N} \rightarrow \mathbb{V}), \subseteq) \xleftrightarrow[\alpha_v \circ \alpha]{\gamma \circ \gamma_v} (\mathbb{N} \dot{\rightarrow} \mathbb{V}^\#, \dot{\sqsubseteq}_i) \quad (4.9)$$

4.1.4 Variables and Linear Memory

We recall that locals, globals, and linear memory are all defined as functions $\mathbb{N} \rightarrow \mathbb{V}$, which corresponds to the typing of the poset N-T-V defined in **Section 4.1.3** (General abstractions). As such, we can use the abstraction and concretization functions we already defined by composition in **Definition 30**, and also use the same point-wise lifted join $\dot{\sqcup}_i$ and widening $\dot{\nabla}_i$ operators.

4.1.5 Operand Stack

We are now concerned with defining the abstract operand stack $\hat{o} \in \mathbb{S}^\#$, which is intuitively a sequence of abstract values, and its relation with the concrete operand stack. In an effort to lighten the following definition, and re-use the abstractions we already defined, we opt for an alternative formulation, that is the one of a function mapping an index to an operand.

Definition 31. Sequence to map

Let $o = (o_0, o_1, \dots, o_n)$ be a finite sequence of operands. We define the associated map $f_o : \mathbb{N}_{\leq n} \rightarrow \mathbb{V}$ as:

$$f_o = \{i \mapsto o_i \mid i \in \mathbb{N}_{\leq n}\}$$

with the conversion function formally defined as:

$$\Phi : \mathbb{V}^{n+1} \rightarrow \{f : \mathbb{N}_{\leq n} \rightarrow \mathbb{V}\}$$

$$\Phi((o_0, o_1, \dots, o_n)) = f_o \text{ where } f_o(i) = o_i \forall i \in \mathbb{N}_{\leq n}$$

Definition 32. Map to sequence

Let $f_o : \mathbb{N}_{\leq n} \rightarrow \mathbb{V}$ be a function mapping indices to operands. We define the associated sequence $o_i \in \mathbb{V}^{n+1}$ as:

$$o_i = (f(0), f(1), \dots, f(n))$$

with the conversion function formally defined as:

$$\Phi^{-1} : \{f : \mathbb{N}_{\leq n} \rightarrow \mathbb{V}\} \rightarrow \mathbb{V}^{n+1}$$

$$\Phi^{-1}(f) = (f(0), f(1), \dots, f(n))$$

There is a clear isomorphism between Φ and Φ^{-1} , as such, we can adopt them so that we can apply the same abstraction template we defined in [Section 4.1.3](#).

Recalling the GC in [Section 4.1.3](#) (General abstractions):

$$(P(\mathbb{N} \rightarrow \mathbb{V}), \subseteq) \xleftrightarrow[\alpha'_v \circ \alpha]{\gamma \circ \gamma'_v} (\mathbb{N} \rightarrow \mathbb{V}^\#, \dot{\subseteq}_i)$$

we can exploit the isomorphism between sequences and index-to-value maps, to use the same GC above to represent the connection between abstract and concrete operand stacks, and as well use the same point-wise lifted join $\dot{\sqcup}_i$ and widening $\dot{\sqcup}_i$ operators.

As mentioned in [Section 4.1](#), reference tables are defined as $\mathbb{RT} : \mathbb{N} \rightarrow \mathbb{N}$. In the abstract, reference tables $\mathbb{RT}^\#$ map indices to sets of $\mathbb{N}$.

We can once again exploit some principles the abstractions we described in [Section 4.1.3](#). Specifically, we are interested in the steps that abstract N-T-V posets to N-T-P(V) posets (see [Definition 28](#)).

We first reformulate our reference tables as posets. In the concrete, we have $(P(\mathbb{N} \rightarrow \mathbb{N}), \subseteq)$, and in the abstract $(\mathbb{N} \rightarrow P(\mathbb{N}), \subseteq)$, and define their abstraction and concretization.

Definition 33. GC between concrete and abstract reference tables

As per abstraction α and concretization γ defined in [Definition 28](#), we can define the GC between $\mathbb{RT}$ and $\mathbb{RT}^\#$ as:

$$(P(\mathbb{N} \rightarrow \mathbb{N}), \subseteq) \xleftrightarrow[\alpha]{\gamma} (\mathbb{N} \rightarrow P(\mathbb{N}), \subseteq) \quad (4.10)$$

with $\subseteq$ is the subset operator. The join and widening operator of abstract reference tables is the set union $\cup$.

Jump maps are identical in the abstract and concrete and, as such, need no further definition. This is guaranteed by how WebAssembly manages value (and stack) typing and control flow. As such, labels in possible flow joins are always guaranteed to be the same.

4.1.6 Abstract State Definition

Having defined all of its components, we now define abstract ‘data’ states $\mathbb{ST}^\#$ and answers $\mathbb{ST}^\#*$.

Definition 34. Abstract data states.

We define abstract states $\mathbb{ST}^\#$ by coalescent lifting of their components.

$$\mathbb{ST}^\# : (\mathbb{S}^\# \setminus \perp) \times (\mathbb{L}^\# \setminus \perp) \times (\mathbb{G}^\# \setminus \perp) \times (\mathbb{LM}^\# \setminus \perp) \times (\mathbb{RT}^\# \setminus \perp) \cup \{\perp\}$$

We also define the coalescent-lifted partial order for abstract states as:

$$f \dot{\sqsubseteq}_s g = \begin{cases} f \equiv \perp \rightarrow \text{true} \\ f \neq \perp \wedge g \neq \perp \rightarrow f \sqsubseteq_s g \\ _ \rightarrow \text{false} \end{cases} \quad (4.11)$$

with $\sqsubseteq_s$ defined as:

$$(\sigma_1, \delta_1, gv_1, lm_1, rt_1) \dot{\sqsubseteq}_s (\sigma_2, \delta_2, gv_2, lm_2, rt_2) = \sigma_1 \dot{\sqsubseteq}_i \sigma_2 \wedge \delta_1 \dot{\sqsubseteq}_i \delta_2 \wedge gv_1 \dot{\sqsubseteq}_i gv_2 \wedge lm_1 \dot{\sqsubseteq}_i lm_2 \wedge rt_1 \subseteq rt_2 \quad (4.12)$$

with the f and g tuples being explicitly unpacked. Note that $\sigma_1 \dot{\sqsubseteq}_i \sigma_2$ is a shorthand for $\Phi(\sigma_1) \dot{\sqsubseteq}_i \Phi(\sigma_2)$, that is the isomorphic transformation described in [Definition 31](#).

Definition 35. Join and widening for data states.

We define the join operator as:

$$f \sqcup g = \begin{cases} f \equiv \perp \rightarrow g \\ g \equiv \perp \rightarrow f \\ _ \rightarrow (\sigma_1, \delta_1, gv_1, lm_1, rt_1) \sqcup_s (\sigma_2, \delta_2, gv_2, lm_2, rt_2) = \sigma_1 \dot{\sqcup}_i \sigma_2, \delta_1 \dot{\sqcup}_i \delta_2, gv_1 \dot{\sqcup}_i gv_2, lm_1 \dot{\sqcup}_i lm_2, rt_1 \cup rt_2 \end{cases}$$

and the widening operator as:

$$f \nabla g = \begin{cases} f \equiv \perp \rightarrow g \\ g \equiv \perp \rightarrow f \\ _ \rightarrow (\sigma_1, \delta_1, gv_1, lm_1, rt_1) \nabla_s (\sigma_2, \delta_2, gv_2, lm_2, rt_2) = \sigma_1 \dot{\nabla}_i \sigma_2, \delta_1 \dot{\nabla}_i \delta_2, gv_1 \dot{\nabla}_i gv_2, lm_1 \dot{\nabla}_i lm_2, rt_1 \cup rt_2 \end{cases}$$

Note that in $\sqcup$, $\sigma_1 \dot{\sqcup}_i \sigma_2$ is a shorthand for $\Phi^{-1} \circ \Phi(\sigma_1) \dot{\sqcup}_i \Phi(\sigma_2)$. We use an analogous shorthand in the definition of ∇ .

Definition 36. Abstract answers.

We define abstract answers $\mathbb{ST}^\#*$ as a sequence of abstract output states of the form

$$\sum_{i=0}^n i : \hat{\mu}_i$$

where $0 : \hat{\mu}_0$ represents the state to be evaluated by the natural continuation of the command under evaluation, and its successors represent the accumulation of previously inferred results from the labeled statements (e.g.: `block`, `loop`) encountered. The partial order $\dot{\sqsubseteq}$, join $\sqcup$, and widening ∇ for abstract answers are all defined as the point-wise lifting of the respective operator for abstract states.

4.2 SEMANTIC DOMAINS

In this section we define the semantic domain of our evaluation functions. As they appear, we will also explain each symbol that has not yet been introduced.

Recall 1. We recall the already defined semantic entities:

- (i) $\iota \in \mathbb{J}$ is the ‘jump map’;
- (ii) $\mu \in \mathbb{ST}$ represents a data state;
- (iii) $c \in \mathbb{C}$ represents a command;

First we define the semantic domain of function declarations.

Definition 37. Declarations of functions.

The semantic domain of declarations $\Delta \in \mathcal{D}$ is defined as:

$$\mathcal{D} : \mathbb{N} \rightarrow (\mathbf{t}^* \times \mathbf{t}^* \times \mathbb{C})$$

Intuitively, they associate function indices to triples of type $\mathbf{t}^* \times \mathbf{t}^* \times \mathbb{C}$, with the two $\mathbf{t}^*$ being vectors of WebAssembly types (see the syntax in [Figure 4.1](#)), with the first one covering the input type, the second one the output type.

We define the semantic domain of evaluation of declaration as:

$$\mathcal{D}[\![\cdot]\!] : (\text{func} \dots) \times \mathcal{D} \rightarrow \mathcal{D} \quad (4.13)$$

We now define the semantic domains of concrete and abstract evaluation of statements $\mathbb{C}$:

$$\mathbb{C}[\![\cdot]\!] : \mathbb{C} \times \mathcal{D} \times \mathbb{J} \times \mathbb{ST} \rightarrow (\mathbb{ST} \times \mathbb{K}) \quad (4.14)$$

$$\mathbb{C}^\#[\![\cdot]\!] : \mathbb{C} \times \mathcal{D} \times \mathbb{J} \times \mathbb{ST}^\# \rightarrow (\mathbb{ST}^\# * \times \mathbb{K}) \quad (4.15)$$

We close this section with the definition of the semantic domains for abstract and concrete evaluations of programs. As per the syntax shown in [Figure 4.1](#), a program $p \in \mathcal{P}$ corresponds to the module syntactical construct. The semantic domains of concrete and abstract interpretation of a program are defined as follows:

$$\mathfrak{P}[\![\cdot]\!] : (\mathcal{P} \cup (\mathcal{P} \times \mathbb{N} \times \mathbb{S})) \rightarrow (\mathbb{ST})$$

$$\mathfrak{P}^\#[\![\cdot]\!] : \mathcal{P} \rightarrow \mathcal{P}(\mathbb{ST}^\#)$$

The semantic domains above differ in two fundamental ways. The concrete one uses as input $\mathcal{P} \cup (\mathcal{P} \times \mathbb{N} \times \mathbb{S})$, covering both the invocation of arbitrary exported functions with an input stack, and the invocation of a default (optional) `start` function and has no input stack. As per the abstract version, nothing other than a program is required to perform the evaluation, as we assume any exported function can be invoked with the most general input (i.e. all $\top$). As per the outputs, the concrete co-domain is $\mathbb{ST}$, whereas the abstract is the power set of $\mathbb{ST}^\#$.

4.3 DECLARATION SEMANTICS

The aim of this section is to formalize how we handle function declarations.

$$\frac{\Delta[n \leftarrow (in^* \times out^* \times c)] = \Delta'}{\langle \text{func } n (in^*, out^* c), \Delta \rangle \rightsquigarrow \Delta'} \quad (4.16)$$

Equation (4.16) describes the semantics of function declarations. A `(func ...)` construct is resolved as an association from its index value `n` to its input and output types, together with its sequence of statements.

$$\frac{\langle d_1, \Delta \rangle = \Delta' \quad \langle d_2, \Delta' \rangle = \Delta''}{\langle d_1; d_2, \Delta \rangle \rightsquigarrow \Delta''} \quad (4.17)$$

Equation (4.17) instead covers the evaluation of sequences of declarations. The step relation $\rightsquigarrow$ has semantic domain **Equation (4.13)**

Once all function declarations have been evaluated, the function declaration map Δ is never mutated again. **Equations (4.16)** and **(4.17)** are identical to their abstract counterpart.

4.4 CONCRETE SEMANTICS FOR STANDARD WEBASSEMBLY

In this section we finally introduce the concrete semantics for pWasm.

Remark 2. In this semantic notation, call graph semantics are omitted. We give intuitive semantics for it in this remark. Any instruction that is not `call i` or `call_indirect t` will produce an empty set of callees $\emptyset$. The `call i` instruction will produce a singleton $\{i\}$, whereas `call_indirect t` will produce a singleton containing the function index corresponding to the resolution of the indirection.

The semantics in this section make use of a big-step transition relation $\Longrightarrow$, with semantic domain corresponding to that of **Equation (4.14)**.

$$\text{nop} \frac{}{\langle \text{nop}, \Delta, \iota, \mu \rangle \Longrightarrow \mu} \quad (4.18)$$

Equation (4.18) shows how no-operations (i.e. `skip`, `nop` in WebAssembly) are evaluated. While trivial, this instruction is useful for handling jump maps.

$$\text{c1c2} \frac{\langle c_1, \Delta, \iota, \mu \rangle \Longrightarrow \mu' \quad \langle c_2, \Delta, \iota, \mu' \rangle \Longrightarrow \mu''}{\langle c_1; c_2, \Delta, \iota, \mu \rangle \Longrightarrow \mu''} \quad (4.19)$$

Equation (4.19) describes the evaluation of sequences of statements. The infix operator `;` separates the ‘head’, `c1` and ‘tail’ `c2` from a sequence of statements.

$$\text{block} \frac{\langle c, \Delta, \text{nop} \gg \iota, \mu \rangle \Longrightarrow \mu'}{\langle \text{block } bt \ c, \Delta, \iota, \mu \rangle \Longrightarrow \mu'} \quad (4.20)$$

Equation (4.20) evaluates a typed block of instructions. The infix auxiliary operator $\gg$ is defined as:

$$\iota \gg c = \{n + 1 \leftarrow jc \mid n \leftarrow jc \in \iota\}[0 \leftarrow c]$$

and is used to generate updated label maps so that `br`-like instructions can follow the proper jumps. A `br`-like instruction targeting a `block` label exits from the `block`, explaining the `nop` instruction that is added in ι .

$$\text{loop} \frac{\langle c, \Delta, (\text{loop } bt \ c) \gg \iota, \mu \rangle \Rightarrow \mu'}{\langle \text{loop } bt \ c, \Delta, \iota, \mu \rangle \Rightarrow \mu'} \quad (4.21)$$

Equation (4.21) concerns the interpretation of `loop` constructs. A `br`-like instruction targeting a `loop` label re-enters the `loop`, explaining the `loop` . . . added in ι .

$$\text{br} \frac{\iota(n) = c \quad \langle c, \Delta, \iota, \mu \rangle \Rightarrow \mu'}{\langle \text{br } n, \Delta, \iota, \mu \rangle \Rightarrow \mu'} \quad (4.22)$$

Equation (4.22) defines how unconditional jumps to labels are handled. A lookup on ι is performed to know which command is to be executed next.

$$\text{br_if1} \frac{\text{pop}(\mu) = \mu', v \quad (v \equiv 0)}{\langle \text{br_if } n, \Delta, \iota, \mu \rangle \Rightarrow \mu'} \quad (4.23)$$

$$\text{br_if2} \frac{\iota(n) = c \quad \text{pop}(\mu) = \mu', v \quad (v \neq 0) \quad \langle c, \Delta, \iota, \mu' \rangle \Rightarrow \mu''}{\langle \text{br_if } n, \Delta, \iota, \mu \rangle \Rightarrow \mu''} \quad (4.24)$$

Equations (4.23) and **(4.24)** respectively cover the branching and non-branching cases of `br_if`. The value used as condition is popped from the stack by `pop` and checked against value 0. If $v \equiv 0$, the jump does not happen, otherwise the control jumps to the command retrieved by $\iota(n)$.

$$\begin{aligned} & \Delta(i) = T_i \times T_o \times c \quad \mu \uparrow T_i = \mu' \times \vec{v}_i \\ & \mu'[\text{locs} \leftarrow \Phi(\vec{v}_i)][\text{stack} \leftarrow \text{nil}] = \mu'' \\ & \langle c, \Delta, \{0 \rightarrow \text{nop}\}, \mu'' \rangle \Rightarrow \mu''' \\ & \mu''' \uparrow T_o = \vec{v}_o \\ & \text{call} \frac{\mu'''[\text{stack} \leftarrow \mu'.\text{stack} \downarrow \vec{v}_o][\text{locs} \leftarrow \mu'.\text{locs}] = \mu''''}{\langle \text{call } i, \Delta, \iota, \mu \rangle \Rightarrow \mu''''} \quad (4.25) \end{aligned}$$

Equation (4.25) evaluates direct function calls. First the function index is resolved via the lookup $\Delta(i)$, then the infix operator $\uparrow$ pops a sequence of $\text{len}(T_i)$ values from the stack, reverses it, and returns an updated state. The function body c is then evaluated under jump map $\{0 \rightarrow \text{nop}\}$ (function bodies can be targeted by `br`), locals initialized with values from $\vec{v}_i$ and `nil` stack. The resulting state μ'' is then ‘peeked’ by the infix operator $\uparrow$, that is similar to $\uparrow$, but does not produce a new state. The post-condition uses the global state of μ'' and replaces its stack with the one from μ' concatenated (infix operator $\downarrow$) with $\vec{v}_o$.

$$\begin{array}{c}
\text{pop}(\mu) = \mu', t_i \quad i = \mu'.\text{reftab}(t_i) \\
\Delta(i) = T_i \times T_o \times c \quad \mu' \uparrow T_i = \mu'' \times \vec{v}_i \\
\mu''[\text{locs} \leftarrow \Phi(\vec{v}_i)][\text{stack} \leftarrow \text{nil}] = \mu''' \\
\langle c, \Delta, \{0 \rightarrow \text{nop}\}, \mu''' \rangle \Longrightarrow \mu'''' \\
\mu'''' \uparrow T_o = \vec{v}_o \\
\text{call_indirect} \frac{\mu''''[\text{stack} \leftarrow \mu''.\text{stack} \downarrow \vec{v}][\text{locs} \leftarrow \mu''.\text{locs}] = \mu'''''}{\langle \text{call_indirect } ty, \Delta, i, \mu \rangle \Longrightarrow \mu'''''}
\end{array} \quad (4.26)$$

Equation (4.26) evaluates indirect calls and is quite similar to Equation (4.25). The only difference lies in the first few steps, where a table index t_i is popped to be then dereferenced to a function index via a lookup on `reftab`.

$$\text{return} \frac{}{\langle \text{return}, \Delta, i, \mu \rangle \Longrightarrow \mu} \quad (4.27)$$

Equation (4.27) handles `return`. Due to how calling semantics have been defined, it acts as an identity to μ .

$$\text{drop} \frac{\text{pop}(\mu) = \mu', v}{\langle \text{drop}, \Delta, i, \mu \rangle \Longrightarrow \mu'} \quad (4.28)$$

Equation (4.28) covers `drop`, that discards the topmost element of the operand stack.

$$\text{const} \frac{\llbracket e \rrbracket \mu = v \quad \text{push}(\mu, v) = \mu'}{\langle \text{i32.const } e, \Delta, i, \mu \rangle \Longrightarrow \mu'} \quad (4.29)$$

Equation (4.29) covers `i32.const`, that pushes a constant value to the stack.

$$\text{bop} \frac{\begin{array}{c} \text{pop}(\mu) = \mu', v_1 \quad \text{pop}(\mu') = \mu'', v_2 \\ \text{push}(\mu'', \text{binop}(v_1, v_2)) = \mu''' \end{array}}{\langle \text{i32.binop}, \Delta, i, \mu \rangle \Longrightarrow \mu'''} \quad (4.30)$$

Equation (4.30) describes how to handle binary operations. Two values are popped, a function $\text{i32} \times \text{i32} \rightarrow \text{i32}$ is then applied to them, and the result is pushed to the stack.

$$\text{uop} \frac{\text{pop}(\mu) = \mu', v_1 \quad \text{push}(\mu', \text{unop}(v_1)) = \mu''}{\langle \text{i32.unop}, \Delta, i, \mu \rangle \Longrightarrow \mu''} \quad (4.31)$$

Equation (4.31) shows how unary operations are handled. A value is popped, a function $\text{i32} \rightarrow \text{i32}$ is then applied to it, and the result is pushed to the stack.

$$\text{load} \frac{\text{pop}(\mu) = \mu', a \quad \text{push}(\mu', \mu'.\text{linmem}(a)) = \mu''}{\langle \text{i32.load}, \Delta, i, \mu \rangle \Longrightarrow \mu''} \quad (4.32)$$

Equation (4.32) describes a read from linear memory. An address is popped and read from `linmem`. The value read is then pushed to the stack.

$$\text{store} \frac{\text{pop}(\mu) = \mu', v \quad \text{pop}(\mu') = \mu'', a \quad \mu''[\text{linmem}[a \setminus v]] = \mu'''}{\langle \text{i32.store}, \Delta, i, \mu \rangle \Longrightarrow \mu'''} \quad (4.33)$$

Equation (4.33) shows how storing to the linear memory works. v , representing a value, and an address a , are popped. Then v is written in a in `linmem`.

4.5 ABSTRACT SEMANTICS FOR STANDARD WEBASSEMBLY

In this section we finally introduce the abstract semantics for statements that are non-homomorphic to their concrete counterpart, introduced in [Section 4.4](#). These abstract semantics are the formalization of the abstract evaluation functions, that will be the foundation to implementing an abstract interpreter.

Remark 3. In this semantic notation, call graph semantics are omitted. We give intuitive semantics for it in this remark. Any instruction that is not `call i` or `call_indirect t` will produce an empty set of callees $\emptyset$. The `call i` instruction will produce a singleton $\{i\}$, whereas `call_indirect t` will produce a set containing the function indices corresponding to the possible resolutions of the indirection.

We now introduce abstract semantics for instructions that are non-homomorphic to the concrete semantics in [Section 4.4](#). The semantics in this section make use of a big-step transition relation $\xRightarrow{\#}$, with semantic domain corresponding to that of [Equation \(4.15\)](#).

Remark 4 (Handwaving in Notation). Given how the sequence-based notation (e.g.: $(0 : (\hat{\mu}') + \sum_{i=1}^n i : \hat{\mu}_i)$) can be heavy in reading, when the resulting abstract state reads as i.e. $\hat{\mu}'$, it means that that sequence is comprised of the natural continuation at index 0 with $\hat{\mu}'$, and the rest of the sequence is left untouched.

$$\text{c1c2}^{\#} \frac{\langle c_1, \Delta, \iota, \mu \rangle \xRightarrow{\#} \sum_{i=0}^n i : \hat{\mu}'_i \quad \langle c_2, \Delta, \iota, \hat{\mu}'_0 \rangle \xRightarrow{\#} \sum_{i=0}^n i : \hat{\mu}''_i}{\langle c_1; c_2, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} 0 : \hat{\mu}''_0 + (\sum_{i=1}^n i : \hat{\mu}'_i \sqcup \hat{\mu}''_i)} \quad (4.34)$$

In the semantics of the sequential, described in [Equation \(4.34\)](#), besides propagating the result of the evaluation of `c1` (e.g., the 0-indexed element from $\sum_{i=0}^n i : \hat{\mu}'_i$) as a precondition for the evaluation of `c2`, we also join the rest of the sequence: this is necessary to account for jumping instructions.

$$\text{block}^{\#} \frac{\langle c, \Delta, \text{nop} \gg \iota, \hat{\mu} \rangle \xRightarrow{\#} \sum_{i=0}^n i : \hat{\mu}'_i}{\langle \text{block } bt \ c, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} 0 : (\hat{\mu}'_0 \sqcup \hat{\mu}'_1) + \sum_{i=2}^n (i-1) : \hat{\mu}'_i} \quad (4.35)$$

$$\text{loop}^{\#} \frac{\langle c, \Delta, (\text{loop } bt \ c) \gg \iota, \hat{\mu} \rangle \xRightarrow{\#} \sum_{i=0}^n i : \hat{\mu}'_i}{\langle \text{loop } bt \ c, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} 0 : (\hat{\mu}'_0 \sqcup \hat{\mu}'_1) + \sum_{i=2}^n (i-1) : \hat{\mu}'_i} \quad (4.36)$$

Similar to their concrete counterparts, the evaluation of `block` and `loop` statements evaluates the command `c`. What changes is that, to account for conditional jumps, the resulting sequence is manipulated

such that the data state $0 : (\hat{\mu}_0 \sqcup \hat{\mu}_1)$ that will be used by the natural continuation of the `block` or `loop` statement is the upper bound of branching instructions targeting said labeled statement (e.g., $\hat{\mu}_1$, and the data state produced by *exhausting* without branching c (e.g., $\hat{\mu}_0$).

Definition 38 (Fold over the outputs of the interpreter).

$$\begin{aligned} \text{acc} &\in \text{ACC} : \text{ST}^{\#*} \\ \text{fold}(f, \text{acc}, \text{cl}) : &\begin{cases} \text{cl} = [] \implies \text{acc} \\ \text{cl} = h :: \text{cl} \implies f \text{ acc } (f \text{ acc } h) \text{ cl} \end{cases} \end{aligned}$$

Intuitively, `call_indirect ty` can be explained in the abstract as a (possibly empty) set of calling relationships starting from the same control point and data state. The main difference w.r.t the `call i` instruction lies in the fact that there might be 0 or more commands (e.g., function bodies) to be evaluated after performing the appropriate manipulations on the state. The results of such evaluations should then be joined in a single final abstract state of type $\text{ST}^{\#*}$. In an effort to ease the notation required, we borrow from the well-known *fold* construct from the world of functional programming, and define our *fold* operator in **Definition 38**.

Briefly, we need a function f of type $\text{ACC} \times \mathbb{C} \rightarrow \text{ACC}$, an accumulator $\text{acc} \in \text{ACC}$, and a list of commands cl . Function f consist in the evaluation function for a parametrized function body and accumulator acc , and returns a resulting accumulator that is the upper bound of the input accumulator and the one resulting from evaluating the function body. In this semantics, the accumulator consists of just $\text{ST}^{\#*}$.

$$\begin{aligned} \lambda \text{acc}. \lambda \chi. &\frac{\langle \chi, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} \hat{\mu}'}{\langle \chi, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} \text{acc} \sqcup \hat{\mu}'} = f \\ &\text{pop}(\hat{\mu}) = \hat{\mu}_1, \hat{i}_t \quad \hat{\mu}_1 \uparrow \text{in}^* = \hat{\mu}_2, \hat{v}_i \\ &\sqcup \{ \hat{\mu}_2.\text{reftab}(i) \mid i \in \gamma_i(\hat{i}_t) \} = I_n \\ &\{ c \mid i \times o \times c \in \Delta \wedge i = \text{in}^* \wedge o = \text{out}^* \} = \delta_t \\ &\hat{\mu}_2[\text{locs} \leftarrow \Phi(\hat{v}_i)][\text{stack} \leftarrow \text{nil}] = \hat{\mu}_3 \\ &\{ \text{fst}(\Delta(i)) \mid i \in I_n \} = \delta_i \quad \delta_t \cap \delta_i = \delta \\ &\text{fold}(f, \delta, \perp) = \hat{\mu}_4 \\ &\hat{\mu}_4 \uparrow \text{out}^* = \hat{v}_o \\ \text{call_indirect}^{\#} &\frac{\hat{\mu}_4[\text{stack} \leftarrow \hat{\mu}_2.\text{stack} \downarrow \hat{v}_o][\text{locs} \leftarrow \hat{\mu}_2.\text{locs}] = \hat{\mu}'}{\langle \text{call_indirect}(\text{in}^* \text{out}^*), \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} \hat{\mu}'} \end{aligned} \quad (4.37)$$

Equation (4.37) explains how the evaluation of abstract indirect call is performed. A given abstract evaluation `call_indirect` might resolve to multiple callees. As such, we first compute the upper bound (auxiliary $\sqcup$) of all the possible dereferences, producing I_n . We then gather the function bodies with same type as the `call_indirect`, producing δ_t . We then collect all function bodies the abstract dereferencing produced with $\{ \text{fst}(\Delta(i)) \mid i \in I_n \}$, with `fst` being an auxiliary function that returns the first element of a tuple. This operation produces a δ_i , that we intersect with δ_t to obtain the set of callees that are both possible and

valid by typing. The possible evaluations are then folded into $\hat{\mu}_4$, and the local context of the joined computations is set to the appropriate one, producing $\hat{\mu}'$.

$$\text{br}^\# \frac{\iota(n) = c \quad \langle c, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} \sum_{i=0}^n i : \hat{\mu}'_i}{\langle \text{br } n, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} (n+1 : (\hat{\mu}'_0 \sqcup \hat{\mu}'_{n+1}) + \sum_{i=n+2}^x i : \hat{\mu}'_i)} \quad (4.38)$$

Equation (4.38) describes the evaluation of unconditional branches.

$$\text{br_if}^\# \frac{\iota(n) = c \quad \text{pop}(\hat{\mu}) = \hat{\mu}_1, \hat{v} \quad \langle c, \Delta, \iota, \hat{\mu}_1 \rangle \xRightarrow{\#} \sum_{i=0}^n i : \hat{\mu}'_i \quad \hat{\mu}'_0 \sqcup \hat{\mu}'_{n+1} = \hat{\mu}_j}{\langle \text{br_if } n, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} 0 : \hat{\mu}_1 + (n+1 : (\hat{\mu}_j) + \sum_{i=n+2}^x i : \hat{\mu}'_i)} \quad (4.39)$$

Equation (4.39) describes the evaluation of branching instructions on conditions that may be either zero or non-zero. The abstract computation performing branching and the one not branching are then joined with the upper bound operator $\sqcup$.

$$\text{load}^\# \frac{\text{pop}(\hat{\mu}) = \hat{\mu}', \hat{a} \quad \sqcup \{ \hat{\mu}'.\text{linmem}(x) \mid x \in \gamma_i(\hat{a}) \} = \hat{v} \quad \text{push}(\hat{\mu}', \hat{v}) = \hat{\mu}''}{\langle \text{i32.load}, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} \hat{\mu}''} \quad (4.40)$$

Equation (4.40) covers how to read values from linear memory. Intuitively, the loaded address is concretized, and a read and push to the stack is performed for each address in the concretization. The upper bound of all the produced states is the result of the abstract load.

$$\text{store}^\# \frac{\text{pop}(\hat{\mu}) = \hat{\mu}', \hat{v} \quad \text{pop}(\hat{\mu}') = \hat{\mu}'', \hat{x} \quad \hat{\mu}''.\text{linmem} = \hat{\text{lm}} \quad \sqcup \{ \hat{\text{lm}}[x \setminus \hat{v}] \mid x \in \gamma_i(\hat{a}) \} = \hat{\text{lm}}' \quad \hat{\mu}''[\text{linmem} \leftarrow \hat{\text{lm}}'] = \hat{\mu}'''}{\langle \text{i32.store}, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} \hat{\mu}'''} \quad (4.41)$$

Equation (4.41), similarly to Equation (4.40), a writing of an abstract value is performed for each of the possible entries of the concretization of the abstract address. The upper bound of the linear memories produced in this manner is substituted to the one in $\hat{\mu}''$, producing $\hat{\mu}'''$.

4.5.1 Fixpoint Algorithm

The *naïve* semantics presented above may cause the analysis not to terminate in case of loops or recursion. For the analysis to be guaranteed to terminate, we ought to use a fixpoint algorithm. First, we enrich the semantic domain of evaluation of commands to:

$$\mathcal{S}^\#[\cdot] : \mathbb{C} \times \mathbb{D} \times \mathbb{J} \times \mathcal{ST}^\# \times \Theta^\# \times \mathbb{K}^\# \rightarrow (\mathcal{ST}^\# \times \mathbb{K}^\# \times \mathcal{P}(\mathbb{C} \times \mathbb{J} \times \mathcal{ST}^\#))$$

where:

- (i) $\Theta^\#$ is a map $\mathbb{C} \times \mathbb{J} \rightarrow \mathbb{ST}^\#$ mapping statements under evaluation to the states they were evaluated under, allowing for the detection of loops and recursion;
- (ii) $K^\#$ is a map $(\mathbb{C} \times \mathbb{J} \times \mathbb{ST}^\#) \rightarrow (\mathbb{ST}^\# * \times \{\text{stable} \mid \text{unstable}\})$ used to keep track of known stable or unstable answers for tuples of control (e.g., command and jump map) and data states under which they were evaluated;
- (iii) $P(\mathbb{C} \times \mathbb{J} \times \mathbb{ST}^\#)$ is a set used to keep track of evaluations (as in calls to the abstract interpreter) $\mathbb{C} \times \mathbb{J} \times \mathbb{ST}^\#$ needing fixed point iteration.

$\Theta^\#$ and $\mathbb{C} \times \mathbb{J} \times \mathbb{ST}^\#$ enjoy a special widening operator that ensures that the interpreter does not re-evaluate statements under smaller abstract states. We already provided intuitive explanations to this type of approach to approximating fix-point computations in [Section 3.3.11](#). Next, we formalize said machinery as inference rules. Following the formalization of the fixpoint computation machinery, we will enrich our abstract semantics with said machinery.

4.5.2 Fixpoint Computation and Iteration Transitions

We formalize the machinery needed for fixpoint computation as two separate transitions with the same type as our evaluation function. The transition for requesting fixpoint computations is $\xrightarrow{\#}_\circ$, and the transition for fixpoint iteration is $\xrightarrow{\#}_{\nabla}$. Transition $\xrightarrow{\#}_\circ$ is responsible for: detection of potentially diverging calls to the interpreter, ensuring the inputs under which an expression is computed always grow (by means of the widening ∇_{sk} , and avoiding unnecessary re-iteration by looking up the cache k . Transition $\xrightarrow{\#}_{\nabla}$ is responsible for the process of fixpoint iteration: it encodes the iteration strategy (i.e., inner-most, outer-most, top-most call), it discerns stable and unstable computations, and may call itself recursively if needed. These two transitions, together with $\xRightarrow{\#}$, are mutually recursive, as $\xRightarrow{\#}$ uses $\xrightarrow{\#}_\circ$, which in turn uses $\xrightarrow{\#}_{\nabla}$, which uses both itself and $\xRightarrow{\#}$.

Definition 39.

$$\begin{aligned}
 &\theta \nabla_{sk}(c, \iota, \hat{\mu}) : \theta', c, \iota, \hat{\mu}' \\
 &\theta : \mathbb{C} \times \mathbb{J} \rightarrow \mathbb{ST}^\#_* \\
 &\theta \nabla_{sk}(c, \iota, \hat{\mu}) = \begin{cases} c, \iota \notin \text{dom}(\theta) \rightarrow \theta[c, \iota \rightarrow \hat{\mu}] \\ \theta(c, \iota) = \hat{\mu}_{sk} \rightarrow \theta[c, \iota \rightarrow \hat{\mu}_{sk} \nabla (\hat{\mu} \sqcup \hat{\mu}_{sk})] \end{cases} \\
 &\theta \nabla_{sk}(c, \iota, \hat{\mu}) = \theta', c, \iota, \hat{\mu}_w \\
 &\kappa(c, \iota, \hat{\mu}_w) = \left(\sum_{i=0}^n i : \hat{\mu}_i, \iota, \text{stable} \right) \\
 &\text{fix1} \frac{}{\theta, \kappa \vdash \langle c, \Delta, \iota, \hat{\mu} \rangle \xrightarrow{\#}_\circ \left(\sum_{i=0}^n i : \hat{\mu}_i, \kappa, \emptyset \right)}
 \end{aligned}$$

Inference rule *fix1* concerns the first case of what can happen when a fixpoint computation is requested. First the input call to the interpreter

is widened by ∇_{sk} , then, if a stable result is found in the cache κ , it is returned unchanged together with the unmodified κ , and, since there is no need for iteration, the set of strongly connected components is returned empty.

$$\text{fix2} \frac{\begin{array}{l} \theta \nabla_{sk}(c, \iota, \hat{\mu}) = \theta', c, \iota, \hat{\mu}_w \\ \kappa(c, \iota, \hat{\mu}_w) = (\sum_{i=0}^n i : \hat{\mu}_i), \iota, \text{unstable} \end{array}}{\theta, \kappa \vdash \langle c, \Delta, \iota, \hat{\mu} \rangle \xrightarrow{\#} (\sum_{i=0}^n i : \hat{\mu}_i), \kappa, \{c, \iota, \hat{\mu}_w\}}$$

Inference rule *fix2* covers the case of having some unstable knowledge in cache κ . After widening the call to the interpreter, the unstable result is returned unchanged together with the unmodified κ , and, since iteration needs to be performed, the set of strongly connected components is returned as a singleton containing the widened call to the interpreter.

$$\text{fix3} \frac{\begin{array}{l} \theta \nabla_{sk}(c, \iota, \hat{\mu}) = \theta', c, \iota, \hat{\mu}_w \\ (c, \iota, \hat{\mu}_w) \notin \text{dom}(\kappa) \wedge \theta \equiv \theta' \\ \kappa[c, \iota, \hat{\mu}_w \rightarrow \perp] = \kappa' \end{array}}{\theta, \kappa \vdash \langle c, \Delta, \iota, \hat{\mu} \rangle \xrightarrow{\#} \perp, \kappa', \{c, \iota, \hat{\mu}_w\}}$$

Inference rule *fix3* is responsible for recognizing true control flow back-edges, and works for recursion and iteration alike. If the widened call to the interpreter is not in the domain of cache κ , and the widened stack θ' is equal to θ , we have found a back-edge, and as such, we return a $\perp$ result, together with an updated cache reflecting this, and a singleton with the widened call to the interpreter, to indicate this needs iteration.

$$\text{fix4} \frac{\begin{array}{l} \theta \nabla_{sk}(c, \iota, \hat{\mu}) = \theta', c, \iota, \hat{\mu}_w \\ \theta \neq \theta' \wedge (c, \iota, \hat{\mu}_w) \notin \text{dom}(\kappa) \\ \theta', \kappa \vdash \langle c, \Delta, \iota, \hat{\mu}_w \rangle \xrightarrow{\#} (\sum_{i=0}^n i : \hat{\mu}_i), \kappa', g \end{array}}{\theta, \kappa \vdash \langle c, \Delta, \iota, \hat{\mu} \rangle \xrightarrow{\#} (\sum_{i=0}^n i : \hat{\mu}_i), \kappa', g}$$

Inference rule *fix4* is responsible for requesting the fixpoint iteration on the widened call to the interpreter. Its result depends solely on the iteration performed by $\theta', \kappa \vdash \langle c, \Delta, \iota, \hat{\mu}_w \rangle \xrightarrow{\#} (\sum_{i=0}^n i : \hat{\mu}_i), \kappa', g$.

$$\text{iter1} \frac{\theta, \kappa \vdash \langle c, \Delta, \iota, \hat{\mu} \rangle \xrightarrow{\#} (\sum_{i=0}^n i : \hat{\mu}_i), \kappa', g \quad (c, \iota, \hat{\mu}) \notin g}{\theta, \kappa \vdash \langle c, \Delta, \iota, \hat{\mu} \rangle \xrightarrow{\#} (\sum_{i=0}^n i : \hat{\mu}_i), \kappa', g}$$

Inference rule *iter1* covers the case of an interpretation that no longer requires iteration. If the triple $(c, \iota, \hat{\mu})$ does not belong in set g , the result of the transition $\theta, \kappa \vdash \langle c, \Delta, \iota, \hat{\mu} \rangle \xrightarrow{\#} (\sum_{i=0}^n i : \hat{\mu}_i), \kappa', g$ is returned as-is.

$$\begin{array}{c}
\theta, \kappa \vdash \langle c, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} (\sum_{i=0}^n \iota : \hat{\mu}_i), \kappa', g \\
(c, \iota, \hat{\mu}) \in g \quad \kappa'(c, \iota, \hat{\mu}) = (\sum_{i=0}^n \iota : \hat{\mu}_i^{\text{old}}, _ \\
(\sum_{i=0}^n \iota : \hat{\mu}_i^{\text{old}}) \nabla (\sum_{i=0}^n \iota : \hat{\mu}_i) = (\sum_{i=0}^n \iota : \hat{\mu}_i^{\text{w}}) \\
(\sum_{i=0}^n \iota : \hat{\mu}_i^{\text{w}}) \sqsubseteq (\sum_{i=0}^n \iota : \hat{\mu}_i^{\text{old}}) \wedge \text{card}(g) \equiv 1 \\
\kappa'[c, \iota, \hat{\mu} \rightarrow (\sum_{i=0}^n \iota : \hat{\mu}_i^{\text{w}}), \text{stable}] = \kappa'' \\
\text{iter2} \frac{}{} \theta, \kappa \vdash \langle c, \Delta, \iota, \hat{\mu} \rangle \xrightarrow{\#} (\sum_{i=0}^n \iota : \hat{\mu}_i^{\text{w}}), \kappa'', \emptyset
\end{array}$$

Inference rule *iter2* concerns with stable iterations results. If the triple $(c, \iota, \hat{\mu})$ belongs to g , a lookup on κ' is performed, obtaining $(\sum_{i=0}^n \iota : \hat{\mu}_i^{\text{old}})$, which is widened with $(\sum_{i=0}^n \iota : \hat{\mu}_i)$, obtaining $(\sum_{i=0}^n \iota : \hat{\mu}_i^{\text{w}})$. If this did not result in a bigger abstract value w.r.t. $(\sum_{i=0}^n \iota : \hat{\mu}_i^{\text{old}})$, and the cardinality of g is 1, this is marked as stable on the cache, producing a new one. Then, the widened abstract value is returned, together with κ'' and $\emptyset$.

$$\begin{array}{c}
\theta, \kappa \vdash \langle c, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} (\sum_{i=0}^n \iota : \hat{\mu}_i), \kappa', g \quad (c, \iota, \hat{\mu}) \in g \\
\kappa'(c, \iota, \hat{\mu}) = (\sum_{i=0}^n \iota : \hat{\mu}_i^{\text{old}}, _ \\
(\sum_{i=0}^n \iota : \hat{\mu}_i^{\text{old}}) \nabla (\sum_{i=0}^n \iota : \hat{\mu}_i) = (\sum_{i=0}^n \iota : \hat{\mu}_i^{\text{w}}) \\
(\sum_{i=0}^n \iota : \hat{\mu}_i^{\text{w}}) \sqsubseteq (\sum_{i=0}^n \iota : \hat{\mu}_i^{\text{old}}) \wedge \text{card}(g) > 1 \\
\kappa'[c, \iota, \hat{\mu} \rightarrow (\sum_{i=0}^n \iota : \hat{\mu}_i^{\text{w}}), \text{unstable}] = \kappa'' \\
g \setminus \{c, \iota, \hat{\mu}\} = g' \\
\text{iter3} \frac{}{} \theta, \kappa \vdash \langle c, \Delta, \iota, \hat{\mu} \rangle \xrightarrow{\#} (\sum_{i=0}^n \iota : \hat{\mu}_i^{\text{w}}), \kappa'', g'
\end{array}$$

Inference rule *iter3* concerns with unstable iterations results. This rule proceeds the same as *iter2*, but the cardinality of g is bigger than 1. As such, it means that the call needs further iteration. Nonetheless, the newly discovered information is put in the cache, marked as unstable. The present call to the interpreter is removed from g , producing g' . Then, the widened abstract value is returned, together with κ'' and g' .

$$\begin{array}{c}
\theta, \kappa \vdash \langle c, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} (\sum_{i=0}^n i : \hat{\mu}_i), \kappa', g \\
(c, \iota, \hat{\mu}) \in g \quad \kappa'(c, \iota, \hat{\mu}) = (\sum_{i=0}^n i : \hat{\mu}_i^{\text{old}}) \\
(\sum_{i=0}^n i : \hat{\mu}_i^{\text{old}}) \nabla (\sum_{i=0}^n i : \hat{\mu}_i) = (\sum_{i=0}^n i : \hat{\mu}_i^w) \\
(\sum_{i=0}^n i : \hat{\mu}_i^{\text{old}}) \sqsubset (\sum_{i=0}^n i : \hat{\mu}_i^w) \\
\kappa'[c, \iota, \hat{\mu} \rightarrow (\sum_{i=0}^n i : \hat{\mu}_i^w), \text{unstable}] = \kappa'' \\
\theta, \kappa'' \vdash \langle c, \Delta, \iota, \hat{\mu} \rangle \xrightarrow{\#} (\sum_{i=0}^n i : \hat{\mu}'_i), \kappa''', g' \\
\text{iter4} \frac{}{\theta, \kappa \vdash \langle c, \Delta, \iota, \hat{\mu} \rangle \xrightarrow{\#} (\sum_{i=0}^n i : \hat{\mu}'_i), \kappa'', g'}
\end{array}$$

Inference rule *iter4* concerns the case of recursive calls to the iteration transition. This rule proceeds the same as *iter2* and *iter4*, up until the order check. Here, we observe that the widened abstract value strictly grew (e.g., the $\sqsubset$ assertion) w.r.t. the cached one. As such, the newly discovered information is put on the cache, marked as *unstable*, and the output of this rule depends on the recursive call to transition $\xrightarrow{\#}$ under κ'' , with the rest of the parameters unchanged.

4.5.3 Fixpoint semantics

We now enrich the semantics laid out in [Section 4.5](#) with the fixpoint machinery.

$$\text{nop}^{\#} \frac{}{\theta, \kappa \vdash \langle \text{nop}, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} \hat{\mu}, \kappa, \emptyset} \quad (4.42)$$

$$\text{return}^{\#} \frac{}{\theta, \kappa \vdash \langle \text{return}, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} \hat{\mu}, \kappa, \emptyset} \quad (4.43)$$

$$\text{drop}^{\#} \frac{\text{pop}(\hat{\mu}) = \hat{\mu}', \hat{v}}{\theta, \kappa \vdash \langle \text{drop}, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} \hat{\mu}', \kappa, \emptyset} \quad (4.44)$$

$$\text{const}^{\#} \frac{\llbracket n \rrbracket = v \quad \text{push}(\hat{\mu}, \alpha(\{v\})) = \hat{\mu}'}{\theta, \kappa \vdash \langle \text{i32.const } n, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} \hat{\mu}', \kappa, \emptyset} \quad (4.45)$$

$$\text{bop}^{\#} \frac{\begin{array}{l} \text{pop}(\hat{\mu}) = \hat{\mu}', \hat{v}_1 \quad \text{pop}(\hat{\mu}') = \hat{\mu}'', \hat{v}_2 \\ \text{push}(\hat{\mu}'', \text{binop}(\hat{v}_1, \hat{v}_2)) = \hat{\mu}''' \end{array}}{\theta, \kappa \vdash \langle \text{i32.binop}, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} \hat{\mu}''', \kappa, \emptyset} \quad (4.46)$$

$$\text{unop}^{\#} \frac{\text{pop}(\hat{\mu}) = \hat{\mu}', v_1 \quad \text{push}(\hat{\mu}', \text{unop}(v_1)) = \hat{\mu}''}{\theta, \kappa \vdash \langle \text{i32.unop}, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} \hat{\mu}'', \kappa, \emptyset} \quad (4.47)$$

$$\text{load}^{\#} \frac{\begin{array}{l} \text{pop}(\hat{\mu}) = \hat{\mu}', \hat{a} \\ \sqcup\{\hat{\mu}'.\text{linmem}(x) \mid x \in \gamma_i(\hat{a})\} = \hat{v} \\ \text{push}(\hat{\mu}', \hat{v}) = \hat{\mu}'' \end{array}}{\theta, \kappa \vdash \langle \text{i32.load}, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} \hat{\mu}'', \kappa, \emptyset} \quad (4.48)$$

$$\text{store}^\# \frac{\begin{array}{c} \text{pop}(\widehat{\mu}) = \widehat{\mu}', \widehat{v} \quad \text{pop}(\widehat{\mu}') = \widehat{\mu}'', \widehat{x} \\ \widehat{\mu}''.\widehat{\text{linmem}} = \widehat{\text{lm}} \sqcup \{\widehat{\text{lm}}[x \setminus \widehat{v}] \mid x \in \gamma_i(\widehat{a})\} = \widehat{\text{lm}}' \\ \widehat{\mu}''[\widehat{\text{linmem}} \leftarrow \widehat{\text{lm}}'] = \widehat{\mu}''' \end{array}}{\theta, \kappa \vdash \langle \text{i32.store}, \Delta, \iota, \widehat{\mu} \rangle \xRightarrow{\#} \widehat{\mu}''', \kappa, \emptyset} \quad (4.49)$$

$$\text{c1c2}^\# \frac{\begin{array}{c} \theta, \kappa \vdash \langle c_1, \Delta, \iota, \widehat{\mu} \rangle \xRightarrow{\#} \sum_{i=0}^n i : \widehat{\mu}'_i, \kappa', g \\ \theta, \kappa' \vdash \langle c_2, \Delta, \iota, \widehat{\mu}'_0 \rangle \xRightarrow{\#} \sum_{i=0}^n i : \widehat{\mu}''_i, \kappa'', g' \end{array}}{\theta, \kappa \vdash \langle c_1; c_2, \Delta, \iota, \widehat{\mu} \rangle \xRightarrow{\#} (0 : \widehat{\mu}'_0 + (\sum_{i=1}^n i : \widehat{\mu}'_i \sqcup \widehat{\mu}''_i), \kappa'', g \cup g')} \quad (4.50)$$

$$\text{block}^\# \frac{\begin{array}{c} \theta, \kappa \vdash \langle c, \Delta, (\text{nop}) \gg \iota, \widehat{\mu} \rangle \xRightarrow{\#} \sum_{i=0}^n i : \widehat{\mu}'_i, \kappa', g \\ (0 : (\widehat{\mu}'_0 \sqcup \widehat{\mu}'_1) + \sum_{i=2}^n (i-1) \widehat{\mu}'_i) = \widehat{\mu}' \end{array}}{\theta, \kappa \vdash \langle \text{block } bt \ c, \Delta, \iota, \widehat{\mu} \rangle \xRightarrow{\#} \widehat{\mu}', \kappa', g} \quad (4.51)$$

$$\text{loop}^\# \frac{\begin{array}{c} \theta, \kappa \vdash \langle c, \Delta, (\text{loop } bt \ c) \gg \iota, \widehat{\mu} \rangle \xRightarrow{\#} \sum_{i=0}^n i : \widehat{\mu}_i, \kappa', g \\ (0 : (\widehat{\mu}'_0 \sqcup \widehat{\mu}'_1) + \sum_{i=2}^n (i-1) \widehat{\mu}_i) = \widehat{\mu}' \end{array}}{\theta, \kappa \vdash \langle \text{loop } bt \ c, \Delta, \iota, \widehat{\mu} \rangle \xRightarrow{\#} \widehat{\mu}', \kappa', g} \quad (4.52)$$

$$\text{br}^\# \frac{\begin{array}{c} \iota(n) = c \quad \langle c, \Delta, \iota, \widehat{\mu} \rangle \xRightarrow{\#} (\sum_{i=0}^n i : \widehat{\mu}_i), \kappa', g \\ n+1 : (\widehat{\mu}'_0 \sqcup \widehat{\mu}'_{n+1}) + \sum_{i=n+2}^x i : \widehat{\mu}'_i = \widehat{\mu}' \end{array}}{\theta, \kappa \vdash \langle \text{br } n, \Delta, \iota, \widehat{\mu} \rangle \xRightarrow{\#} \widehat{\mu}', \kappa', g} \quad (4.53)$$

$$\text{br_if}^\# \frac{\begin{array}{c} \iota(n) = c \quad \text{pop}(\widehat{\mu}) = \widehat{\mu}_1, \widehat{v} \\ \theta, \kappa \vdash \langle c, \Delta, \iota, \widehat{\mu}_1 \rangle \xRightarrow{\#} (\sum_{i=0}^n i : \widehat{\mu}_i), \kappa', g \\ 0 : \widehat{\mu}_1 + (n+1 : (\widehat{\mu}'_0 \sqcup \widehat{\mu}'_{n+1}) + \sum_{i=n+2}^x i : \widehat{\mu}'_i) = \widehat{\mu}' \end{array}}{\theta, \kappa \vdash \langle \text{br_if } n, \Delta, \iota, \widehat{\mu} \rangle \xRightarrow{\#} \widehat{\mu}', \kappa', g} \quad (4.54)$$

$$\text{call}^\# \frac{\begin{array}{c} \Delta(i) = T_i \times T_o \times c \quad \widehat{\mu} \uparrow T_i = \widehat{\mu}_1 \times \widehat{v}_i \\ \widehat{\mu}_1[\widehat{\text{locs}} \leftarrow \Phi(\widehat{v}_i)][\widehat{\text{stack}} \leftarrow \text{nil}] = \widehat{\mu}_2 \\ \theta, \kappa \vdash \langle c, \Delta, \{0 \rightarrow \text{nop}\}, \widehat{\mu}_2 \rangle \xRightarrow{\#} \widehat{\mu}_3, \kappa', g \\ \widehat{\mu}_3 \uparrow T_o = \widehat{v}_o \\ \widehat{\mu}_3[\widehat{\text{stack}} \leftarrow \widehat{\mu}_1.\widehat{\text{stack}} \downarrow \widehat{v}_o][\widehat{\text{locs}} \leftarrow \widehat{\mu}_1.\widehat{\text{locs}}] = \widehat{\mu}' \end{array}}{\theta, \kappa \vdash \langle \text{call } i, \Delta, \iota, \widehat{\mu} \rangle \xRightarrow{\#} \widehat{\mu}', \kappa', g} \quad (4.55)$$

Definition 40 (Fold over enhanced semantics).

$$\begin{aligned} \text{acc} \smile \text{new} &= \text{ST}^\# *_1 \sqcup \text{ST}^\# *_2, \kappa_2, g_1 \cup g_2 \\ \text{acc} &\in \text{ACC} : \text{ST}^\# * \times \mathbb{K} \times \mathbb{P}(\mathbb{C} \times \mathbb{J} \times \text{ST}^\#) \end{aligned}$$

Same as the potentially diverging abstract semantics above, we need to define a *folding* operator we will use to compute the result of `call_indirects`. Following the same tracks laid out in [Definition 38](#), we define in [Definition 40](#) our folding accumulator to be of the same type as the enriched evaluation function.

$$\begin{array}{c}
\theta, \text{snd}(\text{acc}) \vdash \langle \chi, \Delta, \iota, \widehat{\mu} \rangle \xrightarrow{\#} \widehat{\mu}', \kappa', g' \\
\widehat{\mu}', \kappa', g' = \text{new} \\
\lambda \text{acc}. \lambda \chi. \frac{}{} = f \\
\theta, \kappa_a \vdash \langle \chi, \Delta, \iota, \widehat{\mu} \rangle \xrightarrow{\#} \text{acc} \cup \text{new} \\
\text{pop}(\widehat{\mu}) = \widehat{\mu}_1, \widehat{i}_t \quad \widehat{\mu}_1 \uparrow \text{in}^* = \widehat{\mu}_2, \vec{v}_i \\
\sqcup \{ \widehat{\mu}_2.\text{reftab}(i) \mid i \in \gamma_i(\widehat{i}_t) \} = I_n \\
\{c \mid i \times o \times c \in \Delta \wedge i = \text{in}^* \wedge o = \text{out}^*\} = \delta_t \\
\widehat{\mu}_2[\widehat{\text{locs}} \leftarrow \Phi(\vec{v}_i)][\widehat{\text{stack}} \leftarrow \text{nil}] = \widehat{\mu}_3 \\
\{\text{fst}(\Delta(i)) \mid i \in I_n\} = \delta_i \quad \delta_t \cap \delta_i = \delta \\
\text{fold}(f, \delta, (\perp, \kappa, \emptyset)) = \widehat{\mu}_4, \kappa', g \\
\widehat{\mu}_4 \uparrow \text{out}^* = \vec{v}_o \\
\widehat{\mu}_4[\widehat{\text{stack}} \leftarrow \widehat{\mu}_2.\widehat{\text{stack}} \downarrow \vec{v}_o][\widehat{\text{locs}} \leftarrow \widehat{\mu}_2.\widehat{\text{locs}}] = \widehat{\mu}' \\
\text{call_indirect}^{\#} \frac{}{} \\
\theta, \kappa \vdash \langle \text{call_indirect}(\text{in}^* \text{out}^*), \Delta, \iota, \widehat{\mu} \rangle \xrightarrow{\#} \widehat{\mu}', \kappa', g
\end{array} \tag{4.56}$$

5

Proof of Sound Semantics

PREAMBLE

“Non ci credo, dimostramelo.”
—Il Distruttore di Hoepli.

The *cannot miss errors* claims of verification techniques based on Abstract Interpretation lay their theoretical foundation on the existence of Galois Connections between the concrete and abstract realms. In the case of concrete and abstract semantics, this translates to a soundness proposition of the abstract semantics with respect to the concrete semantics. This chapter explains how we proved the validity of the formalism in [Chapter 4](#).

5.1 STRUCTURE OF THE SOUNDNESS PROOF

We prove the soundness of our abstract fixpoint enriched semantics in a multistep fashion:

- (i) we prove soundness for a *bounded version* of the abstract semantics w.r.t. the concrete semantics;
- (ii) we prove that any derivation of the fixpoint semantics terminates;
- (iii) we prove the soundness of the fixpoint semantics w.r.t. the bounded semantics.

In step (i), we prove soundness for a *bounded version* of the abstract semantics w.r.t. the concrete semantics. This semantics is concerned only with derivations of the concrete semantics that are provable in at most L steps, and we prove that, for all L steps that can be used to complete a derivation of the concrete semantics, our bounded semantics is sound w.r.t. the concrete semantics in at most L steps. We define such semantics in [Section 5.2](#) and prove them sound in [Section 5.3](#).

Intuitively, this proof step addresses executions that are guaranteed to be finite. This is especially useful to prove the soundness of rules that are guaranteed not to create loops. In and of itself, it is not a very useful semantics, but we use it as a *proof aid* for the next proof step.

In step (ii), we prove termination of the fixpoint semantics (defined from [Sections 4.5.2](#) and [4.5.3](#)) on any derivation, that is, for all L , the fixpoint computation *always terminates*. We prove termination for all derivations in [Section 5.4.1](#).

With step (iii) we prove our fixpoint semantics soundly approximates both the bounded semantics and unbounded concrete derivations, that

the bounded semantics is unable to prove. We demonstrate that our semantics is sound in [Section 5.4.2](#).

Proposition 1 (Soundness proposition). *Let us recall that:*

- (i) $\mu \in \mathbb{ST}$ represents a concrete data state;
- (ii) $\mathcal{P}(\mathbb{ST})$ the power set of concrete data states;
- (iii) $\hat{\mu} \in \mathbb{ST}^\#$ represents an abstract data state;
- (iv) α and γ are, abstraction and concretization functions;
- (v) c is a command;
- (vi) Δ contains the function declarations;
- (vii) ι is the “jump map”;
- (viii) $\sum_{i=0}^n \iota : \hat{\mu}_i$ is an abstract answer (sequence of abstract data states).

Given the following Galois Connection:

$$\begin{aligned} \mu &\in \mathbb{ST} \\ \alpha : \mathcal{P}(\mathbb{ST}) &\rightarrow \mathbb{ST}^\# \\ \gamma : \mathbb{ST}^\# &\rightarrow \mathcal{P}(\mathbb{ST}) \\ (\hat{\mu} = \alpha(S)) \end{aligned}$$

$$\forall S \subseteq \mathbb{ST}. \forall \mu \in S. \forall c, \Delta, \iota, L$$

if $\langle c, \Delta, \iota, \mu \rangle \Longrightarrow \mu'$ in at most L steps, then:

$$\langle c, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_L \left(\sum_{i=0}^n \iota : \hat{\mu}'_i \right) \text{ and}$$

$$\langle c, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} \left(\sum_{i=0}^n \iota : \hat{\mu}''_i \right)$$

$$\text{such that: } \mu' \in \gamma(\hat{\mu}'_0) \wedge \mu' \in \gamma(\hat{\mu}''_0) \wedge \sum_{i=0}^n \iota : \hat{\mu}'_i \sqsubseteq \sum_{i=0}^n \iota : \hat{\mu}''_i$$

In [Proposition 1](#) we formally lay out the proposition of the multistep soundness proof we will construct in this chapter. Transition $\xRightarrow{\#}_L$ refers to the bounded abstract semantics that will be introduced in the next section, and $\xRightarrow{\#}$ refers to the abstract fixpoint semantics introduced in [Section 4.5.3](#).

Corollary 1.

$$\frac{\langle c, \Delta, \iota, S \rangle \Longrightarrow S'}{\langle c, \Delta, \iota, \alpha(S) \rangle \xRightarrow{\#} \sum_{i=0}^n \iota : \hat{\mu}'_i} \wedge S \subseteq \gamma(\hat{\mu}'_0)$$

5.2 BOUNDED ABSTRACT SEMANTICS

The semantics in this section make use of a big-step transition relation $\xRightarrow{\#}_L$, where L is the bound on the length of the derivation. The semantic domain of this semantics corresponds to that of [Equation \(4.15\)](#). Each rule presented is based on the abstract semantics defined [Section 4.5](#). This semantics, differently from the one in [Section 4.5](#), is guaranteed to terminate in, at most, $L + 1$ steps.

5.2.1 Axiomatic Rules

The first rule regarding our axioms is presented in Eq. (5.1). It states that, for all commands c under $\Delta, \iota, \hat{\mu}$, its 0-long derivation always rewrites to $\perp$.

$$\text{cf}^\# \frac{}{\langle c, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_0 \perp} \quad (5.1)$$

We now introduce, in Eq. (5.2) to Eq. (5.9) the other bounded axiomatic rules. All of them mirror their counterpart in the abstract semantics (see Section 4.5), but with the addition of an explicit bound that guarantees termination. Their bound is $L = 0$ and, as such, they are all guaranteed to terminate with 1-long derivation steps.

$$\text{nop}^\# \frac{}{\langle \text{nop}, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_{L+1} \hat{\mu}} \quad (5.2)$$

$$\text{return}^\# \frac{}{\langle \text{return}, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_{L+1} \hat{\mu}} \quad (5.3)$$

$$\text{drop}^\# \frac{\text{pop}(\hat{\mu}) = \hat{\mu}', \hat{v}}{\langle \text{drop}, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_{L+1} \hat{\mu}'} \quad (5.4)$$

$$\text{const}^\# \frac{\llbracket n \rrbracket = v \quad \text{push}(\hat{\mu}, \alpha(\{v\})) = \hat{\mu}'}{\langle \text{i32.const } n, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_{L+1} \hat{\mu}'} \quad (5.5)$$

$$\text{bop}^\# \frac{\begin{array}{l} \text{pop}(\hat{\mu}) = \hat{\mu}', \hat{v}_1 \quad \text{pop}(\hat{\mu}') = \hat{\mu}'', \hat{v}_2 \\ \text{push}(\hat{\mu}'', \text{binop}(\hat{v}_1, \hat{v}_2)) = \hat{\mu}''' \end{array}}{\langle \text{i32.binop}, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_{L+1} \hat{\mu}'''} \quad (5.6)$$

$$\text{uop}^\# \frac{\text{pop}(\hat{\mu}) = \hat{\mu}', v_1 \quad \text{push}(\hat{\mu}', \text{unop}(v_1)) = \hat{\mu}''}{\langle \text{i32.unop}, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_{L+1} \hat{\mu}''} \quad (5.7)$$

$$\text{load}^\# \frac{\begin{array}{l} \text{pop}(\hat{\mu}) = \hat{\mu}', \hat{a} \\ \sqcup \{ \hat{\mu}'.\text{linmem}(x) \mid x \in \gamma_i(\hat{a}) \} = \hat{v} \\ \text{push}(\hat{\mu}', \hat{v}) = \hat{\mu}'' \end{array}}{\langle \text{i32.load}, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_{L+1} \hat{\mu}''} \quad (5.8)$$

$$\text{store}^\# \frac{\begin{array}{l} \text{pop}(\hat{\mu}) = \hat{\mu}', \hat{v} \quad \text{pop}(\hat{\mu}') = \hat{\mu}'', \hat{x} \\ \hat{\mu}''.\text{linmem} = \widehat{\text{lm}} \sqcup \{ \widehat{\text{lm}}[x \setminus \hat{v}] \mid x \in \gamma_i(\hat{a}) \} = \widehat{\text{lm}}' \\ \hat{\mu}''[\text{linmem} \leftarrow \widehat{\text{lm}}'] = \hat{\mu}''' \end{array}}{\langle \text{i32.store}, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_{L+1} \hat{\mu}'''} \quad (5.9)$$

5.2.2 Composite Rules

We now introduce, in Eq. (5.10) to Eq. (5.16) our composite rules that are guaranteed to terminate in at most $L + 1$ derivation steps. Again, they mirror their counterparts in the abstract semantics (see Section 4.5), but with an explicit $L + 1$ bound of derivation steps under which termination of the semantics is guaranteed.

$$\text{c1c2}^\# \frac{\langle c_1, \Delta, \iota, \mu \rangle \xRightarrow{\#}_L \sum_{i=0}^n i : \hat{\mu}'_i \quad \langle c_2, \Delta, \iota, \hat{\mu}'_0 \rangle \xRightarrow{\#}_L \sum_{i=0}^n i : \hat{\mu}''_i}{\langle c_1; c_2, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_{L+1} 0 : \hat{\mu}''_0 + \left(\sum_{i=1}^n i : \hat{\mu}'_i \sqcup \hat{\mu}''_i \right)} \quad (5.10)$$

$$\text{block}^\# \frac{\langle c, \Delta, \text{nop} \gg \iota, \hat{\mu} \rangle \xRightarrow{\#}_L \sum_{i=0}^n i : \hat{\mu}'_i}{\langle \text{block } bt \ c, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_{L+1} 0 : (\hat{\mu}'_0 \sqcup \hat{\mu}'_1) + \sum_{i=2}^n (i-1) : \hat{\mu}'_i} \quad (5.11)$$

$$\text{loop}^\# \frac{\langle c, \Delta, (\text{loop } bt \ c) \gg \iota, \hat{\mu} \rangle \xRightarrow{\#}_L \sum_{i=0}^n i : \hat{\mu}'_i}{\langle \text{loop } bt \ c, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_{L+1} 0 : (\hat{\mu}'_0 \sqcup \hat{\mu}'_1) + \sum_{i=2}^n (i-1) : \hat{\mu}'_i} \quad (5.12)$$

$$\text{br}^\# \frac{\iota(n) = c \quad \langle c, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_L \sum_{i=0}^n i : \hat{\mu}'_i}{\langle \text{br } n, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_{L+1} (n+1 : (\hat{\mu}'_0 \sqcup \hat{\mu}'_{n+1}) + \sum_{i=n+2}^x i : \hat{\mu}'_i)} \quad (5.13)$$

$$\text{br_if}^\# \frac{\iota(n) = c \quad \text{pop}(\hat{\mu}) = \hat{\mu}_1, \hat{v} \quad \langle c, \Delta, \iota, \hat{\mu}_1 \rangle \xRightarrow{\#}_L \sum_{i=0}^n i : \hat{\mu}'_i \quad \hat{\mu}'_0 \sqcup \hat{\mu}'_{n+1} = \hat{\mu}_j}{\langle \text{br_if } n, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_{L+1} 0 : \hat{\mu}_1 + (n+1 : (\hat{\mu}_j) + \sum_{i=n+2}^x i : \hat{\mu}'_i)} \quad (5.14)$$

$$\text{call}^\# \frac{\begin{array}{l} \Delta(i) = T_i \times T_o \times c \quad \hat{\mu} \uparrow T_i = \hat{\mu}' \times \vec{v}_i \\ \hat{\mu}'[\text{locs} \leftarrow \Phi(\vec{v}_i)][\widehat{\text{stack}} \leftarrow \text{nil}] = \hat{\mu}'' \\ \langle c, \Delta, \{0 \rightarrow \text{nop}\}, \hat{\mu}'' \rangle \xRightarrow{\#}_L \hat{\mu}''' \\ \hat{\mu}''' \uparrow T_o = \vec{v}_o \end{array}}{\begin{array}{l} \hat{\mu}'''[\widehat{\text{stack}} \leftarrow \hat{\mu}'.\widehat{\text{stack}} \downarrow \vec{v}_o][\text{locs} \leftarrow \hat{\mu}'.\text{locs}] = \hat{\mu}'''' \\ \langle \text{call } i, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_{L+1} \hat{\mu}'''' \end{array}} \quad (5.15)$$

$$\begin{array}{c}
\text{acc}.\lambda\chi. \frac{\langle \chi, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} \text{L} \hat{\mu}'}{\langle \chi, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} \text{Lacc} \sqcup \hat{\mu}'} = f \\
\text{pop}(\hat{\mu}) = \hat{\mu}_1, \hat{\iota}_t \quad \hat{\mu}_1 \uparrow \text{in}^* = \hat{\mu}_2, \hat{\nu}_i \\
\sqcup \{ \hat{\mu}_2.\text{reftab}(i) \mid i \in \gamma_i(\hat{\iota}_t) \} = I_n \\
\{ c \mid i \times o \times c \in \Delta \wedge i = \text{in}^* \wedge o = \text{out}^* \} = \delta_t \\
\hat{\mu}_2[\text{locs} \leftarrow \Phi(\hat{\nu}_i)][\text{stack} \leftarrow \text{nil}] = \hat{\mu}_3 \\
\{ \text{fst}(\Delta(i)) \mid i \in I_n \} = \delta_i \quad \delta_t \cap \delta_i = \delta \\
\text{fold}(f, \delta, \perp) = \hat{\mu}_4 \\
\hat{\mu}_4 \uparrow \text{out}^* = \hat{\nu}_o \\
\text{call_indirect}^{\#} \frac{\hat{\mu}_4[\text{stack} \leftarrow \hat{\mu}_2.\text{stack} \downarrow \hat{\nu}_o][\text{locs} \leftarrow \hat{\mu}_2.\text{locs}] = \hat{\mu}'}{\langle \text{call_indirect}(\text{in}^* \text{out}^*), \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} \text{L+1} \hat{\mu}'}
\end{array} \quad (5.16)$$

5.3 SOUNDNESS OF ABSTRACT BOUNDED SEMANTICS

In this section, we tackle issue **(i)** of our proof schema (see [Section 5.1](#)): proving that the bounded version of our semantics soundly approximates all L-long concrete derivations.

Theorem 3 (Soundness of bounded semantics). *For any $S \subseteq \mathbb{ST}$, $\alpha : \mathcal{P}(\mathbb{ST}) \rightarrow \mathbb{ST}^{\#}$, $\gamma : \mathbb{ST}^{\#} \rightarrow \mathcal{P}(\mathbb{ST})$, and for any $\mu \in S$ if $\langle c, \Delta, \iota, \mu \rangle \Longrightarrow \mu'$ can be proved with at most L derivations, then $\langle c, \Delta, \iota, \alpha(S) \rangle \xRightarrow{\#} \text{L} \hat{\mu}'$ and $\mu' \in \gamma(\hat{\mu}')$.*

Proof. Let $S \subseteq \mathbb{ST}$ and $\hat{\mu} = \alpha(S)$. We prove by induction on L that for any $\mu \in S$, if $\langle c, \Delta, \iota, \mu \rangle \Longrightarrow \mu'$ can be proved with at most L derivations then $\langle c, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} \text{L} \hat{\mu}'$ and $\mu' \in \gamma(\hat{\mu}')$.

BASE OF THE INDUCTION (L=1). If $\langle c, \Delta, \iota, \mu \rangle \Longrightarrow \mu'$ has been proved with a single rule, it means we have used one of the axioms in the concrete semantics of [Section 4.4](#). This means one of the following cases should be considered:

(c=nop) Directly from rule [Eq. \(4.18\)](#), we have that:

$$\langle \text{nop}, \Delta, \iota, \mu \rangle \Longrightarrow \mu$$

By applying rule [Eq. \(5.2\)](#) we have that:

$$\langle \text{nop}, \Delta, \iota, \alpha(S) \rangle \xRightarrow{\#} 1 \hat{\mu}$$

Finally, since $\mu \in S$ and $\hat{\mu} = \alpha(S)$, directly from the definition of α and γ we have that $\mu \in \gamma(\hat{\mu})$.

(c=return) From rule [Eq. \(4.27\)](#), we have that:

$$\langle \text{return}, \Delta, \iota, \mu \rangle \Longrightarrow \mu$$

By applying rule in [Eq. \(5.3\)](#) we have that:

$$\langle \text{nop}, \Delta, \iota, \alpha(S) \rangle \xRightarrow{\#}_1 \hat{\mu}$$

Since $\mu \in S$ and $\hat{\mu} = \alpha(S)$, directly from the definition of α and γ we have that $\mu \in \gamma(\hat{\mu})$.

(c=drop) Directly from rule Eq. (4.28), there exists μ' and v such that $\text{pop}(\mu) = \mu', v$, and

$$\langle \text{drop}, \Delta, \iota, \mu \rangle \Longrightarrow \mu'$$

By definition of α and γ , and from the assumptions that $\mu \in S$ and $\hat{\mu} = \alpha(S)$, we have that there exists $\hat{\mu}'$ and $\hat{v}$ such that $\text{pop}(\hat{\mu}) = \hat{\mu}', \hat{v}$ and $\mu' \in \gamma(\hat{\mu}')$. The statement follows by apply rule Eq. (5.4):

$$\langle \text{drop}, \Delta, \iota, \alpha(S) \rangle \xRightarrow{\#}_1 \hat{\mu}'$$

(c=i32.const n) From rule Eq. (4.29), there exists v and μ' such that $\llbracket n \rrbracket \mu = v$ and $\text{push}(\mu, v) = \mu'$ and

$$\langle \text{i32.const } n, \Delta, \iota, \mu \rangle \Longrightarrow \mu'$$

By applying rule Eq. (5.5) we have that:

$$\langle \text{i32.const } n, \Delta, \iota, \alpha(S) \rangle \Longrightarrow \hat{\mu}'$$

where $\hat{\mu}' = \text{push}(\hat{\mu}, \alpha(\{v\}))$. For any $S \subseteq \mathbb{ST}$ and for any value v , we have that $\alpha(\text{push}(S, v)) = \text{push}(\alpha(S), \alpha(\{v\}))$. Hence, $\mu' \in \gamma(\hat{\mu}')$.

(c=i32.binop) From rule Eq. (4.30), we have that there exists $\mu', \mu'', \mu''', v, v'$ and v'' such that $\text{pop}(\mu) = \mu', v$, $\text{pop}(\mu') = \mu'', v'$, $\text{binop}(v, v') = v''$, $\text{push}(\mu'', v'') = \mu'''$ and

$$\langle \text{i32.binop}, \Delta, \iota, \mu \rangle \Longrightarrow \mu'''$$

since $\mu \in S$ and $\hat{\mu} = \alpha(S)$, there exists $\hat{\mu}', \hat{\mu}'', \hat{\mu}''', \hat{v}, \hat{v}'$ and $\hat{v}''$ such that $\text{pop}(\hat{\mu}) = \hat{\mu}', \hat{v}$, $\text{pop}(\hat{\mu}') = \hat{\mu}'', \hat{v}'$, $\text{binop}(\hat{v}, \hat{v}') = \hat{v}''$, $\text{push}(\hat{\mu}'', \hat{v}'') = \hat{\mu}'''$. Moreover, due to the soundness of pop , push and binop we also have that $\mu' \in \gamma(\hat{\mu}')$, $\mu'' \in \gamma(\hat{\mu}'')$ and $\mu''' \in \gamma(\hat{\mu}''')$. Finally, by using rule Eq. (5.6):

$$\langle \text{binop}, \Delta, \iota, \alpha(S) \rangle \xRightarrow{\#}_1 \hat{\mu}'$$

(c=i32.unop) From rule Eq. (4.31), we have that there exists μ', μ'', v and v' such that $\text{pop}(\mu) = \mu', v$, $\text{unop}(v) = v'$, $\text{push}(\mu', v') = \mu''$ and

$$\langle \text{i32.unop}, \Delta, \iota, \mu \rangle \Longrightarrow \mu''$$

since $\mu \in S$ and $\hat{\mu} = \alpha(S)$ there exists $\hat{\mu}', \hat{\mu}'', \hat{v}$ and $\hat{v}'$ such that $\text{pop}(\hat{\mu}) = \hat{\mu}', \hat{v}$, $\text{unop}(\hat{v}) = \hat{v}'$, $\text{push}(\hat{\mu}', \hat{v}') = \hat{\mu}''$. Moreover, due to the soundness of pop , push and unop we also have that $\mu' \in \gamma(\hat{\mu}')$ and $\mu'' \in \gamma(\hat{\mu}'')$. Finally, by using rule Eq. (5.7):

$$\langle \text{unop}, \Delta, \iota, \alpha(S) \rangle \xRightarrow{\#}_1 \hat{\mu}'$$

(c=132.load) From rule Eq. (4.32), we have that there exists μ', μ'', a such that $\text{pop}(\mu) = \mu', a$, $\text{push}(\mu', \mu'.\text{linmem}(a)) = \mu''$ and

$$\langle \text{i32.load}, \Delta, \iota, \mu \rangle \Longrightarrow \mu''$$

since $\mu \in S$ and $\hat{\mu} = \alpha(S)$ there exists $\hat{\mu}', \hat{\mu}''$ and $\hat{a}$ such that $\text{pop}(\hat{\mu}) = \hat{\mu}', \hat{a}$, $\text{push}(\hat{\mu}', \widehat{\mu'.\text{linmem}(\hat{a})}) = \hat{\mu}''$. Moreover, due to the soundness of pop , push and the $\text{linmem}(\cdot)$ lookup we also have that $\mu' \in \gamma(\hat{\mu}')$ and $\mu'' \in \gamma(\hat{\mu}'')$. Finally, by using rule Eq. (5.8):

$$\langle \text{i32.load}, \Delta, \iota, \alpha(S) \rangle \xRightarrow{\#}_1 \hat{\mu}'$$

(c=132.store) From rule Eq. (4.33), we have that there exists μ', μ'', μ''', v and a such that $\text{pop}(\mu) = \mu', v$, $\text{pop}(\mu') = \mu'', a$, $\mu''.\text{linmem}[a \setminus v] = \mu'''$ and

$$\langle \text{i32.store}, \Delta, \iota, \mu \rangle \Longrightarrow \mu'''$$

since $\mu \in S$ and $\hat{\mu} = \alpha(S)$ there exists $\hat{\mu}', \hat{\mu}'', \hat{\mu}'''$, $\hat{a}$ and $\hat{v}$ such that $\text{pop}(\hat{\mu}) = \hat{\mu}', \hat{v}$, $\text{pop}(\hat{\mu}') = \hat{\mu}'', \hat{a}$, $\widehat{\mu''.\text{linmem}[\hat{a} \setminus \hat{v}]} = \hat{\mu}'''$. Moreover, due to the soundness of pop and the d operation $\widehat{\text{linmem}[\cdot \setminus \cdot]}$ we also have that $\mu' \in \gamma(\hat{\mu}')$, $\mu'' \in \gamma(\hat{\mu}'')$ and $\mu''' \in \gamma(\hat{\mu}''')$. Finally, by using rule Eq. (5.9):

$$\langle \text{i32.store}, \Delta, \iota, \alpha(S) \rangle \xRightarrow{\#}_1 \hat{\mu}'$$

□

Soundness of $(c_1; c_2)$.

$$\begin{array}{c}
\frac{}{\langle c_1, \Delta, \iota, \mu \rangle \Rightarrow \mu_1} \\
\Rightarrow \{IH\} \\
\frac{}{\langle c_1, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_{\text{L}} \sum_{i=0}^n i : \hat{\mu}'_i} \wedge \mu_1 \in \gamma(\hat{\mu}'_0) \\
\Rightarrow \{\text{Concrete Semantics}\} \\
\frac{}{\langle c_2, \Delta, \iota, \mu_1 \rangle \Rightarrow \mu'} \\
\Rightarrow \{IH\} \\
\frac{}{\langle c_2, \Delta, \iota, \hat{\mu}'_0 \rangle \xRightarrow{\#}_{\text{L}} \sum_{i=0}^n i : \hat{\mu}''_i} \wedge \mu' \in \gamma(\hat{\mu}''_0) \\
\Rightarrow \frac{}{\langle c_1; c_2, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_{\text{L}+1} \sum_{i=0}^n i : \hat{\mu}''_i} \wedge \mu' \in \gamma(\hat{\mu}''_0)
\end{array}$$

□

Soundness of $(block)$.

$$\begin{array}{c}
\frac{}{\langle c, \Delta, \text{nop} \gg \iota, \mu \rangle \Rightarrow \mu'} \\
\Rightarrow \{IH\} \\
\frac{}{\langle c, \Delta, \text{nop} \gg \iota, \hat{\mu} \rangle \xRightarrow{\#}_{\text{L}} \sum_{i=0}^n i : \hat{\mu}'_i} \wedge \mu' \in \gamma(\hat{\mu}'_0) \\
\Rightarrow \{\text{Abstract Semantics}\} \\
\frac{}{\langle \text{block } bt \ c, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_{\text{L}+1} 0 : (\hat{\mu}'_0 \sqcup \hat{\mu}'_1) + \sum_{i=2}^n (i-1) \hat{\mu}'_i} \\
\text{and } \mu' \in \gamma(\hat{\mu}'_0 \sqcup \hat{\mu}'_1)
\end{array}$$

□

Soundness of $(loop)$.

$$\begin{array}{c}
\frac{}{\langle c, \Delta, (\text{loop } bt \ c) \gg \iota, \mu \rangle \Rightarrow \mu'} \\
\Rightarrow \{IH\} \\
\frac{}{\langle c, \Delta, (\text{loop } bt \ c) \gg \iota, \hat{\mu} \rangle \xRightarrow{\#}_{\text{L}} \sum_{i=0}^n i : \hat{\mu}'_i} \wedge \mu' \in \gamma(\hat{\mu}'_0) \\
\Rightarrow \{\text{Abstract Semantics}\} \\
\frac{}{\langle \text{loop } bt \ c, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_{\text{L}+1} 0 : (\hat{\mu}'_0 \sqcup \hat{\mu}'_1) + \sum_{i=2}^n (i-1) \hat{\mu}'_i} \\
\text{and } \mu' \in \gamma(\hat{\mu}'_0 \sqcup \hat{\mu}'_1)
\end{array}$$

□

Soundness of (br) .

$$\begin{array}{c}
\frac{\iota(n) = c \quad \langle c, \Delta, \iota, \mu \rangle \Longrightarrow \mu'}{\langle br\ n, \Delta, \iota, \mu \rangle \Longrightarrow \mu'} \\
\Longrightarrow \{IH\} \\
\\
\frac{}{\langle c, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_L \sum_{i=0}^n i : \hat{\mu}'_i} \wedge \mu' \in \gamma(\hat{\mu}'_0 \sqcup \hat{\mu}'_{n+1}) \\
\Longrightarrow \{\text{Abstract Semantics}\} \\
\\
\frac{}{\langle br\ n, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#}_{L+1} (n+1 : (\hat{\mu}'_0 \sqcup \hat{\mu}'_{n+1}) + \sum_{i=n+2}^x i : \hat{\mu}'_i)} \\
\text{and } \mu' \in \gamma(\hat{\mu}'_0 \sqcup \hat{\mu}'_{n+1})
\end{array}$$

□

Soundness of (br_if) . First part:

$$\begin{array}{c}
pop(\mu) = \mu', v \quad (v \equiv 0) \\
\\
\frac{}{\langle br_if\ n, \Delta, \iota, \mu' \rangle \Longrightarrow \mu'} \\
\Longrightarrow \{\text{Def. } \alpha, \gamma, \text{Def. } pop(\cdot)\} \\
(*) \quad pop(\hat{\mu}) = \hat{\mu}_1, \hat{v} \wedge v \in \gamma(\hat{v}) \\
\Longrightarrow \{IH\} \\
\\
\frac{}{\langle c, \Delta, \iota, \hat{\mu}_1 \rangle \xRightarrow{\#}_L \sum_{i=0}^n i : \hat{\mu}'_i} \\
\Longrightarrow \{\text{Abstract Semantics}\} \\
\\
\frac{}{\langle br_if\ n, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} 0 : \hat{\mu}_1 + (n+1 : (\hat{\mu}_j) + \sum_{i=n+2}^x i : \hat{\mu}'_i)} \\
\text{and } \mu' \in \gamma(\hat{\mu}_1)
\end{array}$$

Second part:

$$\begin{array}{c}
pop(\mu) = \mu_1, v \quad (v \neq 0) \\
\\
\frac{}{\langle br_if\ n, \Delta, \iota, \mu_1 \rangle \Longrightarrow \mu'} \\
\Longrightarrow \{\text{Def. } \alpha, \gamma, \text{Def. } pop(\cdot)\} \\
(*) \quad pop(\hat{\mu}) = \hat{\mu}_1, \hat{v} \wedge v \in \gamma(\hat{v}) \\
\Longrightarrow \{IH\} \\
\\
\frac{}{\langle c, \Delta, \iota, \hat{\mu}_1 \rangle \xRightarrow{\#}_L \sum_{i=0}^n i : \hat{\mu}'_i} \\
\Longrightarrow \{\text{Abstract Semantics}\} \\
\\
\frac{}{\langle br_if\ n, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} 0 : \hat{\mu}_1 + (n+1 : (\hat{\mu}_j) + \sum_{i=n+2}^x i : \hat{\mu}'_i)} \\
\text{and } \mu' \in \gamma(\hat{\mu}_j)
\end{array}$$

□

Soundness of Call. If $\langle \text{call } i, \Delta, \iota, \mu \rangle \Longrightarrow \mu''''$ (Equation (4.25)):

1. $\Delta(i) = T_i \times T_o \times c$;
2. $\mu \uparrow T_i = \mu' \times \vec{v}_i$;
3. $\mu'[\text{locs} \leftarrow \Phi(\vec{v}_i)][\text{stack} \leftarrow \text{nil}] = \mu''$;
4. $\langle c, \Delta, \{0 \rightarrow \text{nop}\}, \mu'' \rangle \Longrightarrow \mu'''$;
5. $\mu''' \uparrow T_o = \vec{v}_o$;
6. $\mu'''[\text{stack} \leftarrow \mu'.\text{stack} \downarrow \vec{v}_o][\text{locs} \leftarrow \mu'.\text{locs}] = \mu''''$.

Moreover, from the fact that $\mu \in \gamma(\hat{\mu})$, due to inductive hypotheses and due to definition of functions α and γ :

1. $\hat{\mu} \uparrow T_i = \hat{\mu}' \times \vec{v}_i$ and $\vec{v}_i \in \gamma(\vec{v}_i)$;
2. $\hat{\mu}'[\text{locs} \leftarrow \Phi(\vec{v}_i)][\text{stack} \leftarrow \text{nil}] = \hat{\mu}''$ and $\mu'' \in \gamma(\hat{\mu}'')$;
3. $\langle c, \Delta, \{0 \rightarrow \text{nop}\}, \mu'' \rangle \xRightarrow{\#} \text{L} \hat{\mu}'''$ and $\mu''' \in \gamma(\hat{\mu}_0''')$;
4. $\mu'''' \in \gamma(\hat{\mu}_0''')$.

□

Soundness of call_indirect. The proof follows exactly as in the previous case by the inductive hypotheses, and by observing that if $\langle \text{call_indirect } (in^* out^*), \Delta, \iota, \mu \rangle \Longrightarrow \mu''''$ then there exists an index function i referenced by the content in the top of the current stack in μ . Indeed, since $\mu \in \gamma(\hat{\mu})$ it means that an abstract value containing the same reference is on the top of $\hat{\mu}$ and The abstract semantics of $\text{call_indirect } (in^* out^*)$ is obtained as the *folding* (see Definition 38) of the abstract evaluations of all the function indexes resolved the aforementioned abstract references that are coherent with the requested type $(in^* out^*)$. □

5.4 TERMINATION AND SOUNDNESS OF FIXPOINT SEMANTICS

5.4.1 Termination of fixpoint semantics

This section tackles point (ii) of our proof schema, introduced in Section 5.1, that is, to prove the termination of our fixpoint semantics for all possible derivations. We already defined our fixpoint machinery as two labeled transitions $\xrightarrow{\#}$ and $\xrightarrow{\#}$, with the respective rules described in Section 4.5.2.

Theorem 4 (Termination of the fixpoint semantics).

$$\forall \theta, \kappa, c, \Delta, \iota, \hat{\mu}$$

$$\theta, \kappa \vdash \langle c, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} \left(\sum_{i=0}^n i : \hat{\mu}'_i \right), \kappa', g$$

always terminates.

Proof. Let us assume that there exist $\theta, \kappa, c, \Delta, \iota, \hat{\mu}$ such that there exist a series of infinite derivations of $\xRightarrow{\#}$ from $\theta, \kappa \vdash \langle c, \Delta, \iota, \hat{\mu} \rangle$. This is possible only if the rules *call*, *call_indirect*, or *loop* are applied infinitely often.

Consequently, rules *iter1-4* and *fix3-4* are applied infinitely often. However, this is not possible. Indeed, since the program is finite and is finite the number of variables and memory locations, eventually, a stable result is reached due to the application of the widening operators¹. Hence, either rule *fix1* or *fix2* is applied, and the derivation terminates. $\square$

¹Note that the stack widening operator ∇_{sk} uses an upper bound operator to force ascending chains of recurrent calls.

5.4.2 Soundness of fixpoint semantics (w.r.t. bounded semantics)

Having proved termination in the preceding section, in this section we focus on proving point (iii) of our proof schema (see Section 5.1). After this proof, we can combine them all to prove the *original* assertion of soundness of the fixpoint semantics w.r.t. the concrete semantics.

Theorem 5 (Soundness of fixpoint semantics w.r.t. Bounded semantics).

$$\begin{aligned} & \forall c, \iota, \Delta, \hat{\mu}, \sum_{i=0}^n i : \hat{\mu}'_i, \sum_{i=0}^n i : \hat{\mu}''_i \\ & \langle c, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} \perp \sum_{i=0}^n i : \hat{\mu}'_i \text{ and} \\ & \theta, \kappa \vdash \langle c, \Delta, \iota, \hat{\mu} \rangle \xrightarrow{\#} (\sum_{i=0}^n i : \hat{\mu}''_i), \kappa', g \\ & \text{such that } \sum_{i=0}^n i : \hat{\mu}'_i \sqsubseteq \sum_{i=0}^n i : \hat{\mu}''_i \end{aligned}$$

That is, for all post-fixpoints proven by the bounded semantics, there exists a corresponding post-fixpoint proven by the fixpoint semantics that is greater or equal.

Proof. The proof proceeds easily by induction on the bound L of $\xRightarrow{\#} \perp$ and by observing that for any $\theta, \kappa, c, c', \iota, \hat{\mu}_1, \hat{\mu}_2, \hat{\mu}'_1, \hat{\mu}'_2, g, \kappa'$ and κ'' if $\theta, \kappa[c', \iota', \hat{\mu}_1 \mapsto \hat{\mu}'_1] \vdash \langle c, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} (\sum_{i=0}^n i : \hat{\mu}'_i), \kappa', g$ then $\theta, \kappa[c', \iota', \hat{\mu}_2 \mapsto \hat{\mu}'_2] \vdash \langle c, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} (\sum_{i=0}^n i : \hat{\mu}''_i), \kappa'', g''$ and $(\sum_{i=0}^n i : \hat{\mu}'_i) \sqsubseteq (\sum_{i=0}^n i : \hat{\mu}''_i)$. This property follows directly from the definition of $\xRightarrow{\#}$ (e.g., fixpoint semantics). $\square$

Theorem 6 (Soundness of the fixpoint semantics).

$$\begin{aligned} & \forall c, \iota, \Delta, \mu, \hat{\mu} : \mu \in \gamma(\hat{\mu}) \\ & \text{if } \langle c, \Delta, \iota, \mu \rangle \Longrightarrow \mu' \text{ then } \theta, \kappa \vdash \langle c, \Delta, \iota, \hat{\mu} \rangle \xRightarrow{\#} (\sum_{i=0}^n i : \hat{\mu}_i), \kappa', g \end{aligned}$$

such that $\mu' \in \gamma(\hat{\mu}'_0)$.

We prove **Theorem 6** by combining the proofs laid out so far. Following the proof schema **Proposition 1**, we proved: **Theorem 5** (e.g., soundness of bounded semantics); **Theorem 4** (e.g., fixpoint semantics always terminates), and **Theorem 6** (e.g., soundness of the fixpoint semantics w.r.t. bounded semantics).

Proof. By combining **Theorems 4 to 6**:

Let us assume that there exist $\theta, \kappa, c, \Delta, \iota, \mu, \hat{\mu}$ such that $\mu \in \gamma(\hat{\mu})$, if

$$\begin{aligned} \langle c, \Delta, \iota, \mu \rangle &\Longrightarrow \mu' \\ \text{and } \langle c, \Delta, \iota, \hat{\mu} \rangle &\xRightarrow{\#} \perp (\sum_{i=0}^n i : \hat{\mu}'_i) \\ \text{and } \theta, \kappa \vdash \langle c, \Delta, \iota, \hat{\mu} \rangle &\xRightarrow{\#} (\sum_{i=0}^n i : \hat{\mu}'_i), \kappa', g \end{aligned}$$

we prove that, since $\mu \in \gamma(\hat{\mu})$, and $\xRightarrow{\#} \perp$ soundly approximates $\Longrightarrow$ (e.g., $\Longrightarrow \sqsubseteq \xRightarrow{\#} \perp$), and $\xRightarrow{\#} \sqsubseteq \xRightarrow{\#} \perp$, therefore $\Longrightarrow \sqsubseteq \xRightarrow{\#} \perp \sqsubseteq \xRightarrow{\#}$ and, as such, $\Longrightarrow \sqsubseteq \xRightarrow{\#}$. $\square$

We follow the formal proof of **Proposition 1** with a paragraph containing an intuitive explanation of why the fixpoint semantics soundly approximate the concrete semantics.

INTUITION ON SOUNDNESS OF FIXPOINT SEMANTICS. At the beginning of each derivation of the fixpoint semantics (e.g., rule *fix4*) under a newly updated θ' , we request a fixed point iteration by $\overset{\#}{\Phi}$. Each rule of $\overset{\#}{\Phi}$ performs a step S with $\xRightarrow{\#}$, producing a $(\sum_{i=0}^n i : \hat{\mu}'_i), \kappa', g$. Case *iter1* (e.g., no back-edge discovered) does not contribute to this proof, so we ignore it. We then look up on κ' on all cases of *iter2-4*, obtaining the information gathered so far about the known converging executions, starting from $\perp$ (e.g.: *seeded* by rule *fix3* when it detects a true back-edge). After performing step S , we widen the previously known knowledge with the freshly discovered one via ∇ , that over-approximates the abstract join and enforces a finite height (e.g., the plus/minus infinite bounds) on infinite-height domains. If this does not yield a growing result, the iteration for this call returns a bottom, and we know that this abstract interpretation will diverge, hence the concrete interpretation diverges as well.

In the concrete realm, we would just consider the newly discovered information, if any, so this proves that we do not lose soundness when applying the ∇ operator: we are merely collecting all information as if it were a join allowing for guaranteed (and accelerated) convergence.

This request for iteration is applied recursively (e.g., recursive call to $\overset{\#}{\Phi}$ in *iter4*) with the widened result marked unstable in κ'' until the result stops growing (e.g., the old result is included or equal to the widened one, done in the $\sqsubseteq$ constraint in *iter2-3*). The analysis of the cardinality of g , and the manipulation of g in *iter3*, are necessary to enable this machinery to work with not only 1-recursive structures but

all k -recursive structures, keeping track of which result of calls to the interpreter stopped growing.

This proves that a result cannot indefinitely grow, and the fixpoint iteration terminates when the result stops growing, and, due to the properties of the widening, that it is sound w.r.t. the concrete realm.

6

From in Vitro to in Vivo

6.1 DIRECT IMPACT

Taking the analysis from *in vitro* to *in vivo*, from a paradigm of closed modules to modules interacting with an environment, requires first a deeper understanding of what changes from the two paradigms. This chapter will attempt to close this gap, better explaining the concepts hinted in [Sections 1.4](#) and [2.3](#). In particular, we will cover how the host can directly impact the module state, and how it can indirectly mutate it via callbacks. The examples of direct impact described in this chapter will be the possible interactions of JavaScript and WebAssembly, via the WebAssembly JavaScript API¹.

*“If you know the enemy and know yourself,
you need not fear the result of a hundred
battles.”*
—Sun Tzu, The Art of War.

¹<https://www.w3.org/TR/wasm-js-api-2>

6.1.1 *Linear Memory*

When a WebAssembly module exports its linear memory, the host can access and mutate it via typed array views (e.g., `Uint8Array`). This shared memory buffer serves as a direct channel through which JavaScript can read and write values that affect module computation, as shown in [Listing 6.1](#).

```
1 const memory = instance.exports.memory;  
2 const view = new Uint8Array(memory.buffer);  
3 view[0] = 42;
```

Listing 6.1: Direct memory manipulation.

First, the exported memory object is accessed, then the `buffer` object it contains is used to create an array view. The modification to `view` in the example in [Listing 6.1](#) immediately impacts the WebAssembly module, as memory instructions (e.g.: `load`, `store`) reference the same buffer accessed via `memory.buffer`. Consequently, JavaScript’s direct writes to memory likely will influence the logic and control flow within the module. This first example shows how two different hosts coupled with the same module might produce drastically different behaviors.

6.1.2 Reference Table

Modification of exported reference tables directly impact what functions are called via `call_indirect`.

```
1 const table = instance.exports.table;
2 table.set(0, someExportedFunction);
```

Listing 6.2: Direct table manipulation.

Listing 6.2 shows how an exported reference table may be directly altered. Modifying a reference table dynamically rewires the dispatch behavior of the module, allowing the host to redefine the module's interprocedural control flow at runtime.

6.1.3 Globals

Exported global variables are also part of what can be read and reassigned through the host. This provides another route for state manipulation, as shown in **Listing 6.3**.

```
1 const globalVar = instance.exports.counter;
2 globalVar.value = 10;
```

Listing 6.3: Direct global manipulation.

As oftentimes globals serve as flags, counters, or configuration toggles within the module, changing them externally may affect subsequent module execution, enabling reactive and externally-driven computational behavior, which can hinder the capability of an analyzer to produce both sound and complete behavior.

6.1.4 Injection

Though not a direct mutation in the strictest sense, WebAssembly allows importing functions defined in the host that the WebAssembly module invokes. These functions can perform side effects or alter shared state from outside the module. These functions can also be injected, as shown in **Listing 6.4**.

```
1 const imports = {
2   env: {
3     toggleFlag: () => { instance.exports.flag = !instance.exports.flag; }
4   }
5 };
6 const instance = await WebAssembly.instantiate(module, imports);
```

Listing 6.4: Behavior injection.

The module remains agnostic to the implementation details, but the host retains full control over the behavior of these imported functions. In effect, JavaScript acts as an authoritative controller of behavioral state, capable of intercepting or injecting logic during execution.

CLOSING REMARKS ON DIRECT IMPACT The aforementioned direct mutation vectors demonstrate how analyzing a module without consideration of its context can be meaningless, as the host can significantly impact the module’s state due to the weak encapsulation mechanism of WebAssembly. The techniques known in the library analysis field, such as the one proposed in [KM19], may be applicable. Such techniques might fall short when there is either a lack of knowledge about the host application that will operate a given WebAssembly module or an interest in analyzing a specific family of host-module applications.

[KM19] Kristensen and Møller, “Reasonably-most-general for JavaScript library analysis” clients

CALLING AND CALLBACK IMPACT While Section 6.1 discussed how the host code can directly impact the module state, there is also the possibility that the host could be calling exported functions, or having imported functions call back exported functions. In an assembly-like language it is almost a guarantee that, if a function in the module is called, this action will impact the global state, in particular the linear memory.

6.2 METHODOLOGIES OF ANALYSIS COMPARED

With added knowledge of how different host code can impact the semantics of a host-module application, we expand what we stated in Section 2.3.2 (Inconclusive or Unsound?) regarding possible ways to analyze WebAssembly applications. We briefly recall how we can approach analyzing a WebAssembly application:

- (i) assume nothing is known about the host;
- (ii) analyze the module under a set of assumptions about host behavior that, if held, make the analysis result sound;
- (iii) provide information about host behavior that the analyzer can leverage.

In light of the information in Section 6.1, approach (i) is, clearly, not a viable option. Option (ii) is an attractive option if we are concerned about analyzing a module as if it were a library, without knowledge of what possible hosts might use it. We explain this analysis methodology in Section 6.2.1. If instead our use-case is a specific host or family of host programs, an analysis methodology similar to (iii), or even a multi-language analysis, might be best suited, especially if we plan on leveraging the freedoms WebAssembly allows for. An additional viable option might include the verification of host code performing actions that fall under the *direct impact* category or, even better, automatically produce a behavioral specification by analyzing the host code. Specifically, the assumption of a Reasonably Most General Client [KM19] in the context of WebAssembly, as used in [Bra+23] does not capture a host writing to exported tables and memories.

[Bra+23] Brandl et al., “Modular Abstract Definitional Interpreters for WebAssembly”

6.2.1 *Reasonably Most General Clients Explained*

[KM19] Kristensen and Møller, “Reasonably-most-general clients for JavaScript library analysis”

[Rin+07] Rinetzky et al., “Modular Shape Analysis for Dynamically Encapsulated Programs”

[AKS15] Allen et al., “Combining type-analysis with points-to analysis for analyzing Java library source-code”

[Ste96] Steensgaard, “Points-to Analysis in Almost Linear Time”

[AL05] Andersen and Lee, “Program Analysis and Specialization for the C Programming Language”

[Bra+23] Brandl et al., “Modular Abstract Definitional Interpreters for WebAssembly”

Reasonably Most General Clients, RMGCs, are a concept introduced in [KM19]. They are a restricted version of Most General Clients (MGC) [Rin+07; AKS15] respectively developed for reasoning about heap structures and points-to ([Ste96; AL05]) analysis for Java libraries. MGCs are a set of assumptions that are made to allow for library analysis. Analyzing a library substantially differs from the more straightforward program analysis, as there is no knowledge of what a client may do with the library under analysis. MGCs are a default safe assumption about client behavior that allows the abstract client to use a library in any possible way. This approach is generally applicable to the analysis of languages that have strong encapsulation mechanisms. RMGCs were instead designed as a similar solution that restricts the behavioral freedom of languages with weaker encapsulation mechanisms. To be specific, RMGCs are tailored to JavaScript library analysis, but their theory applies to other dynamic languages as well, that, for instance, allow overwriting library functions. In this context, the freedom of MGCs would allow for actions that would be too impactful to have meaningful analysis results in general, and in the context of library usage, would be unrealistic.

In the context of WebAssembly, in [Bra+23] the authors use a RMGC that simulates host calls and exercises all interleavings of all exported functions in a loop. In each loop iteration, all exported functions are interpreted in isolation and their results joined to update the analysis state. The fixpoint algorithm iterates this loop until the analysis state is stable. The final states produced by the analysis soundly approximate all possible sequences of exported functions. Noting that a WebAssembly host can also write to exported tables and memory, the RMGC they adopted does not capture this behavior, which may cause the analysis result to be unsound for such clients.

6.3 A FIRST PROPOSAL

All we have just said reinforces the fact that we must know the aim of an analysis in order to perform practical and fruitful analyses. Understanding if the subject of an analysis is a library or library-like, methods like [KM19] employed by [Bra+23] are viable, trading soundness for practical precision, as such assumptions made by RMGCs make sense in the context of a library-like usage, and it is generally unknown how a developer might use such a library. But, if the target of an analysis is a specific WebAssembly-based host/module application, unless the host behaves as an RMGC, the analysis is bound to be potentially unsound, especially if there are hand-crafted functions that directly manipulate the memory/tables state. These are native features of WebAssembly and WebAssembly’s JavaScript API and should not be ignored. Given these issues, the use case we are interested in exploring is a bit related to multi-program analysis, and a first stepping stone to move in that direction is to analyze WebAssembly code by providing the analyzer with knowledge of the semantics of the module. In a nutshell, a tool like *sturdy* [Bra+23]

cares about library-like WebAssembly. In the next chapter, we will introduce our proposed *in vivo* approach. Our approach focuses on analyzing specific and well-defined applications, allowing for a guarantee on soundness under the assumption of a correctly specified semantics and possibly enhanced precision in light of semantic knowledge.

Part III

IN VIVO

7

wassilly

PREAMBLE

This chapter presents the more technical contribution of this work, *wassilly* [PRL26]. We introduce *wassilly*'s design choices, its functionalities, its Domain Specific Language for modeling external dependencies, and comment on its performance compared to similar known tools. We will also address its limitations, how to mitigate them, and the lessons learned from this work. Lessons that can help shape the future of analyzing WebAssembly code.

7.1 PILLS ON THE DESIGN OF WASSILLY

At its core, *wassilly* (WebAssembly Ain't So SILLY) [PRL26] performs value analysis. Recalling Chapter 2, value analysis is necessary to construct a precise call graph for Wasm. Alternative approaches, such as Class Hierarchy Analysis [DGC95], or similar approaches [BS96; TP00], or reference set-based points-to analysis [AL05; Ste96; BS16], are not viable when applied to Wasm due to its lack of nominal typing, dynamic dispatch through tables, and access to arbitrarily-addressed linear memory. *wassilly* does whole-program analysis: possible entry points to the Wasm module are deduced, first by looking for a `start` function if present, then by observing the `exports` section in the module. During the analysis the function calls are accumulated in a graph. *wassilly* does not provide any proof of the absence of runtime errors that may arise. Computations that would yield certain errors (i.e., division by constant 0) are instead represented as $\perp$, as detection of runtime errors is not of interest to this work.

7.1.1 Intended Use of wassilly

Any analysis performed by *wassilly* begins with two key inputs: a WebAssembly module and, if imported resources are present, a mandatory specification file. The specification encodes behavior and memory content of the external dependencies of the module, such as imported or injected functions or external state (i.e., values for imported globals). The specification language is described in Section 7.2. Besides the writing

"It's over, Anakin. I have the high ground."
—Obi-Wan Kenobi, Star Wars: Episode III.

[PRL26] Paccamiccio et al., "wassilly: Spec-enhanced Call Graph Analysis for WebAssembly"

[DGC95] Dean et al., "Optimization of Object-Oriented Programs Using Static Class Hierarchy Analysis"

[BS96] Bacon and Sweeney, "Fast Static Analysis of C++ Virtual Function Calls"

[TP00] Tip and Palsberg, "Scalable propagation-based call graph construction algorithms"

[AL05] Andersen and Lee, "Program Analysis and Specialization for the C Programming Language"

[Ste96] Steensgaard, "Points-to Analysis in Almost Linear Time"

[BS16] Balatsouras and Smaragdakis, "Structure-Sensitive Points-To Analysis for C and C++"

of the specification, the analysis proceeds automatically and outputs a call graph, that can then be used for further analysis.

7.1.2 Component Architecture

wassilly is structured into interconnected components, organized in a layered architecture, that we show in [Figure 7.1](#). Following, we introduce the major components.

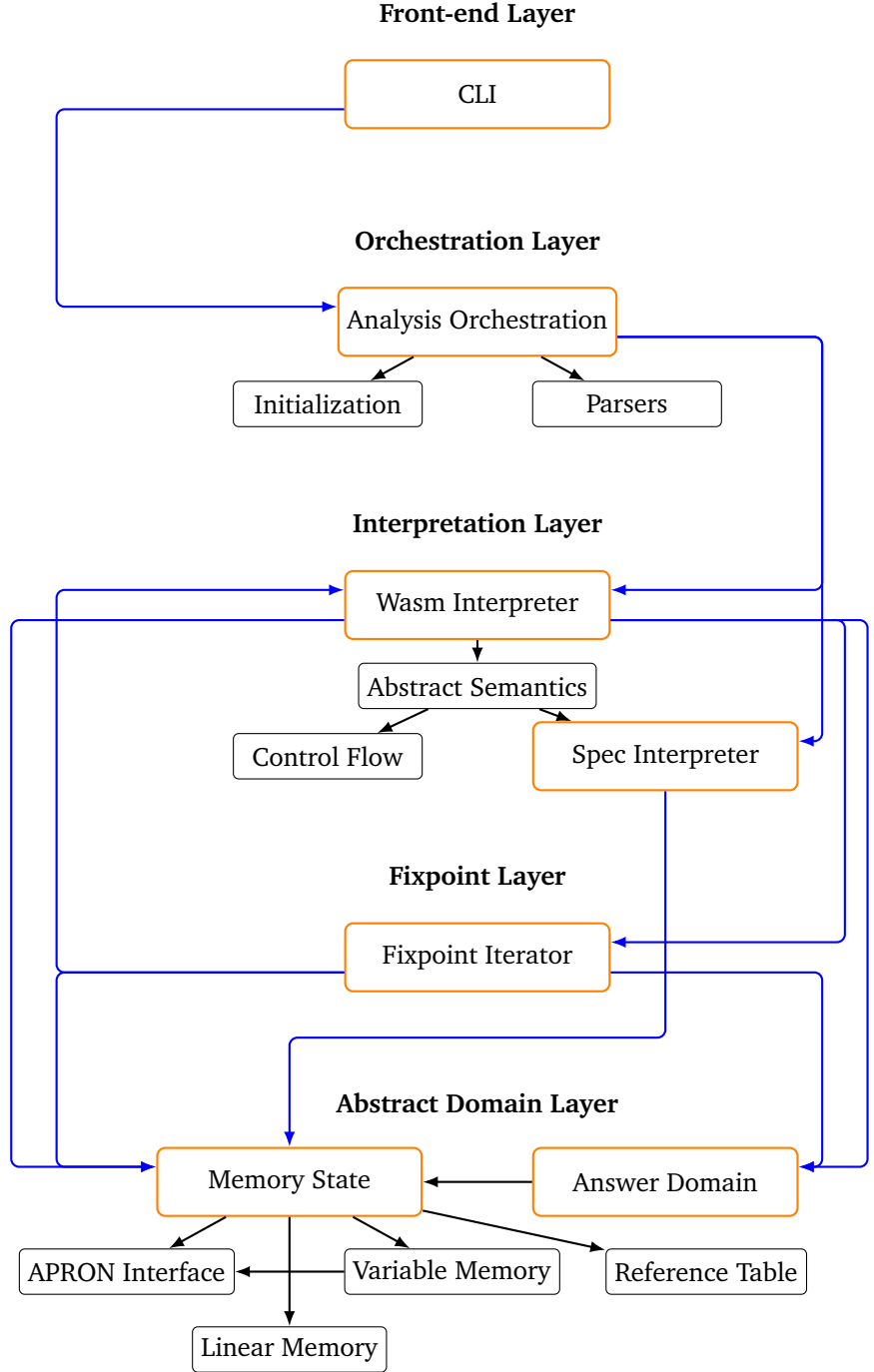


FIGURE 7.1: Component architecture of *wassilly*. Orange components are *public*, black components are *private*. Black and blue arrows represent intra- and inter-layer component interactions, respectively.

Front-end Layer

CLI The command-line interface is a front-end for interacting with *wassilly*, accepting WebAssembly binary files, specification files, and output configuration parameters. This component handles argument parsing and invokes the high-level analysis orchestration.

Orchestration Layer

ANALYSIS ORCHESTRATION The analysis orchestration module coordinates the overall analysis workflow. Internally, it uses subcomponents that handle abstract state initialization and parsing. Besides that, it interacts with the interpretation layer for analysis requests.

INITIALIZATION The initialization module is a subcomponent of the analysis orchestration component, and it is responsible for setting up the initial abstract state of the WebAssembly module, including memory initialization, global variable initialization, and table initialization. It processes data and element segments, as well as pre-invocation specification rules. To this end, it interacts with the interpretation layer via the orchestration module.

PARSING MODULE The parsing module is a subcomponent of the analysis orchestration component. It is solely responsible for parsing the Wasm bytecode and the specification file.

Interpretation Layer

WASM INTERPRETER The Wasm interpreter component walks the syntax tree of parsed Wasm programs and delegates the instruction semantics to the abstract semantics subcomponent. It interacts with the fixpoint iterator to detect possibly recursive calls to the interpreter. Besides that, it interacts with the memory state and answer domain components.

ABSTRACT SEMANTICS The core abstract interpreter implements the transfer functions for WebAssembly instructions, operating on abstract memory states and answers, from the abstract domain layer. It evaluates the big-step abstract semantics of WebAssembly bytecode. Delegates non-linear control-flow handling to the control flow subcomponent, and fixpoint construction to the fixpoint layer. Individual instruction semantics are implemented for most of the WebAssembly instruction set.

CONTROL FLOW The control flow module handles complex control flow constructs: branching, conditionals, and especially stack manipulations within the abstract interpreter. It implements path-sensitive analysis through abstract domain filtering on conditional branches.

SPEC INTERPRETER The specification interpreter module interprets specification terms and transforms the abstract state accordingly during the interpretation. As such, it interacts with the memory state component.

Fixpoint Layer

FIXPOINT ITERATOR The fixpoint iterator implements the core fixpoint computation algorithm using chaotic iteration with widening. It handles analysis of recursive functions and loops and ensures termination of the analysis. It interacts with the following components: Wasm interpreter, memory state, and answer domain.

Abstract Domain Layer

[JM09] Jeannet and Miné, “Apron: A Library of Numerical Abstract Domains for Static Analysis”

APRON INTERFACE The APRON [JM09] domain interface provides a wrapper around the APRON library’s abstract domains, exposing operations such as join, widening, meet, constraint filtering, and assignments.

MEMORY STATE The memory state is a product abstract domain representing the complete state of a WebAssembly program execution. It consists of four sub-components:

- **Operand Stack:** Abstract representation of the WebAssembly value stack.
- **Variable Memory:** Numeric abstract domain for local and global variables.
- **Linear Memory:** Byte-addressable memory abstraction.
- **Reference Table:** Abstract representation of function reference tables.

ANSWER DOMAIN The answer domain represents analysis results as a combination of natural continuation state, branch states for each live label, and return state. In our theory, this corresponds to the sequence of abstract states.

7.1.3 Symbolic Enhancements for Operand Stacks

[Min06] Miné, “Symbolic Methods to Enhance the Precision of Numerical Abstract Domains”

wassilly presents a minor deviation from the semantics presented in [Section 4.5](#) in how it treats abstract operands. We use symbolic constant propagation (see [Min06]) to enhance the precision of the operand stack and allow for some degree of relational properties, even if the underlying abstraction is based on intervals. This approach of avoiding directly evaluating WebAssembly expressions into values allows us to use ‘filtering’ functions (i.e. applying constraints to the abstract domain [Section 3.3.9](#)) that can greatly, and positively, affect the precision of the analysis in the presence of conditionals with little added costs. Another added benefit consists in possibly discarding unfeasible execution paths, possibly rendering the total analysis time smaller. We now explain how the symbolically enhanced abstract operand stack works, first by intuition and then in a more in-depth and formal way.

INTUITION The abstract operand stack requires the availability of an underlying abstract domain representing program variables. Using references to variables held in such a domain allows us to express relations as symbolic expressions or constraints.

This is achieved in three steps:

- (i) the result of evaluating any instruction that can never generate errors is a symbolic expression or constraint;
- (ii) when possible, we use symbolic expressions or constraints with references to variables in the abstract domain to represent the operands;
- (iii) when a program variable, the assignee, is targeted by an assignment, substitute all occurrences of such references to the variable in the operand stack with the abstract value the assignee may hold, then perform the assignment.

The first step is, on its own, a trivial performance optimization pass that allows for fewer invocations of the property extraction feature of the abstract domain. The second step helps the precision of the analysis when performing join, widening, and meet operations, without increasing the number of abstract variables, which will **always** be equal to the number of in-context program variables. Finally, the third step ensures the correctness of assignments: when a program variable is to be assigned, its references in the operand stack are substituted with its abstract value expressed as an interval, only to then perform the assignment.

7.1.4 Abstract Operands

Abstract operands represent an abstraction of operands in WebAssembly. In our reformulation, presented in [Section 4.2](#), operands correspond to values. We lift this into an abstraction where an operand can be either a symbolic expression representing a value, a symbolic Boolean expression, or a bottom $\perp$, as shown in [Listing 7.1](#).

```

1 type sym_operand =
2   | SymBoolExpr of sym_constr
3   | SymValExpr  of sym_val * numeric_type
4   | Bottom

```

Listing 7.1: Symbolic-enhanced operands.

The existence of the `SymBoolExpr` constructor is justified by how WebAssembly performs boolean evaluation, encoding `true` or `false` as `int32` values holding non-zero or zero, respectively.

Both `SymBoolExpr` and `SymValExpr` express symbolic values, allowing referencing variables in the abstract domain or to directly express values as *raw* intervals. As mentioned in the intuition in [Section 7.1.4](#) in point (ii), variable reads are treated similarly to references and, as point (iii) states, for the sake of analysis correctness, operands containing references to the assignee need a reference-to-value substitution before an assignment can take place. We describe the reference-to-value substitution algorithm in [Listing 7.2](#).

This naive implementation of our substitution algorithm recursively searches for a given reference in a symbolic expression. We can significantly optimize its performance by tracking the pseudo-references a given expression holds, reducing its time complexity with a relatively

```

Function substitute (ref, symexp, abs_dom) :
  if symexp is Binop(l, op, r) then
    | return Binop(substitute (ref, l, abs_dom), op,
    |   substitute (ref, r, abs_dom))
  else if symexp is Unop(o, op) then
    | return Unop(substitute (ref, o, abs_dom), op)
  else if symexp is Ref(r) then
    | if r = ref then
    |   | return Const(abs_dom(r))
    | else
    |   | return Ref(r)
    | end
  else if symexp is Const(c) then
    | return Const(c)

```

Listing 7.2: Pre-assignment substitution algorithm.

contained memory cost. We now define the abstract join $\sqcup$ and widening ∇ operators for symbolically-enhanced operands.

Definition 41 (Abstract operators for symbolically-enhanced operands).

$$\begin{aligned}
 (o_1, d_1) \sqcup (o_2, d_2) &= \begin{cases} o_1 & \text{if } o_1 = o_2 \\ d_1(o_1) \sqcup d_2(o_2) & \text{otherwise} \end{cases} \\
 (o_1, d_1) \nabla (o_2, d_2) &= \begin{cases} o_1 & \text{if } o_1 = o_2 \\ d_1(o_1) \nabla d_2(o_2) & \text{otherwise} \end{cases}
 \end{aligned}$$

where $d(o)$ is the numerical property of expression o in domain d .

At first glance, the definition might seem counterintuitive. Indeed, a new abstract domain d is never computed in the definition in **Definition 41**. This concern is instead handled by the computation of join and widen for abstract states: the new d is computed *after* joining the operands so that (structurally) equivalent symbolic expressions in d_1 and d_2 are not lost during the transformation as intervals, improving precision and keeping the pseudo-relational properties of symbolic expressions intact. Now, given that abstract operands have been endowed with their widening and join operators, the join and widening operators for the abstract stacks are, trivially, the application of either the operand widen or join to each element of the two same-length sequences, yielding a new one.

7.2 THE SPECIFICATION DSL

In this section, we introduce the DSL for external dependencies embedded in *wassilly*. This language is designed with the specific challenges of WebAssembly in mind. Wasm's runtime does not enjoy strong encapsulation; by design, host code may be seen as the *owner* of a running Wasm instance. It can influence instance semantics by direct manipulation (i.e., injection, direct table mutation) or opaque imports. This arrangement forces an analyzer either to make unsound assumptions or

to fall back to overly conservative approximations. Our DSL provides a middle ground: users can supply lightweight specifications describing how each imported function affects module-visible state. These specifications restore precision without necessarily requiring complete host models (for instance, the user can provide a conservative specification), enabling sound analysis of modules that depend on complex or partially unknown hosts.

Program	P	::=	<i>term</i> *
Term	term	::=	<i>func</i> <i>def</i> <i>preinv sides</i> * <i>importobj term</i>
Side effects	sides	::=	<i>effect tg</i> *
Target, Definition	tg, def	::=	<i>glob id type value</i> <i>memory id value value type</i> <i>table id (value → value)</i>
Function	func	::=	<i>func id funsig impl</i>
Func. Signature	funsig	::=	<i>param</i> * → <i>type</i> *
Parameter	param	::=	<i>name type</i>
Implication	impl	::=	<i>fix</i> <i>when cond {impl} else {impl}</i>
Condition	cond	::=	SYMBOLIC_CON+
Fixpoint	fix	::=	<i>res</i> * <i>sides</i> * (<i>calls fidx</i>)*
Result	res	::=	<i>type value</i>
Wasm Type	type	::=	<i>i32</i>
Abstract Value	value	::=	[<i>n</i> ; <i>n</i>] SYMBOLIC_EXP
Identifier	id, n, fidx	::=	INT_LITERAL
Name	name	::=	STRING_LITERAL

Listing 7.3: Syntax of the specification DSL.

The syntax of our specification Domain Specific Language is shown in [Listing 7.3](#). A program specification is a sequence of *term* statements. There are four possible *terms*: *func*, *def*, *preinv*, or *importobj*. We will describe each of them in the following section, with corollary examples.

7.2.1 Terms

FUNC The *func* statement is used to define summaries for imported functions in the form of conditionals, each associated with its fixpoint. Conditionals allow the application of constraints (linear inequalities) on the abstract state to improve precision. The result of the call to the specified function is then computed by joining the result of the conditional evaluation. A function specification (e.g., *func id funsig impl*) is characterized by the *func* keyword, an identifier, the function signature (using WebAssembly types), and an implication *impl*, consisting of two sub-rules: *fix*, and the conditional form *when cond {impl} else {impl}*.

The *fix* rule is composed by a triple consisting of: a vector of abstract values or symbolic expressions representing the return values, a sequence of *effect* statements, representing side effects and a *call* statement carrying the indices of what functions calls should be added to the call graph. *fix* is used to represent the end state of execution traces originated from a call to an exported function, with the conditional form *when* being useful in aiding precision. *effect* statements are quite trivial and will be shown later in [Section 7.2.3](#), as not to clutter the more relevant constructs of this language.

We show incrementally more precise examples of basic specifications of the Fibonacci function with the rules we have introduced so far.

```

1 func fibStub ([param i32 x] -> [i32])
2   i32 [0;PINF] calls 0

```

Listing 7.4: A loose definition of the Fibonacci function in our DSL.

In [Listing 7.4](#) we only state that the evaluation of *fibStub* returns the interval $[0; +\infty]$ and calls 0, which corresponds to the index of the *fibStub* function.

```

1 func fibStub2 ([param i32 x] -> [i32])
2   when x = 0 { i32 [0;0] }
3   else {
4     when x = 1 { i32 [1;1] }
5     else { i32 [1;PINF] calls 0 }
6   }

```

Listing 7.5: A more precise definition of the Fibonacci function in our DSL.

In addition to the capabilities of the specification in [Listing 7.4](#), in the specification of [Listing 7.5](#), the parameter *x* is compared to 0 and 1 so that a potentially more precise result can be obtained. For instance, if we were to evaluate this specification with *x* equal to $[0; 1]$ we would obtain a return value of $[0; 1]$.

```

1 func fibPrecise ([param i32 x] -> [i32])
2   when x >= 0 {
3     when x = 0 { i32 [0;0] }
4     else {
5       when x=1 { i32 [1;1] }
6       else { i32 [1;PINF] calls 0 }
7     }
8   }
9   else { i32 ⊥ calls 0 }

```

Listing 7.6: An even more precise definition of the Fibonacci function in our DSL.

In [Listing 7.6](#), parameter *x* is also checked for being greater or equal to 0 to provide a result, else $\perp$. These examples should highlight how expressing this kind of summary using conditional fixpoints might be beneficial for precision with little added cost.

DEF Rule *def*, definition, is comprised of *table* and *glob*, and are used to define the contents of imported tables and global variables respectively.

`IMPORTOBJ importobj` is the statement used to define objects that are to be imported in an already compiled module. To distinguish this from a WebAssembly `import` statement, we dub this interaction *injection*. Injection happens in one of the benchmarks from [Leh+23]. The host code portion of interest can be found in Listing 7.7.

[Leh+23] Lehmann et al., “That’s a Tough Call: Studying the Challenges of Call Graph Construction for WebAssembly”

```

1 const bin = fs.readFileSync('./main.wasm');
2 const impObj = {
3   'host': {
4     'print': (arg) => {
5       console.log(arg);
6     },
7   },
8 };
9 const r = await WebAssembly.instantiate(bin, impObj);

```

Listing 7.7: A function is *injected* in a pre-compiled WebAssembly module.

The benchmark injects a printing function `console.log`, that we know does not alter the state and returns nothing. We can express this interaction in our DSL with the code in Listing 7.8.

```

1 importobj func print ([param i32 x] -> [])

```

Listing 7.8: Function *injection* in *wassilly*’s specification language.

PREINV The `preinv` (pre-invocation) rule allows for the specification of a sequence of effect statements. These are used to model side effects happening between the creation of the instance and running the module from an entrypoint. In the host code of the same benchmark considered before, we find a mutation performed by the host on the reference table, highlighted in Listing 7.9.

```

1 result.instance.exports.table.set (
2   0, result.instance.exports.export2
3 );

```

Listing 7.9: The reference table is modified by the host

We can model this behavior using `preinv`, as shown in Listing 7.10.

```

1 preinv effect table tab ( 0 -> 2 );

```

Listing 7.10: `preinv` in action. Note that `tab` is the exported name of the reference table, and $0 \rightarrow 2$ means ‘set (index) 0 to 2’. This action changes what function index 0 references.

7.2.2 Intuitive Semantics of the DSL

The semantics of the specification language in Listing 7.3 can be divided into three groups: i) the ones performing assignment-like computations; ii) the ones performing conditional computations with side effects; iii) the ones *injecting* new objects into the module. The first group consists

of `table`, `glob`, `preinv`, and `effect`. `effect` and `preinv` allow modeling side effects on known structures, whereas `table` and `glob` are only used in the initialization phase, in which there is awareness of a given table or global variable allocated in the module but no knowledge of their initialization value(s). The second group consists of `func`, intuitively used to define functions by constructing the summary of the functions themselves and any downstream function call. The third group consists of `importobj` *term* alone, used to simulate the runtime *injection* of an object that is never mentioned or allocated in the module.

7.2.3 Modeling WASI Syscalls

Although the *opaque* direct impact of imported JavaScript functions (see [Section 6.1](#)) might be regarded as exceptional, WASI system calls constitute a notable exception to this general rule.

In fact, their functionality regularly relies on manipulating the linear memory of the model. We can state that, by design, and for efficiency reasons, WASI syscalls are *by definition* violating the RMGC contract [\[KM19\]](#). The good news is that these calls are easily identifiable and can be modeled using our DSL. We show an example of such specifications for `args_sizes_get`, `args_get`, and `proc_exit` in [Listing 7.11](#). These can all be modeled, easily, using `effect` statements performing memory mutations (e.g., `memmut`) writing a given value of a given type to a location (see line 3 of [Listing 7.11](#)). If a function is read-only w.r.t. the module state, we can just state it is an identity w.r.t. the module state with `id`.

[KM19] Kristensen and Møller, clients
“Reasonably-most-general
for JavaScript library analysis”

```

1 # we assume a maximum arg size of 10.
2 func args_sizes_get ([param i32 x param i32 y] -> [i32])
3   i32 [0;PINF]
4   effect memmut memory x [0;10] i32
5   effect memmut memory y [0;PINF] i32
6
7 func args_get ([param i32 x param i32 y] -> [i32])
8   i32 [0;PINF]
9   effect memmut memory x [0;10] i32
10  effect memmut memory y [0;PINF] i32
11
12 func proc_exit ([param i32 x] -> [])
13   id

```

Listing 7.11: Specifications of the WASI functions `args_sizes_get`, `args_get`, and `proc_exit`.

7.3 EXECUTION MODEL

wassilly assumes a simplified execution model: the result of the analysis is the join of the result of all the traces generated by separated analyses starting from each possible entry point (e.g., exported functions).

This is contrast with the actual execution model of WebAssembly, where the host code might be a mixed sequence of calls to the module, and modifications performed directly on the module. We simplify and propose this approach instead because, to our judgment, modeling

the behavior of unknown resources and interactions represents a more foundational challenge in the analysis of WebAssembly code.

This simplification is still pertinent because there is no fundamental difference if functions in the module are invoked more than once by the host: the resulting abstract state computed for the call to any given function f can become input for the call to another function g . This can include a pass where pre-invocation or post-invocation side effects are applied using the mechanism used by the pre-invocation rule of the DSL.

Extending this simplified execution model to a more realistic one is a matter of extending the DSL to model a sequence of calls from the host, as opposed to using exported functions as entry points, or approaching the analysis using a *RMGC loop* [Bra+23; KM19] (e.g., iterating and joining calls to the exported functions until stabilization is reached).

In short, assuming our host performs the sequence of module instantiation into (optional) side effects application into invocation, our implementation is sound and can be extended to more unrestricted scenarios. Given this limitation, the evaluation of *wassilly* is, for now, only relevant to “benchmark” programs, even though the analyzer is capable of analyzing, though without soundness guarantees, more realistic programs.

[Bra+23] Brandl et al., “Modular Abstract Definitional Interpreters for WebAssembly”

7.4 EXPERIMENTS AND CONSIDERATIONS

In this section, we discuss the results of the experimental evaluation performed against the benchmarks provided by [Leh+23] and compare our analysis results against the ones produced by *wassilly*, *sturdy* [Bra+23] and the rest of the tools already analyzed in [Leh+23]. Besides our implementation, *sturdy* is the only other option for a static semantics-aware analysis. In contrast, the analyzers compared in [Leh+23] restrict the possible targets of indirect calls based on a type-based syntactic analysis, or based on the mere content of `elem` segments (e.g., the static representation of Wasm tables). A more in-depth discussion on the approaches used by the various tools is available at Section 2.4 (WebAssembly Static Call Graph Construction). Then, considering the implications stated in Section 1.4 (Challenges in analysis of WebAssembly), we expand and articulate considerations about how host-module applications can be better analyzed.

[Leh+23] Lehmann et al., “That’s a Tough Call: Studying the Challenges of Call Graph Construction for WebAssembly”

7.4.1 Experimental Setup and Evaluation

The benchmark programs proposed in [Leh+23] cover all function indirection concerns in WebAssembly. The authors provide a *ground truth* containing the edges of the call graph, with knowledge of what edges are actually reachable from potential entry points. We use this ground truth to determine whether the analysis is sound and precise. On that ground truth, we consider only reachable edges and vertices of the call graph: this is a valid approach for constructing call graphs, as we cannot traverse unreachable edges and can safely ignore them. In short, we consider the set of reachable edges and the set of reachable functions in

the ground truth $\mathbb{E}_{gt}, \mathbb{F}_{gt}$, and compare it for set inclusion to the results of the analyses $\mathbb{E}_a, \mathbb{F}_a$.

Thus, we define the order relation necessary to obtain a sound analysis as:

$$\mathbf{S} : \mathbb{E}_{gt} \subseteq \mathbb{E}_a \wedge \mathbb{F}_{gt} \subseteq \mathbb{F}_a$$

and the relation necessary to obtain a precise (complete) analysis as:

$$\mathbf{C} : \mathbb{E}_{gt} \supseteq \mathbb{E}_a \wedge \mathbb{F}_{gt} \supseteq \mathbb{F}_a$$

All the tools under analysis, except for *WAVM* can generate a call graph out of the box. *WAVM* instead just generates LLVM-lifted code, and further analysis is delegated to LLVM’s built-in call graph analysis. After a call graph is generated, it is compared against the *ground truth* for soundness and precision.

7.4.2 The Benchmark Set

[Leh+23] Lehmann et al., “That’s a Tough Call: Studying the Challenges of Call Graph Construction for WebAssembly”

The set of benchmark programs of [Leh+23] aims to systematically categorize the specific challenges in Wasm call graph construction. These challenges are categorized as follows:

- **EntryPoint**: Wasm standard (e.g., `start`), and non-standard (e.g., WASI `start`) initialization functions;
- **HostCallbacks**: host functions call module functions;
- **TableIndirection**: indirect calls are involved;
- **TableMutation**: reference table is mutated by the host;
- **TableInitialState**: the host initializes the table (e.g., it controls the table offset value);
- **LowLevelTypes**: types are low-level and imprecise;
- **TableIndexValue**: table index values (e.g., emulating function pointers) are hard to identify and determine statically;
- **MemoryMutable**: linear memory is writable everywhere, even data that typically do not change during execution are potentially mutable;
- **MultiPL**: binaries are compiled from different source languages, there is no standard library.

Some challenges are self-explanatory; therefore, we will explain only the non-obvious ones in more depth. **TableInitialState** refers to statically unknown table initial state, due to an unknown initialization offset value. For instance, this offset value may be an imported global variable (see benchmark 10). **LowLevelTypes** refers to the low-level typing of Wasm offering limited help for call graph construction, as opposed to stronger types typical of object-oriented languages: an intuitive example would be C types `size_t`, `const char *`, and `int`, all compiled in Wasm as `int32`. **MultiPL** refers to the fact that Wasm is a compilation target for many different languages. Analyses that assume certain language features, e.g., *vtables*, in binaries compiled from C++ (see benchmark 24), are not applicable to all binaries.

In Table 7.1 we give an overview of all benchmarks (1 to 24), together with the respective challenges for call graph construction. Each binary can have one or more challenges.

TABLE 7.1: A summary of the features used in benchmark programs under analysis.

#	Description	Challenges
1	Simple direct call <i>direct-call-simple</i>	
2	Transitive direct call <i>direct-call-transitive</i>	
3	Direct call to imported function <i>direct-call-imported-func</i>	
4	Implicit entry point: Wasm start section <i>entry-point-start</i>	EntryPoints
5	Implicit entry point: WASI start function <i>entry-point-wasi</i>	EntryPoints
6	Imported host code calls exported function <i>host-callbacks-exports</i>	HostCallbacks
7	Functions in exported table are reachable <i>host-reachable-table-export</i>	HostCallbacks TableIndirection
8	Functions in imported table are reachable <i>host-reachable-table-import</i>	HostCallbacks TableIndirection
9	Table is mutated by host <i>host-code-table-mutable</i>	TableMutation TableIndirection
10	Table init. offset is imported from host <i>table-init-offset-imported-global</i>	TableInitialState TableIndirection
11	Memory init. offset is imported from host <i>memory-init-offset-imported-global</i>	TableIndirection
12	Functions must be in table for indirect call <i>indirect-call-func-in-table</i>	TableIndirection
13	Types can constrain indirect call targets <i>indirect-call-type-based</i>	LowLevelTypes TableIndirection
14	Constant table index value <i>indirect-call-index-expr-const</i>	TableIndexValue TableIndirection
15	Idx. value data-flow through local variable <i>indirect-call-index-expr-local</i>	TableIndexValue TableIndirection
16	Masked index value <i>indirect-call-index-expr-mask</i>	TableIndexValue TableIndirection
17	Inter-procedural index value, parameter <i>indirect-call-index-expr-interprocedural-param</i>	TableIndexValue TableIndirection
18	Inter-procedural index value, function result <i>indirect-call-index-expr-interprocedural-result</i>	TableIndexValue TableIndirection
19	Idx. from memory, constant address <i>indirect-call-index-expr-load-const-addr</i>	TableIndexValue TableIndirection
20	Idx. from memory, address inter-procedural parameter <i>indirect-call-index-expr-load-interprocedural-param</i>	TableIndexValue TableIndirection
21	Idx. from memory, address inter-procedural result <i>indirect-call-index-expr-load-interprocedural-result</i>	TableIndexValue TableIndirection
22	Idx. from memory, double indirect load <i>indirect-call-index-expr-double-load</i>	TableIndexValue TableIndirection
23	Idx. from memory, memory is mutable <i>indirect-call-index-expr-memory-mutable</i>	MemoryMutable TableIndirection
24	C++ virtual calls from unrelated classes <i>cpp-vtable-layout-source-type-info</i>	MultiPL LowLevelTypes MemoryMutable TableIndirection

7.4.3 Evaluation and Comments

The comparison between both the results obtained by *wassily*, *sturdy*, and the rest of the tools featured in [Leh+23] are shown in Table 7.2. The table shows the results on the 24 benchmark programs from [Leh+23], and whether or not they were sound (S) and/or precise (P) w.r.t. the ground truth provided in [Leh+23]. First, we focus on soundness. Our implementation provides sound results in all the test cases. On the other hand, *sturdy* is unsound in 6, 7, 9. It is unsound: in 6 because *sturdy* assumes that imported functions do not call any reachable function; in 7 because it misses that functions in an exported or imported table are host-reachable, even if the functions are not exported themselves; in 9 because the RMGC assumption does not cover external mutation of reference tables. *wassail* is unsound on 7 and 8, because it misses that

TABLE 7.2: Comparison of analysis results between *wassilly* and all other call graph construction tools. The targets are the same as Table 7.1. S represents soundness, P represents completeness.

#	wassilly		sturdy		wassail		WAVM+LLVM		MetaDCE		Twiggy	
	S	P	S	P	S	P	S	P	S	P	S	P
1	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
2	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
3	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
4	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	crash	
5	✓	✓	✓	✓	✓	✓	✓	✓	X	X	✓	✓
6	✓	X	X	✓	✓	✓	✓	✓	X	X	✓	✓
7	✓	✓	X	✓	X	X	X	X	X	X	✓	✓
8	✓	✓	crash		X	X	X	X	✓	✓	X	X
9	✓	✓	X	X	✓	X	✓	X	X	X	✓	✓
10	✓	✓	✓	✓	crash		X	X	crash		✓	X
11	✓	✓	✓	✓	✓	X	X	X	crash		X	✓
12	✓	✓	✓	✓	✓	✓	X	X	✓	✓	✓	✓
13	✓	✓	✓	✓	✓	✓	X	X	✓	X	✓	X
14	✓	✓	✓	✓	✓	X	X	X	✓	X	✓	X
15	✓	✓	✓	✓	✓	X	X	X	✓	X	✓	X
16	✓	✓	✓	✓	✓	X	X	X	✓	X	✓	X
17	✓	✓	✓	✓	✓	X	X	X	✓	X	✓	X
18	✓	✓	✓	✓	✓	X	X	X	✓	X	✓	X
19	✓	✓	✓	✓	✓	X	X	X	✓	X	✓	X
20	✓	✓	✓	✓	✓	X	X	X	✓	X	✓	X
21	✓	✓	✓	✓	✓	X	X	X	✓	X	✓	X
22	✓	✓	✓	✓	✓	X	X	X	✓	X	✓	X
23	✓	✓	✓	✓	✓	X	X	X	✓	X	✓	X
24	✓	X	✓	X	✓	X	X	X	✓	✓	✓	✓

functions in an exported or imported table are host-reachable, even if the functions are not exported themselves. *WAVM+LLVM* is unsound for the same reason, as is *MetaDCE* for 7 and *Twiggy* for 8. *MetaDCE* fails to identify the implicit WASI entry point on 5, and in benchmark 6 it assumes that an imported function can call any exported function and does not make the exported functions transitively reachable. *MetaDCE* is also unsound in 9, because it does not consider that host code can mutate an exported table. Additionally, *WAVM+LLVM* is unsound on 11-24. This is because the LLVM call graph analysis cannot handle any of the indirect calls that were generated by *WAVM*. In general, as already mentioned throughout this work, and in [Leh+23], a common source of unsoundness is WebAssembly’s interaction with the host environment, which is the main reason why we decided to endow *wassilly* with a host specification engine.

On the issue of imprecision, *wassilly* is imprecise: in 6 because we analyze it from two separate possible entry points (e.g., exported functions), whereas in the ground truth ([Leh+23] assumes an all-knowing host analysis) only one entry point is called; in 24 because the interpreter fails to prove a property stronger than $\top$, hence, we must conservatively handle the call site. We deem the first imprecision as acceptable, whereas the second one could benefit from improvement. *sturdy* is instead imprecise in 9 because it cannot capture table mutation performed by the host, and 24 for the same reason as *wassilly*. As per the rest of the analyzers, they fail to take data-flow into account (through locals and inter-procedurally) and lack pointer analysis to precisely determine the targets of indirect calls in benchmarks 14 to 23, leading to imprecise analyses. Among these analyzers, *wassail* is the only one that constrains call targets of indirect

[Leh+23] Lehmann et al., “That’s a Tough Call: Studying the Challenges of Call Graph Construction for WebAssembly”

calls by their type signature, which gives it a precise result for benchmark 13. *MetaDCE* and *Twiggy* choose a particularly conservative approach to analyzing the table section in general. If a function is in a table, it is considered reachable, even if the types are mismatched (e.g., this would result in a run-time error). They neither use the index value nor type of indirect calls to further constrain the targets of indirect calls.

We can observe that all existing analyses are limited; in particular, *WAVM+LLVM* destroys Wasm’s semantics in the LLVM translation pass, resulting in both imprecision and unsoundness in a considerable number of benchmarks. *MetaDCE* and *Twiggy* are overly conservative, and ignore information statically available in the binary, causing excessive imprecision. In comparison, *wassail* constructs better call graphs because it can leverage type information in `elem` segments. *sturdy* and *wassily* can also prove stronger properties, by keeping track of data-flow and memory facts, and can still resort to using only type-based constraints if stronger facts are not available. Overall, we can state that *wassily* manages to obtain better results w.r.t. other tools because: (i) it can leverage type-based information to restrict indirect call targets; (ii) it can statically keep track of memory and data-flow facts; (iii) it allows modeling specific host-module interactions that the RMGC assumption of *sturdy*, or the *soundy* [Liv+15] approach of *wassail* do not consider.

In general, manual intervention for static analysis is undesirable and, as such, we address how to reduce the manual burden in the next section.

7.5 FUTURE DIRECTIONS ON ANALYSIS OF WEBASSEMBLY MODULES

Requiring manual intervention to perform a static analysis represents a significant burden on usability, as writing a correct specification is not trivial, especially if the specification is a post-fixpoint of a function call, including all downstream effects stemming from the call itself. This issue begs the question: can host-module interactions be reasonably analyzed and/or verified automatically? Constraining language features that typically produce coarse analysis results is common practice in practical static analysis. One example of this would be industrial-grade analyzers not supporting recursion, as in the cases of Frama-C or Astrée [BBY17; Min13; Che20]. The proposal of RMGCs in [KM19] leverages a configurable set of guidelines comprising realistic assumptions about what a client can do while using a JavaScript library. Given that WebAssembly is, in fact, mainly used as a high-performance runtime environment for higher-level programs, the solution that [Bra+23] use is applicable if such constraints are respected. However, even the use of WASI runtime syscalls implies a violation of these rules. We demonstrate the validity of this last point by observing that in the evaluation of *cpp-vtable-layout-source-type-info*, *sturdy* gives an accidentally sound call graph. This unsoundness depends on the inability of *sturdy* to simulate WASI’s syscalls. More specifically, *args_sizes_get* and *args_get* never affect the program state the way they should. Intuitively, these functions are responsible for reading the command line arguments passed to a WebAssembly program, and the assumptions of *sturdy* about host behavior render these

[Liv+15] Livshits et al., “In defense of soundness: a manifesto”

[BBY17] Blazy et al., “Structuring Abstract Interpreters Through State and Value Abstractions”

[Min13] Miné, *Static analysis by abstract interpretation of concurrent programs. (Analyse statique par interprétation abstraite de programmes concurrents)*

[Che20] Chevalier, “Proving the security of software-intensive embedded systems by abstract interpretation. (Analyse de la sécurité de systèmes critiques embarqués à forte composante logicielle par interprétation abstraite)”

[KM19] Kristensen and Møller, “Reasonably-most-general clients for JavaScript library analysis”

[Bra+23] Brandl et al., “Modular Abstract Definitional Interpreters for WebAssembly”

function calls useless because, in fact, they modify the linear memory and return a status code. Because of the RMGC’s assumptions made by *sturdy*, and the absence of the semantics of WASI syscalls, the analysis of this benchmark results in a diverging program. This issue can be addressed by endowing the existing approaches with a library of stubs of commonly used functions. In this case, this library would cover the WASI standard and “popular” JavaScript imports. However, even if such a library were to come to life, assuming that a host will never mutate those sections deemed as immutable in the state of the module could pose functional or security risks if the absence of unwanted behavior is never verified. It may help to address those risks with a declaration and subsequent verification of “legal” effectful interactions between host and module. This kind of analysis would be similar, as a goal, to coding guideline verification (i.e., C coding standards enforcement) with the potential of being conducive to having more verifiable concerns in analysis in similar (host-module, client-library) multi-program settings. Consider the example question whether a host could modify a given resource or type of resource. This kind of analysis could both provide an answer to the question, and guarantee the soundness of the analysis in a way that is not based on unverified assumptions.

7.6 *isserious*

As already stated in [Section 6.1](#), the optimal way to analyze WebAssembly modules is influenced by how they are used. More specifically, especially in the common library-like use-case, the semantics of the host that cause *opaque* impacts on the module instance pose a threat to the soundness of the analysis. To address this, we propose a blueprint of an analysis that we are actively developing under the guise of *isserious*¹, a wordplay alluding to a high degree of synergy with *wassilly*. *isserious* is based on points-to [[Ste96](#); [AL05](#); [BS16](#)] analysis with taint [[Den76](#); [LL05](#)] and WebAssembly-specific enhancements to detect mutations of possibly Wasm-originating, *tainted*, memory locations. In this section, we lay out a sketch of said analysis, so that the reader can understand its potential impact.

CHARACTERISTICS OF *isserious* The proposed analysis aims to identify violations of the *Reasonably Most General Client* (RMGC) contract [[KM19](#)] in JavaScript programs that interact with WebAssembly modules. This section describes the essential characteristics that such an analysis must exhibit to be sound, expressive, and produce meaningful results.

BEHAVIORAL CONTRACT SENSITIVITY The analysis detects potential violations of a behavioral contract that is semantically defined rather than syntactically explicit. Specifically, it aims at identifying “illegal” mutations to instantiated WebAssembly modules. Particularly, we are interested in mutations that violate the RMGC contract and possible variants of it. As such, the analysis must be *semantics-aware* and capable of abstracting over program actions.

¹A first toy implementation is available at <https://github.com/ringzer0x00/isserious>, although we do not provide any soundness/completeness guarantees.

[Ste96] Steensgaard, “Points-to Analysis in Almost Linear Time”

[AL05] Andersen and Lee, “Program Analysis and Specialization for the C Programming Language”

[BS16] Balatsouras and Smaragdakis, “Structure-Sensitive Points-To Analysis for C and C++”

[Den76] Denning, “A Lattice Model of Secure Information Flow”

[LL05] Livshits and Lam, “Finding Security Vulnerabilities in Java Applications with Static Analysis”

[KM19] Kristensen and Møller, “Reasonably-most-general for JavaScript library analysis”

HEAP- AND OBJECT-SENSITIVITY Given that Wasm module instances can be passed as opaque objects into JavaScript, the analysis must maintain information about heap-allocated objects. It should distinguish between different allocation sites and model aliasing between JavaScript values and WebAssembly components. Object sensitivity is necessary to reasonably disambiguate between harmless interactions (e.g., method calls) and problematic ones (e.g., property writes).

MUTATION TRACKING A central feature is the tracking of mutations at specific program points. The analysis must record write operations (e.g., assignments, property updates) and correlate them to abstract heap objects. If these objects correspond to or are derived from a Wasm module, the write operation is considered a violation, according to the RMGC contract. Possible variants the contract might allow for writing to specific Wasm-originating locations.

WEBASSEMBLY-AWARE ALLOCATION SEMANTICS The analysis must identify Wasm instantiations (Wasm-origin) and model how the resulting values are allocated and propagated. This includes recognizing common allocation patterns such as `new WebAssembly.Instance(...)`. The resulting abstract objects are marked with a Wasm-origin predicate. Any abstract object reachable from a Wasm-origin allocation is also a Wasm-originating object.

7.6.1 Intuition Behind the Analysis

The intuition behind the analysis is based on the notions of *ownership and mutability* in multi-program setting. WebAssembly modules are typically treated as libraries by their JavaScript clients: initialized once and interacted with via exported functions. The RMGC contract assumes that such modules' structure should be only directly impacted by WebAssembly, assuming a stronger encapsulation mechanism that the WebAssembly runtime lacks. In fact, WebAssembly allows manipulating the export section of its instances externally. This analysis seeks to catch *direct mutations* that JavaScript code performs on these instances: for example, writing to an exported `memory` or `global`, or in general overwriting any structure reachable from a WebAssembly instance.

The analysis proceeds by:

- tracking the flow of references to Wasm-originating structures through the memory.
- monitoring write operations and identifying whether their targets originate from a Wasm instantiation.

A mutation is suspicious if the modified structure points to:

- the Wasm instance object itself, or
- an object reachable from a Wasm instantiation.

The analysis augments a points-to [Ste96; AL05; BS16] analysis with an additional origin-labeling mechanism:

- allocation sites corresponding to Wasm instantiations are labeled as `wasm-origin`.
- the taint [Den76; LL05] labels propagate through the points-to

[Den76] Denning, "A Lattice Model of Secure Information Flow"

[LL05] Livshits and Lam, "Finding Security Vulnerabilities in Java Applications with Static Analysis"

graph.

- for each write operation, the analysis checks if the target is in the transitive closure of these labeled allocations.

This approach provides a sound but practical approximation of violations of the RMGC contract.

7.6.2 Preliminary Formalism

We sketch a formal basis for this analysis. The abstract state consists of points-to information extended with origin labeling and mutation tracking and is computed as the least fixpoint of monotone transfer functions (with widening if needed).

Abstract Memory Model

Let:

- $\text{Loc}^\#$ be the (finite) set of *abstract locations* (allocation sites),
- $\text{Field}^\#$ be the (finite) set of *object fields/indices* (including a distinguished $\top$ for unknown),
- $\text{Val}^\#$ be the (finite) set of *abstract values*, abstracting primitives and references, with $\text{locs} : \text{Val}^\# \rightarrow \mathcal{P}(\text{Loc}^\#)$ extracting the (may) set of locations a value may reference,
- $A^\# \in \text{Heap}^\# : \text{Loc}^\# \rightarrow \text{Field}^\# \rightarrow \mathcal{P}(\text{Val}^\#)$ be the *abstract heap*.

We write $A^\#(\ell, f)$ for the set of abstract values at field f of object ℓ .

An abstract state is $\hat{o} \in \text{State}^\# : \text{Env}^\# \times \text{Heap}^\#$, where $\text{Env}^\#$ maps variables to $\mathcal{P}(\text{Val}^\#)$. Both $\text{Env}^\#$ and $\text{Heap}^\#$ form complete lattices under pointwise $\sqsubseteq$ (subset), with joins given by pointwise unions.

Points-to and Wasm Taints

Let $\text{isWasm}^\# : \text{Loc}^\# \rightarrow \mathbb{B}$ be a *may* predicate marking allocation sites that may correspond to WebAssembly instantiations.

From heap $A^\#$, induce a directed edge relation $E_{A^\#} \subseteq \text{Loc}^\# \times \text{Loc}^\#$:

$$(\ell, \ell') \in E_{A^\#} \iff \exists f \in \text{Field}^\#, v \in A^\#(\ell, f). \ell' \in \text{locs}(\hat{v}).$$

Define reachability as the reflexive-transitive closure

$$\rightsquigarrow_{A^\#} = (E_{A^\#})^*.$$

Define the set of tainted locations:

$$T(A^\#) = \{\ell \in \text{Loc}^\# \mid \exists \ell_0 \in \text{Loc}^\#. \text{isWasm}^\#(\ell_0) \wedge \ell_0 \rightsquigarrow_{A^\#} \ell\}.$$

(Note: by reflexivity, $\text{isWasm}^\#(\ell_0) \Rightarrow \ell_0 \in T(A^\#)$.)

Abstract Stores and Mutations

Let $\mathbb{P}$ be the set of program points and let $\mathcal{E}^\#$ be the set of abstract mutation events. The abstract semantics of a store at $p \in \mathbb{P}$ yields a (may) set of targets:

$$\text{Targets}(p, A^\#) \subseteq \text{Loc}^\# \times \text{Field}^\# \times \text{Val}^\#,$$

meaning the analysis may write value $\hat{v}$ into field $\hat{f}$ of location ℓ_t (weak update). We assume $\text{Targets}(\cdot, \cdot)$ is monotone in $A^\#$ and over-approximates all concrete store targets at p .

We record abstract mutations:

$$\text{Mut}(p, \ell_t, f, \hat{v}) \in \mathcal{E}^\# \text{ iff } (\ell_t, f, \hat{v}) \in \text{Targets}(p, A^\#).$$

We report writes *to tainted objects*:

$$\text{Viol} = \{p \in \mathbb{P} \mid \exists \ell_t, f, \hat{v}. \text{Mut}(p, \ell_t, f, \hat{v}) \wedge \ell_t \in T(A^\#)\}.$$

Summary and Design Space of the Analysis

To summarize, the analysis:

- (i) tracks allocation sites and labels *Wasm-origin* locations (may predicate);
- (ii) propagates labels via points-to reachability, i.e., the reflexive transitive closure of the heap-induced edge relation E_A ;
- (iii) detects abstract stores (mutations) and *inspects* all possible targets produced by the store semantics;
- (iv) reports program points whose targets are in $T(A)$ (writes *to* Wasm-tainted objects) as potential violations.

Policy variants. A stricter or looser policy can be chosen. Besides (iv) above (“writes *to* tainted objects”), one may instead flag “writes *of* tainted values into any object”, i.e., report whenever $\text{locs}(v) \cap T(A) \neq \emptyset$ for some assigned v . While this is not strictly relevant to the analysis of WebAssembly modules, it may be interesting from the perspective of analyzing possible information leaks from WebAssembly instances to host programs.

Precision knobs. The underlying points-to analysis is a may-analysis and can be tuned along standard axes: flow/context sensitivity; field sensitivity (including an unknown field $\top$); heap abstraction (allocation-site vs. context-sensitive); and weak vs. strong updates where sound. Algorithmically, one may choose Andersen-style (inclusion-based), Steensgaard-style (unification-based), or hybrids to balance precision and performance.

Contract flexibility. The RMGC contract can be parameterized to allow specific sanctioned mutations (e.g., direct writes to `memory` or `global`), which the checker can whitelist to avoid spurious reports. The same framework can classify WebAssembly clients as *passive* (interacting only via exports) or *active* (performing direct mutations).

Applications. Potential uses include validation of the RMGC contract, security vetting and malware detection (e.g., identifying host-module obfuscation via Wasm-origin mutations). Besides these, this analysis could be used as well in aiding, or avoiding altogether, the construction of an appropriate specification for a given host program, easing analysis burden by thinning host specifications.

Part IV

EPILOGUE

8

Conclusions

8.1 PREFACE

“From nothing, power!”
—Ethereal Lackey, Hearthstone.

This thesis explored the challenges and opportunities in performing static analysis of WebAssembly programs, focusing on call graph construction using Abstract Interpretation and environmental specification. Throughout this work, we aimed to tailor formal program analysis techniques to the semantics of WebAssembly’s runtime behavior, so as to allow for meaningful analysis in realistic host-module configurations.

8.2 REFLECTION AND LIMITATIONS

The semantics and tool developed in this thesis demonstrate strong analytical capabilities. The formal framework is sound, and its implementation, *wassilly*, as per our experimental evaluation in [Section 7.4](#), successfully analyzes all benchmark programs from [\[Leh+23\]](#), producing sound call graphs, whereas other tools considered fail to achieve soundness due to stronger host assumptions. This is mainly achieved by leveraging the specification framework embedded in *wassilly*, which is tailored to this specific gap, and provides a manual way to build tailored host assumptions. However, this specification framework introduces significant limitations, mainly in usability, in the form of a burden in specification.

[Leh+23] Lehmann et al., “That’s a Tough Call: Studying the Challenges of Call Graph Construction for WebAssembly”

Manual specification of host behavior, while powerful, can be labor-intensive. We will attempt to address this via a dedicated library of stubs targeting WASI syscalls, and the refinement of *isserious*. *isserious* has strong potential for proving that a (JavaScript) host’s specification can oftentimes be avoided altogether, making the call graph extraction of a WebAssembly module a fully automated task, as simple as, for instance, running *wassilly*.

Regarding scalability limitations, the analysis cost grows with program complexity. Although *wassilly* employs standard widening/joining strategies, the scaling nature of indirect calls still poses a bottleneck in large binaries due to the whole-program analysis approach *wassilly* uses. Specifically, each indirect call can (potentially) invoke any same-type function in the function table. This happens when the analyzer is unable to prove a *strong enough* bound as a target for the indirect call.

When this happens, the analyzer is forced to explore each new potential path, leading to complexity that scales with both the table size and the size of the analyzed program. In large binaries with many indirect calls and large function tables, this effect compounds and can become a practical bottleneck.

We note, however, that more specialized abstract domains that may be able to track, for instance, symbolic and non-convex properties (e.g., symbolic values, strided intervals), as well as ad-hoc join/widening strategies, may mitigate this bottleneck, though this hypothesis requires further investigation and is speculative at the time of writing. A statistical analysis from [Leh+23] shows that most of the indirect calls in WebAssembly depend on data flowing from linear memory. Being able to prove stronger properties about said data could therefore help both the precision and the performance of the analysis, a phenomenon that has been observed and repeatedly exploited with symbolic or refinement-based techniques [Min06; ADZ24].

[Leh+23] Lehmann et al., “That’s a Tough Call: Studying the Challenges of Call Graph Construction for WebAssembly”

[Min06] Miné, “Symbolic Methods to Enhance the Precision of Numerical Abstract Domains”

[ADZ24] Arceri et al., “Speeding up static analysis with the split operator”

8.3 SUMMARY OF CONTRIBUTIONS

We began by highlighting why WebAssembly is both promising and problematic from a program analysis standpoint. Its design favors performance and portability but introduces obstacles due to the loose coupling with host environments and its low-level indirect control flow mechanisms. Our key contributions include:

- (i) The formal (abstract and concrete) semantics of pWasm, a significant subset of WebAssembly (in [Sections 4.4](#) and [4.5](#));
- (ii) Soundness proof of the aforementioned semantics (in [Chapter 5](#));
- (iii) The implementation and validation of *wassilly* (in [Chapter 7](#)) and its host specification framework (in [Section 7.2](#));
- (iv) A first formalization of *isserious*, an analysis for host programs, aimed at strongly reducing or eliminating the host specification burden (in [Section 7.6](#)).

8.4 CLOSING THOUGHTS

Analyzing WebAssembly statically is an urgent necessity as the platform continues to mature and see widespread deployment. This work provides a principled yet pragmatic foundation for WebAssembly analysis that is both sound and context-aware. We believe it sets the stage for deeper integration between general formal methods and more specialized engineering demands.

Bibliography

- [AKS15] Nicholas Allen, Padmanabhan Krishnan, and Bernhard Scholz. “Combining type-analysis with points-to analysis for analyzing Java library source-code”. In: *Proceedings of the 4th ACM SIGPLAN International Workshop on State Of the Art in Program Analysis*. SOAP 2015. Portland, OR, USA: Association for Computing Machinery, 2015, 13–18. ISBN: 9781450335850. DOI: [10.1145/2771284.2771287](https://doi.org/10.1145/2771284.2771287). URL: <https://doi.org/10.1145/2771284.2771287> (cit. on pp. 19, 22, 92).
- [All19] Bytecode Alliance. *WebAssembly Micro Runtime*. <https://github.com/bytecodealliance/wasm-micro-runtime>. 2019 (cit. on p. 13).
- [AL05] Lars Ole Andersen and Peter Lee. “Program Analysis and Specialization for the C Programming Language”. PhD thesis. DIKU, University of Copenhagen, 2005 (cit. on pp. 8, 92, 97, 112, 113).
- [ADZ24] Vincenzo Arceri, Greta Dolcetti, and Enea Zaffanella. “Speeding up static analysis with the split operator”. In: *Int. J. Softw. Tools Technol. Transf.* 26.5 (2024), pp. 573–588. DOI: [10.1007/s10009-024-00761-2](https://doi.org/10.1007/s10009-024-00761-2). URL: <https://doi.org/10.1007/s10009-024-00761-2> (cit. on p. 120).
- [Arc+24] Vincenzo Arceri, Saverio Mattia Merenda, Greta Dolcetti, Luca Negrini, Luca Olivieri, and Enea Zaffanella. “Towards a Sound Construction of EVM Bytecode Control-Flow Graphs”. In: *Proceedings of the 26th ACM International Workshop on Formal Techniques for Java-like Programs*. FTFJP 2024. Association for Computing Machinery, 2024, 11–16. ISBN: 9798400711114. DOI: [10.1145/3678721.3686227](https://doi.org/10.1145/3678721.3686227). URL: <https://doi.org/10.1145/3678721.3686227> (cit. on p. 14).
- [BS96] David F. Bacon and Peter F. Sweeney. “Fast Static Analysis of C++ Virtual Function Calls”. In: *Proceedings of the 1996 ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages & Applications, OOPSLA 1996, San Jose, California, USA, October 6-10, 1996*. Ed. by Lougie Anderson and James Coplien. ACM, 1996, pp. 324–341. DOI: [10.1145/236337.236371](https://doi.org/10.1145/236337.236371). URL: <https://doi.org/10.1145/236337.236371> (cit. on pp. 8, 97).
- [BS16] George Balatsouras and Yannis Smaragdakis. “Structure-Sensitive Points-To Analysis for C and C++”. In: *Static Analysis - 23rd International Symposium, SAS 2016, Edinburgh, UK, September 8-10, 2016, Proceedings*. Ed. by Xavier Rival. Vol. 9837. Lecture Notes in Computer Science. Springer, 2016, pp. 84–104. DOI: [10.1007/978-3-662-53413-7_5](https://doi.org/10.1007/978-3-662-53413-7_5). URL: https://doi.org/10.1007/978-3-662-53413-7_5 (cit. on pp. 8, 97, 112, 113).
- [Bal+18] Roberto Baldoni, Emilio Coppa, Daniele Cono D’Elia, Camil Demetrescu, and Irene Finocchi. “A Survey of Symbolic Execution Techniques”. In: *ACM Comput. Surv.* 51.3 (2018), 50:1–50:39. DOI: [10.1145/3182657](https://doi.org/10.1145/3182657). URL: <https://doi.org/10.1145/3182657> (cit. on pp. 5, 8).
- [Bas+22] Iulia Bastys, Maximilian Algehed, Alexander Sjösten, and Andrei Sabelfeld. “SecWasm: Information Flow Control for WebAssembly”. In: *Program Analysis for WebAssembly (PAW)*. 2022 (cit. on p. 22).
- [Bin23] Binaryen. *Optimizer and compiler/toolchain library for WebAssembly*. <https://github.com/WebAssembly/binaryen>. 2023 (cit. on p. 21).
- [BMP24] Allan Blanchard, Claude Marché, and Virgile Prevosto. “Formally Expressing What a Program Should Do: The ACSL Language”. In: *Guide to Software Verification with Frama-C: Core Components, Usages, and Applications*. Ed. by Nikolai Kosmatov, Virgile Prevosto, and Julien Signoles. Cham: Springer International Publishing, 2024, pp. 3–80. ISBN: 978-3-031-55608-1. DOI: [10.1007/978-3-031-55608-1_1](https://doi.org/10.1007/978-3-031-55608-1_1). URL: https://doi.org/10.1007/978-3-031-55608-1_1 (cit. on pp. 20, 22).
- [BBY17] Sandrine Blazy, David Bühler, and Boris Yakobowski. “Structuring Abstract Interpreters Through State and Value Abstractions”. In: *Verification, Model Checking, and Abstract Interpretation - 18th International Conference, VMCAI 2017, Paris, France, January 15-17, 2017, Proceedings*. Ed. by Ahmed Bouajjani and David Monniaux. Vol. 10145. Lecture Notes in Computer Science. Springer, 2017, pp. 112–130. DOI: [10.1007/978-3-319-52234-0_7](https://doi.org/10.1007/978-3-319-52234-0_7). URL: https://doi.org/10.1007/978-3-319-52234-0_7 (cit. on pp. 5, 20, 111).
- [Bob+11] François Bobot, Jean-Christophe Filliâtre, Claude Marché, and Andrei Paskevich. “Why3: Shepherd Your Herd of Provers”. In: *Boogie 2011: First International Workshop on Intermediate Verification Languages*. <https://hal.inria.fr/hal-00790310>. Wrocław, Poland, 2011, pp. 53–64 (cit. on p. 6).

- [Bod+19] Martin Bodin, Philippa Gardner, Thomas P. Jensen, and Alan Schmitt. “Skeletal semantics and their interpretations”. In: *Proc. ACM Program. Lang.* 3.POPL (2019), 44:1–44:31. DOI: [10.1145/3290357](https://doi.org/10.1145/3290357). URL: <https://doi.org/10.1145/3290357> (cit. on p. 45).
- [BPR16] Marc Böhme, Van-Thuan Pham, and Abhik Roychoudhury. “Coverage-based Greybox Fuzzing as Markov Chain”. In: *ACM Conference on Computer and Communications Security (CCS)*. 2016, pp. 1032–1045 (cit. on p. 5).
- [Bou93] François Bourdoncle. “Efficient chaotic iteration strategies with widenings”. In: *Formal Methods in Programming and Their Applications, International Conference, Akademgorodok, Novosibirsk, Russia, June 28 - July 2, 1993, Proceedings*. Ed. by Dines Bjørner, Manfred Broy, and Igor V. Pottosin. Vol. 735. Lecture Notes in Computer Science. Springer, 1993, pp. 128–141. DOI: [10.1007/BFB0039704](https://doi.org/10.1007/BFB0039704). URL: <https://doi.org/10.1007/BFB0039704> (cit. on p. 45).
- [Bra+23] Katharina Brandl, Sebastian Erdweg, Sven Keidel, and Nils Hansen. “Modular Abstract Definitional Interpreters for WebAssembly”. In: *37th European Conference on Object-Oriented Programming, ECOOP 2023, July 17-21, 2023, Seattle, Washington, United States*. Ed. by Karim Ali and Guido Salvaneschi. Vol. 263. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023, 5:1–5:28. DOI: [10.4230/LIPIcs.ECOOP.2023.5](https://doi.org/10.4230/LIPIcs.ECOOP.2023.5). URL: <https://doi.org/10.4230/LIPIcs.ECOOP.2023.5> (cit. on pp. 19, 21, 22, 91, 92, 107, 111).
- [Bre+23] Florian Breitfelder, Tobias Roth, Lars Baumgärtner, and Mira Mezini. “WasmA: A Static WebAssembly Analysis Framework for Everyone”. In: *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 2023, pp. 753–757. DOI: [10.1109/SANER56733.2023.00085](https://doi.org/10.1109/SANER56733.2023.00085) (cit. on p. 21).
- [Bri+22] Tiago Brito, Pedro Lopes, Nuno Santos, and José Fragoso Santos. “Wasmati: An efficient static vulnerability scanner for WebAssembly”. In: *Computers & Security* 118 (2022), p. 102745. ISSN: 0167-4048. DOI: <https://doi.org/10.1016/j.cose.2022.102745>. URL: <https://www.sciencedirect.com/science/article/pii/S0167404822001407> (cit. on pp. 3, 21, 22).
- [Büh17] David Bühler. “Structuring an Abstract Interpreter through Value and State Abstractions:EVA, an Evolved Value Analysis for Frama-C. (Structurer un interpréteur abstrait au moyen d’abstractions de valeurs et d’états :Eva, une analyse de valeur évoluée pour Frama-C)”. PhD thesis. University of Rennes 1, France, 2017. URL: <https://tel.archives-ouvertes.fr/tel-01664726> (cit. on pp. 5, 20).
- [CDE08] Cristian Cadar, Daniel Dunbar, and Dawson Engler. “KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs”. In: *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 2008, pp. 209–224 (cit. on p. 5).
- [Che20] Marc Chevalier. “Proving the security of software-intensive embedded systems by abstract interpretation. (Analyse de la sécurité de systèmes critiques embarqués à forte composante logicielle par interprétation abstraite)”. PhD thesis. École Normale Supérieure, Paris, France, 2020. URL: <https://tel.archives-ouvertes.fr/tel-03127921> (cit. on pp. 5, 20, 111).
- [CC76] P. Cousot and R. Cousot. “Static determination of dynamic properties of programs”. In: *Proceedings of the Second International Symposium on Programming*. Dunod, Paris, France, 1976, pp. 106–130 (cit. on p. 57).
- [Cou21] Patrick Cousot. *Principles of Abstract Interpretation*. The MIT Press, 2021 (cit. on p. 23).
- [CC77] Patrick Cousot and Radhia Cousot. “Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints”. In: *Conference Record of the Fourth ACM Symposium on Principles of Programming Languages, Los Angeles, California, USA, January 1977*. Ed. by Robert M. Graham, Michael A. Harrison, and Ravi Sethi. ACM, 1977, pp. 238–252. DOI: [10.1145/512950.512973](https://doi.org/10.1145/512950.512973). URL: <https://doi.org/10.1145/512950.512973> (cit. on pp. 23, 29).
- [CC92] Patrick Cousot and Radhia Cousot. “Comparing the Galois connection and widening/narrowing approaches to abstract interpretation”. In: *Programming Language Implementation and Logic Programming*. Ed. by Maurice Bruynooghe and Martin Wirsing. Berlin, Heidelberg: Springer Berlin Heidelberg, 1992, pp. 269–295. ISBN: 978-3-540-47297-1 (cit. on p. 50).
- [Dar+17] David Darais, Nicholas Labich, Phuc C. Nguyen, and David Van Horn. “Abstracting definitional interpreters (functional pearl)”. In: *Proc. ACM Program. Lang.* 1.ICFP (2017), 12:1–12:25. DOI: [10.1145/3110256](https://doi.org/10.1145/3110256). URL: <https://doi.org/10.1145/3110256> (cit. on p. 45).

- [DGC95] Jeffrey Dean, David Grove, and Craig Chambers. “Optimization of Object-Oriented Programs Using Static Class Hierarchy Analysis”. In: *ECOOP’95 - Object-Oriented Programming, 9th European Conference, Århus, Denmark, August 7-11, 1995, Proceedings*. Ed. by Walter G. Olthoff. Vol. 952. Lecture Notes in Computer Science. Springer, 1995, pp. 77–101. DOI: [10.1007/3-540-49538-X_5](https://doi.org/10.1007/3-540-49538-X_5). URL: https://doi.org/10.1007/3-540-49538-X_5 (cit. on pp. 8, 97).
- [Den76] Dorothy E. Denning. “A Lattice Model of Secure Information Flow”. In: *Commun. ACM* 19.5 (1976), pp. 236–243. DOI: [10.1145/360051.360056](https://doi.org/10.1145/360051.360056). URL: <https://doi.org/10.1145/360051.360056> (cit. on pp. 112, 114).
- [FS20] Antonio Flores-Montoya and Eric M. Schulte. “Datalog Disassembly”. In: *29th USENIX Security Symposium, USENIX Security 2020, August 12-14, 2020*. Ed. by Srdjan Capkun and Franziska Roesner. USENIX Association, 2020, pp. 1075–1092 (cit. on p. 14).
- [GKS05] Patrice Godefroid, Nils Klarlund, and Koushik Sen. “DART: Directed Automated Random Testing”. In: *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2005, pp. 213–223 (cit. on p. 5).
- [GL02] Benjamin Grégoire and Xavier Leroy. “A compiled implementation of strong reduction”. In: *Proceedings of the Seventh ACM SIGPLAN International Conference on Functional Programming (ICFP ’02), Pittsburgh, Pennsylvania, USA, October 4-6, 2002*. Ed. by Mitchell Wand and Simon L. Peyton Jones. ACM, 2002, pp. 235–246. DOI: [10.1145/581478.581501](https://doi.org/10.1145/581478.581501). URL: <https://doi.org/10.1145/581478.581501> (cit. on p. 6).
- [Gro20] WASI Subgroup @ W3C WebAssembly Community Group. *WebAssembly System Interface - WASI*. <https://wasi.dev>. 2020 (cit. on pp. 18, 20).
- [Gro19] WebAssembly Community Group. *WebAssembly Control Instructions*. <https://webassembly.github.io/spec/core/syntax/instructions.html>. 2019 (cit. on p. 14).
- [GKN15] Arie Gurfinkel, Temesghen Kahsai, and Jorge A. Navas. “SeaHorn: A Framework for Verifying C Programs”. In: *International Conference on Fundamental Approaches to Software Engineering (FASE)*. 2015, pp. 343–350 (cit. on p. 5).
- [Haa+17] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and J. F. Bastien. “Bringing the web up to speed with WebAssembly”. In: *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2017, Barcelona, Spain, June 18-23, 2017*. Ed. by Albert Cohen and Martin T. Vechev. ACM, 2017, pp. 185–200. DOI: [10.1145/3062341.3062363](https://doi.org/10.1145/3062341.3062363). URL: <https://doi.org/10.1145/3062341.3062363> (cit. on pp. 3, 12, 13, 21).
- [He+24] Ningyu He, Zhehao Zhao, Hanqin Guan, Jikai Wang, Shuo Peng, Ding Li, Haoyu Wang, Xiangqun Chen, and Yao Guo. “SeeWasm: An Efficient and Fully-Functional Symbolic Execution Engine for WebAssembly Binaries”. In: *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis. ISSTA 2024*. New York, NY, USA: Association for Computing Machinery, 2024, 1816–1820. ISBN: 9798400706127. DOI: [10.1145/3650212.3685300](https://doi.org/10.1145/3650212.3685300). URL: <https://doi.org/10.1145/3650212.3685300> (cit. on pp. 3, 22).
- [He+23] Ningyu He, Zhehao Zhao, Jikai Wang, Yubin Hu, Shengjian Guo, Haoyu Wang, Guangtai Liang, Ding Li, Xiangqun Chen, and Yao Guo. “Eunomia: Enabling User-Specified Fine-Grained Search in Symbolically Executing WebAssembly Binaries”. In: *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Seattle, WA, USA, July 17-21, 2023*. Ed. by René Just and Gordon Fraser. ACM, 2023, pp. 385–397. DOI: [10.1145/3597926.3598064](https://doi.org/10.1145/3597926.3598064). URL: <https://doi.org/10.1145/3597926.3598064> (cit. on p. 22).
- [JM09] Bertrand Jeannet and Antoine Miné. “Apron: A Library of Numerical Abstract Domains for Static Analysis”. In: *Computer Aided Verification, 21st International Conference, CAV 2009, Grenoble, France, June 26 - July 2, 2009. Proceedings*. Ed. by Ahmed Bouajjani and Oded Maler. Vol. 5643. Lecture Notes in Computer Science. Springer, 2009, pp. 661–667. DOI: [10.1007/978-3-642-02658-4_52](https://doi.org/10.1007/978-3-642-02658-4_52). URL: https://doi.org/10.1007/978-3-642-02658-4_52 (cit. on p. 100).
- [Joh+23] Evan Johnson, Evan Laufer, Zijie Zhao, Dan Gohman, Shravan Narayan, Stefan Savage, Deian Stefan, and Fraser Brown. “WaVe: a verifiably secure WebAssembly sandboxing runtime”. In: *2023 IEEE Symposium on Security and Privacy (SP)*. 2023, pp. 2940–2955. DOI: [10.1109/SP46215.2023.10179357](https://doi.org/10.1109/SP46215.2023.10179357) (cit. on p. 21).

- [KE19] Sven Keidel and Sebastian Erdweg. “Sound and reusable components for abstract interpretation”. In: *Proc. ACM Program. Lang.* 3.OOPSLA (2019), 176:1–176:28. DOI: [10.1145/3360602](https://doi.org/10.1145/3360602). URL: <https://doi.org/10.1145/3360602> (cit. on p. 45).
- [KEH23] Sven Keidel, Sebastian Erdweg, and Tobias Hombücher. “Combinator-Based Fixpoint Algorithms for Big-Step Abstract Interpreters”. In: *Proc. ACM Program. Lang.* 7.ICFP (2023), pp. 955–981. DOI: [10.1145/3607863](https://doi.org/10.1145/3607863). URL: <https://doi.org/10.1145/3607863> (cit. on pp. 45–48, 50).
- [KM19] Erik Krogh Kristensen and Anders Møller. “Reasonably-most-general clients for JavaScript library analysis”. In: *Proceedings of the 41st International Conference on Software Engineering, ICSE 2019, Montreal, QC, Canada, May 25–31, 2019*. Ed. by Joanne M. Atlee, Tefvik Bultan, and Jon Whittle. IEEE / ACM, 2019, pp. 83–93. DOI: [10.1109/ICSE.2019.00026](https://doi.org/10.1109/ICSE.2019.00026). URL: <https://doi.org/10.1109/ICSE.2019.00026> (cit. on pp. 19, 22, 91, 92, 106, 107, 111, 112).
- [KT03] Daniel Kroening and Christoph Tautschnig. “CBMC: C Bounded Model Checker”. In: *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. 2003, pp. 389–391 (cit. on p. 5).
- [Lat02] Chris Lattner. “LLVM: An Infrastructure for Multi-Stage Optimization”. See <http://llvm.cs.uiuc.edu>. MA thesis. Urbana, IL: Computer Science Dept., University of Illinois at Urbana-Champaign, 2002 (cit. on p. 13).
- [LA04] Chris Lattner and Vikram Adve. “LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation”. In: *Proceedings of the 2004 International Symposium on Code Generation and Optimization (CGO’04)*. Palo Alto, California, 2004 (cit. on p. 13).
- [LSS24] Tom Lauwaerts, Robbert Gurdeep Singh, and Christophe Scholliers. “WARDuino: An embedded WebAssembly virtual machine”. In: *J. Comput. Lang.* 79 (2024), p. 101268. DOI: [10.1016/J.COLA.2024.101268](https://doi.org/10.1016/j.cola.2024.101268). URL: <https://doi.org/10.1016/j.cola.2024.101268> (cit. on p. 21).
- [LKP20] Daniel Lehmann, Johannes Kinder, and Michael Pradel. “Everything Old is New Again: Binary Security of WebAssembly”. In: *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, Aug. 2020, pp. 217–234. ISBN: 978-1-939133-17-5. URL: <https://www.usenix.org/conference/usenixsecurity20/presentation/lehmann> (cit. on pp. 3, 21).
- [Leh+23] Daniel Lehmann, Michelle Thalakkottur, Frank Tip, and Michael Pradel. “That’s a Tough Call: Studying the Challenges of Call Graph Construction for WebAssembly”. In: *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Seattle, WA, USA, July 17–21, 2023*. Ed. by René Just and Gordon Fraser. ACM, 2023, pp. 892–903. DOI: [10.1145/3597926.3598104](https://doi.org/10.1145/3597926.3598104). URL: <https://doi.org/10.1145/3597926.3598104> (cit. on pp. 21, 105, 107–110, 119, 120).
- [Lei10] K. Rustan M. Leino. “Dafny: An Automatic Program Verifier for Functional Correctness”. In: *Logic for Programming, Artificial Intelligence, and Reasoning - 16th International Conference, LPAR-16, Dakar, Senegal, April 25–May 1, 2010, Revised Selected Papers*. Ed. by Edmund M. Clarke and Andrei Voronkov. Vol. 6355. Lecture Notes in Computer Science. Springer, 2010, pp. 348–370. DOI: [10.1007/978-3-642-17511-4_20](https://doi.org/10.1007/978-3-642-17511-4_20). URL: https://doi.org/10.1007/978-3-642-17511-4_20 (cit. on p. 6).
- [LZK22] Zhenkai Liang, Yuhao Zhang, and Hong Jin Kang. “Machine Learning-Guided Static Analysis for Bug Detection”. In: *IEEE Transactions on Software Engineering* 48.4 (2022), pp. 1234–1250 (cit. on p. 5).
- [Liv+15] Benjamin Livshits, Manu Sridharan, Yannis Smaragdakis, Ondrej Lhoták, José Nelson Amaral, Bor-Yuh Evan Chang, Samuel Z. Guyer, Uday P. Khedker, Anders Møller, and Dimitrios Vardoulakis. “In defense of soundness: a manifesto”. In: *Commun. ACM* 58.2 (2015), pp. 44–46. DOI: [10.1145/2644805](https://doi.org/10.1145/2644805). URL: <https://doi.org/10.1145/2644805> (cit. on pp. 19, 111).
- [LL05] V. Benjamin Livshits and Monica S. Lam. “Finding Security Vulnerabilities in Java Applications with Static Analysis”. In: *Proceedings of the 14th USENIX Security Symposium, Baltimore, MD, USA, July 31 - August 5, 2005*. Ed. by Patrick D. McDaniel. USENIX Association, 2005. URL: <https://www.usenix.org/conference/14th-usenix-security-symposium/finding-security-vulnerabilities-java-applications-static> (cit. on pp. 112, 114).

- [Mar+22] Filipe Marques, José Fragoso Santos, Nuno Santos, and Pedro Adão. “Concolic Execution for WebAssembly”. In: *36th European Conference on Object-Oriented Programming (ECOOP 2022)*. Ed. by Karim Ali and Jan Vitek. Vol. 222. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022, 11:1–11:29. ISBN: 978-3-95977-225-9. DOI: [10.4230/LIPIcs.ECOOP.2022.11](https://doi.org/10.4230/LIPIcs.ECOOP.2022.11). URL: <https://drops.dagstuhl.de/opus/volltexte/2022/16239> (cit. on p. 22).
- [Min06] Antoine Miné. “Symbolic Methods to Enhance the Precision of Numerical Abstract Domains”. In: *Verification, Model Checking, and Abstract Interpretation, 7th International Conference, VMCAI 2006, Charleston, SC, USA, January 8-10, 2006, Proceedings*. Ed. by E. Allen Emerson and Kedar S. Namjoshi. Vol. 3855. Lecture Notes in Computer Science. Springer, 2006, pp. 348–363. DOI: [10.1007/11609773_23](https://doi.org/10.1007/11609773_23). URL: https://doi.org/10.1007/11609773_23 (cit. on pp. 100, 120).
- [Min13] Antoine Miné. *Static analysis by abstract interpretation of concurrent programs. (Analyse statique par interprétation abstraite de programmes concurrents)*. 2013. URL: <https://tel.archives-ouvertes.fr/tel-00903447> (cit. on pp. 5, 20, 111).
- [Min17] Antoine Miné. “Tutorial on Static Inference of Numeric Invariants by Abstract Interpretation”. In: *Found. Trends Program. Lang.* 4.3-4 (2017), pp. 120–372. DOI: [10.1561/25000000034](https://doi.org/10.1561/25000000034). URL: <https://doi.org/10.1561/25000000034> (cit. on p. 23).
- [Mon21] Raphaël Monat. “Static type and value analysis by abstract interpretation of Python programs with native C libraries. (Analyse statique, de type et de valeur, par interprétation abstraite, de programmes Python utilisant des bibliothèques C)”. PhD thesis. Sorbonne University, Paris, France, 2021. URL: <https://tel.archives-ouvertes.fr/tel-03533030> (cit. on pp. 20, 58).
- [Mos+19] Mark Mossberg, Felipe Manzano, Eric Hennenfent, Alex Groce, Gustavo Grieco, Josselin Feist, Trent Brunson, and Artem Dinaburg. “Manticore: A User-Friendly Symbolic Execution Framework for Binaries and Smart Contracts”. In: *34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019, San Diego, CA, USA, November 11-15, 2019*. IEEE, 2019, pp. 1186–1189. DOI: [10.1109/ASE.2019.00133](https://doi.org/10.1109/ASE.2019.00133). URL: <https://doi.org/10.1109/ASE.2019.00133> (cit. on pp. 5, 22).
- [Mou+15] Leonardo Mendonça de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. “The Lean Theorem Prover (System Description)”. In: *Automated Deduction - CADE-25 - 25th International Conference on Automated Deduction, Berlin, Germany, August 1-7, 2015, Proceedings*. Ed. by Amy P. Felty and Aart Middeldorp. Vol. 9195. Lecture Notes in Computer Science. Springer, 2015, pp. 378–388. DOI: [10.1007/978-3-319-21401-6_26](https://doi.org/10.1007/978-3-319-21401-6_26). URL: https://doi.org/10.1007/978-3-319-21401-6_26 (cit. on p. 6).
- [Mus+19] Marius Musch, Christian Wressnegger, Martin Johns, and Konrad Rieck. “New Kid on the Web: A Study on the Prevalence of WebAssembly in the Wild”. In: *Detection of Intrusions and Malware, and Vulnerability Assessment - 16th International Conference, DIMVA 2019, Gothenburg, Sweden, June 19-20, 2019, Proceedings*. Ed. by Roberto Perdisci, Clémentine Maurice, Giorgio Giacinto, and Magnus Almgren. Lecture Notes in Computer Science. Springer, 2019, pp. 23–42. DOI: [10.1007/978-3-030-22038-9_2](https://doi.org/10.1007/978-3-030-22038-9_2). URL: https://doi.org/10.1007/978-3-030-22038-9_2 (cit. on p. 3).
- [Mé+21] Jämes Ménétreay, Marcelo Pasin, Pascal Felber, and Valerio Schiavoni. “Twine: An Embedded Trusted Runtime for WebAssembly”. In: *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. 2021, pp. 205–216. DOI: [10.1109/ICDE51399.2021.00025](https://doi.org/10.1109/ICDE51399.2021.00025) (cit. on p. 21).
- [NS07] Nicholas Nethercote and Julian Seward. “Valgrind: a framework for heavyweight dynamic binary instrumentation”. In: *Proceedings of the ACM SIGPLAN 2007 Conference on Programming Language Design and Implementation, San Diego, California, USA, June 10-13, 2007*. Ed. by Jeanne Ferrante and Kathryn S. McKinley. ACM, 2007, pp. 89–100. DOI: [10.1145/1250734.1250746](https://doi.org/10.1145/1250734.1250746). URL: <https://doi.org/10.1145/1250734.1250746> (cit. on p. 7).
- [Nor08] Ulf Norell. “Dependently Typed Programming in Agda”. In: *Advanced Functional Programming, 6th International School, AFP 2008, Heijen, The Netherlands, May 2008, Revised Lectures*. Ed. by Pieter W. M. Koopman, Rinus Plasmeijer, and S. Doaitse Swierstra. Vol. 5832. Lecture Notes in Computer Science. Springer, 2008, pp. 230–266. DOI: [10.1007/978-3-642-04652-0_5](https://doi.org/10.1007/978-3-642-04652-0_5). URL: https://doi.org/10.1007/978-3-642-04652-0_5 (cit. on p. 6).

- [OM20] Abdelraouf Ouadjaout and Antoine Miné. “A Library Modeling Language for the Static Analysis of C Programs”. In: *Static Analysis - 27th International Symposium, SAS 2020, Virtual Event, November 18-20, 2020, Proceedings*. Ed. by David Pichardie and Mihaela Sighireanu. Vol. 12389. Lecture Notes in Computer Science. Springer, 2020, pp. 223–247. DOI: [10.1007/978-3-030-65474-0_11](https://doi.org/10.1007/978-3-030-65474-0_11). URL: https://doi.org/10.1007/978-3-030-65474-0_11 (cit. on pp. 20, 22).
- [Oz+25] Harun Oz, Güliz Seray Tuncay, Ahmet Aris, Abbas Acar, Leonardo Babun, and A. Selcuk Uluagac. “Ransomware Over Modern Web Browsers: A Novel Strain and a New Defense Mechanism”. In: *ACM Trans. Web* 19.1 (2025), 6:1–6:28. DOI: [10.1145/3708514](https://doi.org/10.1145/3708514). URL: <https://doi.org/10.1145/3708514> (cit. on p. 3).
- [PM22] Mattia Paccamiccio and Leonardo Mostarda. “Reasoning About Inter-procedural Security Requirements in IoT Applications”. In: *Advanced Information Networking and Applications - Proceedings of the 36th International Conference on Advanced Information Networking and Applications (AINA-2022), Volume 3*. Ed. by Leonard Barolli, Farookh Hussain, and Tomoya Enokido. Vol. 451. Lecture Notes in Networks and Systems. Springer, 2022, pp. 245–254. DOI: [10.1007/978-3-030-99619-2_24](https://doi.org/10.1007/978-3-030-99619-2_24). URL: https://doi.org/10.1007/978-3-030-99619-2_24 (cit. on p. ix).
- [PRL24] Mattia Paccamiccio, Franco Raimondi, and Michele Loreti. “Building call graph of WebAssembly programs via abstract semantics”. In: *Proceedings of the 8th Italian Conference on Cyber Security (ITASEC 2024), Salerno, Italy, April 8-12, 2024*. Ed. by Gianni D’Angelo, Flaminia L. Luccio, and Francesco Palmieri. Vol. 3731. CEUR Workshop Proceedings. CEUR-WS.org, 2024. URL: <https://ceur-ws.org/Vol-3731/paper30.pdf> (cit. on pp. ix, 9, 55).
- [PRL26] Mattia Paccamiccio, Franco Raimondi, and Michele Loreti. “wassilly: Spec-enhanced Call Graph Analysis for WebAssembly”. In: *Under Revision*. 2026 (cit. on pp. ix, 9, 55, 97).
- [Pau93] Lawrence C. Paulson. *Isabelle’s Object-Logics*. Technical Report 286. Updated version maintained as part of Isabelle documentation. University of Cambridge, Computer Laboratory, 1993. URL: <https://isabelle.in.tum.de/dist/Isabelle2025/doc/logics.pdf> (visited on 06/20/2024) (cit. on p. 6).
- [Ric53] H. G. Rice. “Classes of Recursively Enumerable Sets and Their Decision Problems”. In: *Trans. Amer. Math. Soc.* 74 (1953), pp. 358–366 (cit. on p. 3).
- [Rin+07] N. Rinetzkzy, A. Poetzsch-Heffter, G. Ramalingam, M. Sagiv, and E. Yahav. “Modular Shape Analysis for Dynamically Encapsulated Programs”. In: *Programming Languages and Systems*. Ed. by Rocco De Nicola. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 220–236. ISBN: 978-3-540-71316-6 (cit. on pp. 19, 22, 92).
- [RY20] Xavier Rival and Kwangkeun Yi. *Introduction to Static Analysis - An Abstract Interpretation Perspective*. The MIT Press, 2020 (cit. on p. 23).
- [Sch95] David A. Schmidt. “Natural-Semantics-Based Abstract Interpretation (Preliminary Version)”. In: *Static Analysis, Second International Symposium, SAS’95, Glasgow, UK, September 25-27, 1995, Proceedings*. Ed. by Alan Mycroft. Vol. 983. Lecture Notes in Computer Science. Springer, 1995, pp. 1–18. DOI: [10.1007/3-540-60360-3_28](https://doi.org/10.1007/3-540-60360-3_28). URL: https://doi.org/10.1007/3-540-60360-3_28 (cit. on p. 45).
- [Sho+16] Yan Shoshitaishvili, Ruoyu Wang, Christopher Salls, Nick Stephens, Mario Polino, Audrey Dutcher, John Grosen, Siji Feng, Christophe Hauser, Christopher Kruegel, and Giovanni Vigna. “SoK: (State of) The Art of War: Offensive Techniques in Binary Analysis”. In: *IEEE Symposium on Security and Privacy*. 2016 (cit. on p. 5).
- [SMR24] Lucas Silva, José Carlos Metrôlho, and Fernando Reinaldo Ribeiro. “Efficient Data Exchange between WebAssembly Modules”. In: *Future Internet* 16.9 (2024), p. 341. DOI: [10.3390/fi16090341](https://doi.org/10.3390/fi16090341). URL: <https://doi.org/10.3390/fi16090341> (cit. on p. 21).
- [SM21] Benedikt Spies and Markus Mock. “An Evaluation of WebAssembly in Non-Web Environments”. In: *2021 XLVII Latin American Computing Conference (CLEI)*. 2021, pp. 1–10. DOI: [10.1109/CLEI53233.2021.9640153](https://doi.org/10.1109/CLEI53233.2021.9640153) (cit. on p. 21).
- [Ste96] Bjarne Steensgaard. “Points-to Analysis in Almost Linear Time”. In: *Conference Record of POPL’96: The 23rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Papers Presented at the Symposium, St. Petersburg Beach, Florida, USA, January 21-24, 1996*. Ed. by Hans-Juergen Boehm and Guy L. Steele Jr. ACM Press, 1996, pp. 32–41. DOI: [10.1145/237721.237727](https://doi.org/10.1145/237721.237727). URL: <https://doi.org/10.1145/237721.237727> (cit. on pp. 8, 92, 97, 112, 113).

- [SDR20] Quentin Stiévenart and Coen De Roover. “Compositional information flow analysis for webassembly programs”. In: *2020 IEEE 20th International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 2020, pp. 13–24 (cit. on pp. 19, 21, 22).
- [SDR21] Quentin Stiévenart and Coen De Roover. “Wassail: a webassembly static analysis library”. In: *Fifth International Workshop on Programming Technology for the Future Web*. 2021 (cit. on pp. 19, 21).
- [Tar55] Alfred Tarski. “A lattice-theoretical fixpoint theorem and its applications.” In: *Pacific Journal of Mathematics* (1955), pp. 285–309 (cit. on p. 50).
- [Tek+21] Ege Tekiner, Abbas Acar, A. Selcuk Uluagac, Engin Kirda, and Ali Aydin Selçuk. “SoK: Cryptojacking Malware”. In: *IEEE European Symposium on Security and Privacy, EuroS&P 2021, Vienna, Austria, September 6-10, 2021*. IEEE, 2021, pp. 120–139. DOI: 10.1109/EUROSP51992.2021.00019. URL: <https://doi.org/10.1109/EuroSP51992.2021.00019> (cit. on p. 3).
- [TP00] Frank Tip and Jens Palsberg. “Scalable propagation-based call graph construction algorithms”. In: *Proceedings of the 2000 ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages & Applications, OOPSLA 2000, Minneapolis, Minnesota, USA, October 15-19, 2000*. Ed. by Mary Beth Rosson and Doug Lea. ACM, 2000, pp. 281–293. DOI: 10.1145/353171.353190. URL: <https://doi.org/10.1145/353171.353190> (cit. on pp. 8, 97).
- [Tur36] Alan M. Turing. “On Computable Numbers, with an Application to the Entscheidungsproblem”. In: *Proceedings of the London Mathematical Society* 2.42 (1936), pp. 230–265. URL: <http://www.cs.helsinki.fi/u/gionis/cc05/OnComputableNumbers.pdf> (cit. on p. 3).
- [Twi22] Twiggy. *A code size profiler for Wasm*. <https://rustwasm.github.io/twiggy/>. 2022 (cit. on p. 21).
- [WKD22] Stefan Wallentowitz, Bastian Kersting, and Dan Mihai Dumitriu. “Potential of WebAssembly for Embedded Systems”. In: *2022 11th Mediterranean Conference on Embedded Computing (MECO)*. 2022, pp. 1–4. DOI: 10.1109/MECO55406.2022.9797106 (cit. on p. 21).
- [Wat18] Conrad Watt. “Mechanising and verifying the WebAssembly specification”. In: *Proceedings of the 7th ACM SIGPLAN International Conference on certified programs and proofs*. 2018, pp. 53–65 (cit. on p. 21).
- [WRPP19] Conrad Watt, Andreas Rossberg, and Jean Pichon-Pharabod. “Weakening WebAssembly”. In: *Proc. ACM Program. Lang.* 3.OOPSLA (Oct. 2019). DOI: 10.1145/3360559. URL: <https://doi.org/10.1145/3360559> (cit. on p. 21).
- [WAVnd] WAVM. *WebAssembly virtual machine, designed for use in non-browser applications*. <https://wasm.github.io/>. n.d. (Cit. on p. 21).
- [WCR19] Guannan Wei, Yuxuan Chen, and Tiark Rompf. “Staged abstract interpreters: fast and modular whole-program analysis via meta-programming”. In: *Proc. ACM Program. Lang.* 3.OOPSLA (2019), 126:1–126:32. DOI: 10.1145/3360552. URL: <https://doi.org/10.1145/3360552> (cit. on p. 45).
- [Zak11] Zakai et al. *Emscripten - Complete compiler toolchain to WebAssembly*. <https://emscripten.org/>. 2011 (cit. on p. 12).